



AI-POWERED LEARNING PLATFORM

Personalized Learning, Powered by AI

Structured courses, an AI mentor, adaptive roadmaps, an in-browser coding IDE, gamified progress and verifiable certificates — in one unified ecosystem.

VERSION

1.4.0

STACK

React · TS · Vite · Supabase

AI

Gemini 2.5 Flash

DATE

June 2026

Technical & Product Documentation

LearnLoom

AI-Powered Personalized Learning Platform

Technical & Product Documentation

Project	LearnLoom — AI-Powered Personalized Learning Platform
Repository	https://github.com/shaikbashe1/Learn-Loom
Live Demo	https://learn-loom-indol.vercel.app
Developer	Shaik Bashe — GitHub @shaikbashe1
Product Version	1.4.0 (per CHANGELOG.md ; npm package version 0.0.1)
Document Date	June 2026
Document Status	Repository-verified technical documentation

About this document. Every technical statement below was verified directly against the LearnLoom source repository (file structure, React components, route table, Supabase SQL migrations, Edge Functions and Vercel API routes). Where a capability exists in code but its live behaviour could not be independently confirmed from this environment, that is stated explicitly rather than assumed. Key screens are **live captures from the deployed application**; the remaining screens are **brand-accurate reference frames** that mark the exact route and purpose of each screen and can be swapped for live captures — see the Screenshots section.

Table of Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Solution Overview
- 4. Objectives
- 5. Feature Documentation - 5.1 Landing Page - 5.2 Authentication - 5.3 Student Dashboard - 5.4 Course Explorer & Catalog - 5.5 Course Player & Strict Progression - 5.6 AI Mentor - 5.7 Personalized AI Roadmap - 5.8 Coding Practice IDE - 5.9 Assessments, Quizzes & Certification - 5.10 Assignments - 5.11 Certificates & Public Verification - 5.12 Community Forum - 5.13 Leaderboard, Gamification, XP & Credits - 5.14 Student Profile & Progress Tracking - 5.15 Notifications - 5.16 Search - 5.17 Subscriptions & Payments - 5.18 Responsive Design - 5.19 Admin Console
- 6. System Architecture
- 7. Technology Stack
- 8. AI Architecture
- 9. Database Design
- 10. User Flows
- 11. Security
- 12. Performance
- 13. Accessibility
- 14. Deployment
- 15. Installation Guide
- 16. API Documentation
- 17. Challenges Faced
- 18. Future Enhancements
- 19. Impact
- 20. Conclusion
- 21. References
- 22. Screenshots Gallery

1. Executive Summary

LearnLoom is a production-oriented EdTech platform that unifies structured courses, an AI mentor, an adaptive learning-roadmap generator, an in-browser coding environment, gamified progress and verifiable certification inside a single application. It is built as a modern single-page application using React 18, TypeScript and Vite on the front end, with Supabase (PostgreSQL, Auth, Storage, Realtime and Edge Functions) as the backend, and Google's Gemini 2.5 Flash model providing the intelligence behind the mentor, the roadmap generator and the course-generation tooling.

What it is. Rather than being a single-purpose app — a video catalogue, *or* a quiz engine, *or* a coding playground — LearnLoom stitches these capabilities together so a learner can move from "I want to become a backend engineer" to a phased roadmap, to enrolling in courses, to practising code, to passing a gated assessment, to holding a publicly verifiable certificate, without ever leaving the platform. The route table in `src/routes.tsx` confirms this breadth: public auth routes, a student area of roughly twenty screens, and a dedicated admin console.

Why it was built. Online learning today is abundant but unstructured. Learners face a paradox of choice, little personalization, weak feedback loops and no credible proof of what they have actually learned. LearnLoom's design — strict linear module progression, an assessment gate before any certificate is issued, and an AI layer that personalizes both guidance and pathways — is a direct response to those gaps.

Who it helps. The repository models two first-class roles in its `user_role` enum (`user` and `admin` , later extended with role splitting in the migrations):

- **Students / self-directed learners** who want a guided, end-to-end path from goal to certification, with on-demand mentoring and hands-on practice.
- **Administrators / content owners** who publish and moderate courses, manage students, oversee certificates and community content, and read platform analytics.

Expected impact. By collapsing several tools into one coherent loop and adding AI personalization on top, LearnLoom aims to raise completion rates, shorten the path from intention to competence, and make learning outcomes *verifiable* — for the learner, for an employer checking a certificate code, and for an institution running the platform.

At a glance	Detail (verified in repository)
Application type	Client-rendered SPA (Vite build, <code>dist/</code> static output)
Front end	React 18, TypeScript, React Router v7, Tailwind CSS, Radix UI (shadcn/ui "new-york"), Lucide
Backend / data	Supabase — PostgreSQL with Row-Level Security, Auth, Storage, Realtime, Edge Functions
AI model	Google Gemini 2.5 Flash (via Vercel Edge routes and Supabase Edge Functions)
Code execution	Piston engine (proxied), with Judge0 referenced as an alternative
Payments	Razorpay (order creation + verification Edge Functions)
Primary hosting	Vercel (static SPA + Edge API routes + daily cron)
Distinct student screens	~20 routes (see <code>src/routes.tsx</code>)
Admin screens	9+ routes under <code>/admin</code>
Database migrations	48 SQL migration files under <code>supabase/migrations/</code>

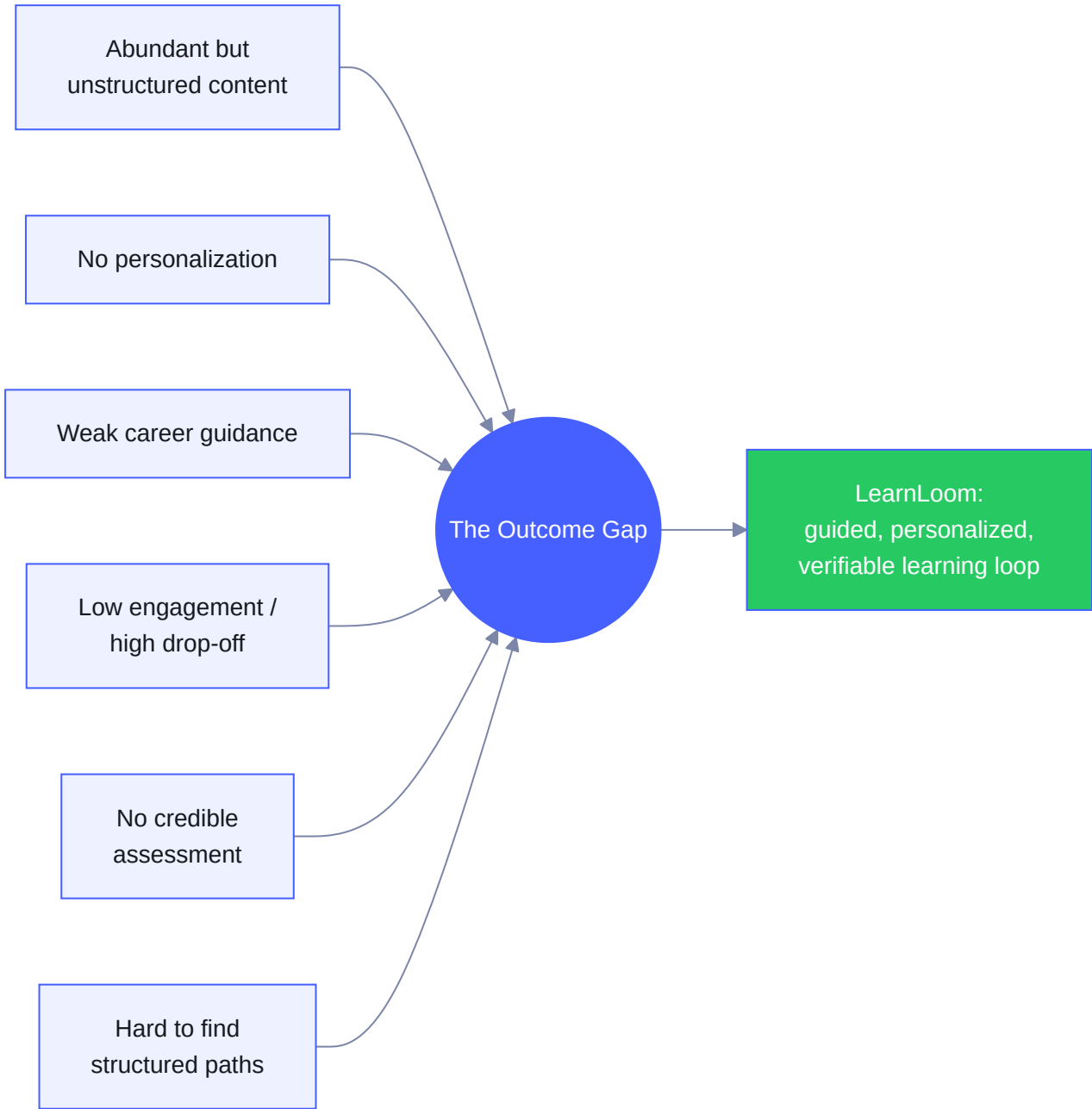
2. Problem Statement

Online education has solved *access* but not *outcomes*. The following challenges — each of which LearnLoom’s architecture explicitly targets — define the problem space.

- 1. The structure gap.** Free and paid content exists in enormous quantity, but it is scattered across platforms with no coherent ordering. A learner rarely knows what to study first, what depends on what, or when they are "done." LearnLoom answers this with **strict linear module progression** (`user_module_progress` with a `locked / unlocked / completed` status enum) so a course is consumed in a deliberate sequence rather than as a pile of videos.
- 2. Lack of personalization.** Most catalogues show everyone the same list. They do not adapt to a learner’s goal, current level or target role. LearnLoom introduces a **personalized roadmap generator** that takes a domain, a target role and a difficulty level and produces a phased plan, and an **AI mentor** that answers in the learner’s own context.
- 3. Poor career guidance.** Learners frequently cannot translate an ambition ("become a data analyst") into concrete next steps. The roadmap generator (`api/ai-roadmap.ts`) is prompted specifically as an *"expert EdTech curriculum designer"* that maps a domain and role into 3–6 phases with topics and assignments and an estimated number of weeks.
- 4. Low engagement and high drop-off.** Without feedback or motivation, learners disengage. LearnLoom layers **gamification** — a credits economy, daily streaks, a leaderboard, and a community forum — so progress is visible, social and rewarded (e.g. credits awarded on module

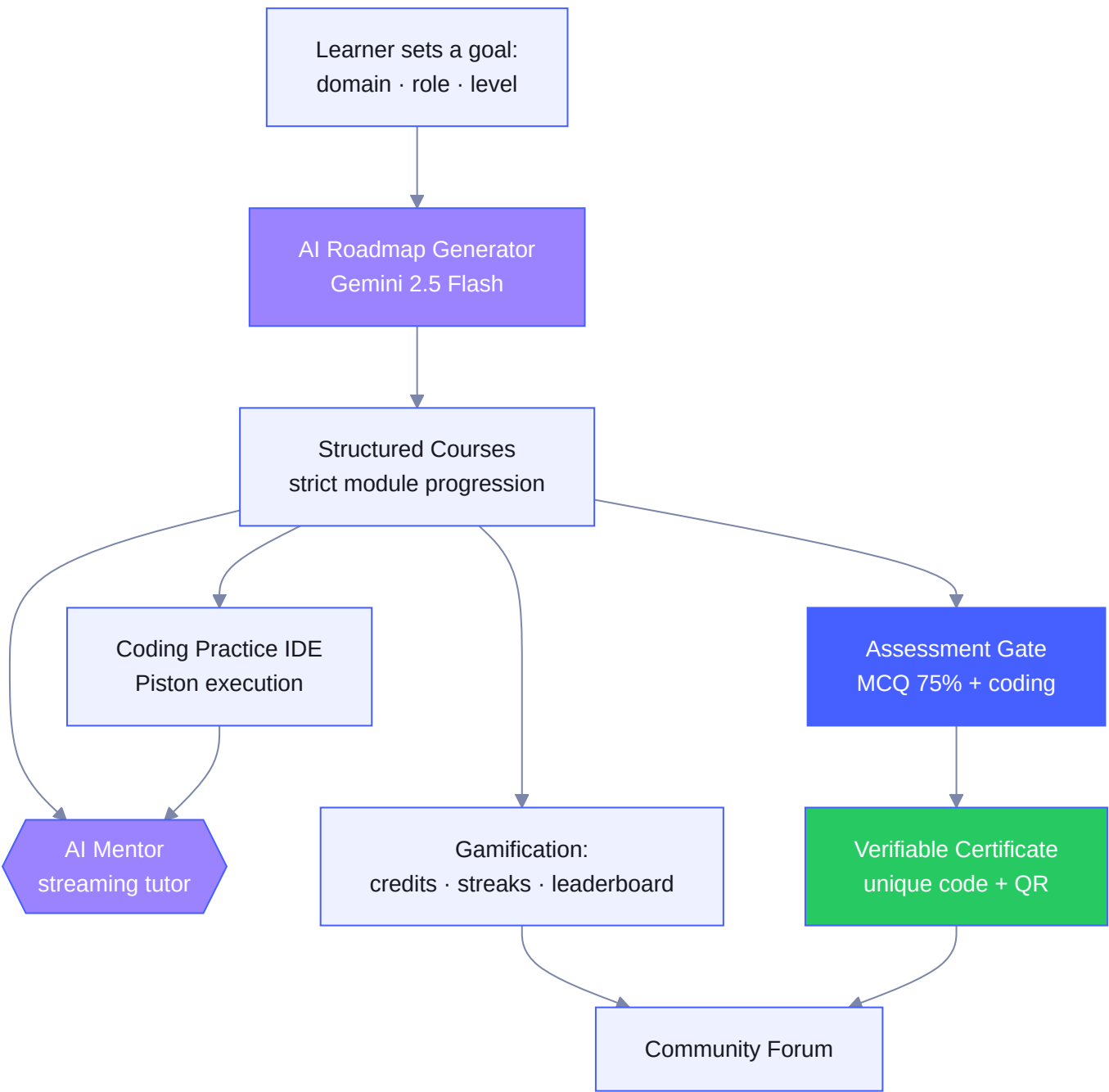
completion and on passing the Grand Test, via database triggers).

- 5. No adaptive learning or credible assessment.** Many platforms hand out completion badges for merely watching content. LearnLoom only issues a certificate **after an assessment gate**: reaching 100% progress unlocks a final MCQ exam (75% pass mark) and, when a course contains coding questions, a hands-on coding assessment as well.
- 6. Difficulty finding structured learning paths.** Even motivated learners struggle to assemble a path themselves. By combining the catalogue, the roadmap generator and the strict progression engine, LearnLoom turns "where do I even start?" into a guided, trackable journey.



3. Solution Overview

LearnLoom's solution is best understood as a **single learning loop** in which AI is woven into each stage rather than bolted on as a chatbot. The same platform that recommends a path also delivers the courses, mentors the learner through difficulty, verifies competence and certifies the result.



How AI solves each problem:

- **Personalization** → the *Personalized AI Roadmap* converts a free-text goal into a phased, week-estimated plan, and the *AI Mentor* tailors explanations to the learner's questions.
- **Structure** → generated roadmaps and the strict progression engine give every learner a concrete, ordered path; AI course/module generators (`api/ai-course-generator.ts` , `api/ai-module-generator.ts`) help administrators produce well-structured content quickly.
- **Engagement** → credits, streaks, the leaderboard and the forum turn solitary study into a motivated, social activity.

- **Credibility** → the assessment gate plus cryptographically-unique certificate verification codes make outcomes provable rather than assumed.

The result is not a collection of features but a *closed loop*: goal → path → learning → practice → assessment → proof → community, with the AI layer reducing friction at every hop.

4. Objectives

4.1 Functional objectives

- Deliver structured, sequential courses with video, rich notes, quizzes, assignments and coding questions, governed by a strict **locked → unlocked → completed** progression model.
- Provide an **AI Mentor** that answers learning questions in real time via token streaming.
- Generate **personalized roadmaps** on demand from a domain, target role and difficulty.
- Offer an **in-browser coding IDE** with multi-language execution and automated test cases.
- Enforce a rigorous **certification pipeline** (progress → MCQ gate → optional coding gate → verified certificate) with public verification.
- Support a full **admin console** for courses, students, certificates, community, roadmaps, submissions and analytics.

4.2 Business objectives

- Operate as a **self-hostable EdTech SaaS**: the README positions LearnLoom as deployable on a user's own Supabase project and any static host (Vercel, Render, Nginx).
- Monetize through **tiered subscriptions** (Free / Pro / Premium plans seeded in **subscription_plans**) with monthly credit allowances, integrated with **Razorpay**.
- Keep infrastructure costs low by shipping a **static SPA** plus serverless Edge functions, avoiding a heavyweight always-on backend.

4.3 Educational objectives

- Increase **course completion** through guided progression and visible momentum.
- Make learning **outcome-based** rather than consumption-based via the assessment gate.
- Provide **employable proof** of skills through verifiable certificates.
- Lower the barrier to *starting* by removing the "what do I learn next?" decision.

4.4 Future objectives

The repository and roadmap point toward an expanding career-readiness suite — an AI interview coach, résumé analysis (the schema already contains **resumes** and **resume_analysis** tables), richer analytics, multilingual support and a mobile experience. These are detailed in Future Enhancements.

5. Feature Documentation

This section documents each major feature. For every feature the same lens is applied: **purpose**, **workflow**, **user interaction**, **business value**, **technical implementation (from the repository)**, **advantages and future improvements**. Each feature is paired with a reference frame marking the exact route it lives on.

5.1 Landing Page

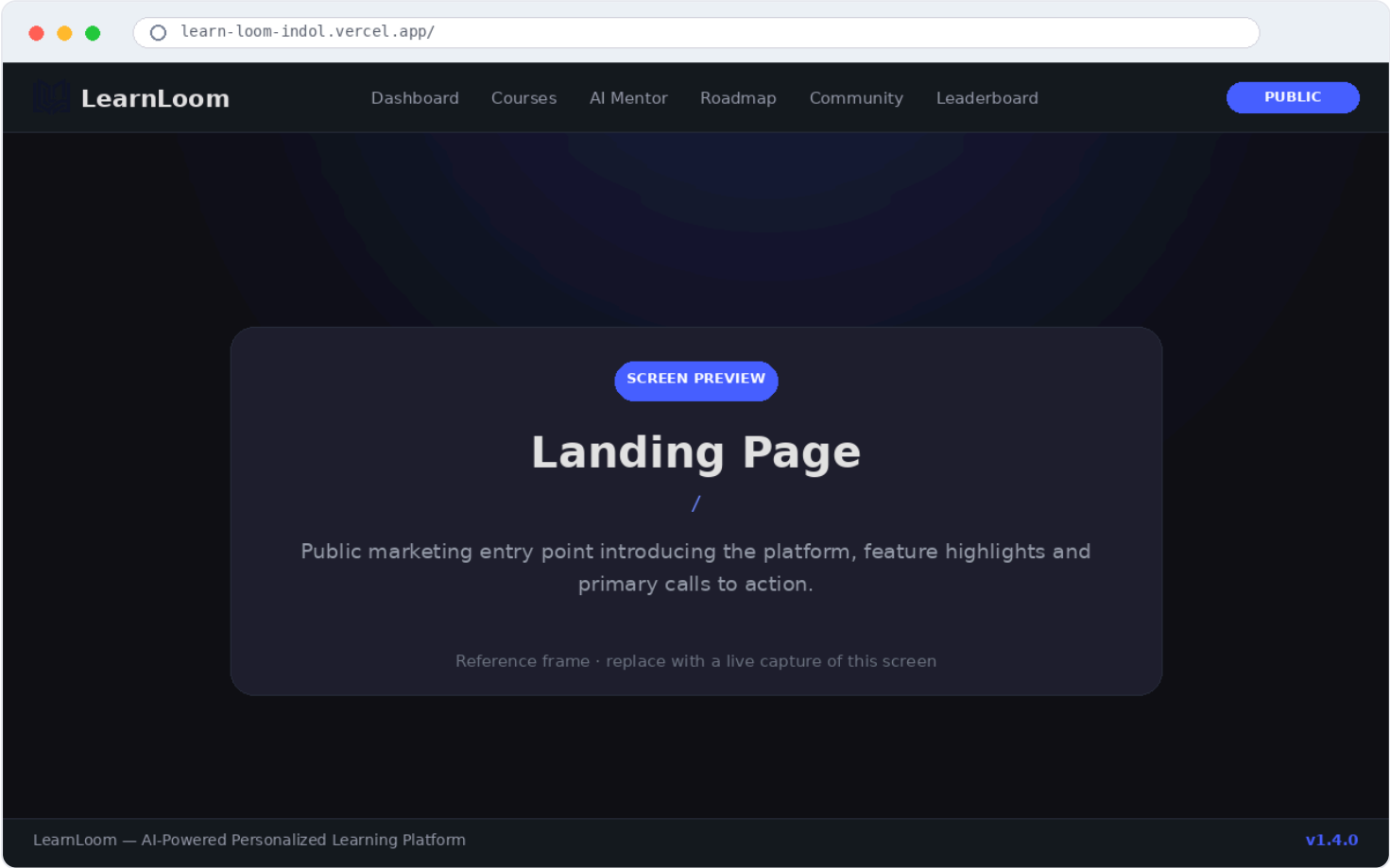


Figure 5.1 — Landing page (/). The public entry point that introduces the platform and routes visitors to sign-up or login. Reference frame; replace with a live capture.

- Purpose.** Convert a first-time visitor into a registered learner by communicating what LearnLoom is and offering clear calls to action.
- Workflow.** The landing page is the application's index route (`{ path: '/', component: LandingPage, public: true }` in `src/routes.tsx`) and is a *public* route — `RouteGuard` allows unauthenticated access. Its presentation lives in `src/pages/LandingPage.tsx` and a dedicated `src/components/landing/` directory.
- User interaction.** Visitors browse the value proposition and feature highlights and choose *Login* or *Sign up*. Authenticated users who return are redirected by `RouteGuard` to their dashboard (or the admin console for the authorized admin account).
- Business value.** As the top of the funnel, the landing page frames the product for hackathon judges, investors and prospective users in a single screen.
- Technical implementation.** Lazy-loaded via `React.lazy` and rendered inside the app's `Suspense / ErrorBoundary` shell (`src/App.tsx`). Static, public, and SEO-friendly through `react-helmet-async` (a declared dependency) for per-page metadata.
- Advantages.** Zero-auth entry, fast first paint (code-split bundle), and consistent theming with the rest of the app's glassmorphism design system.

Future improvements. Add live social proof (learner counts, certificate totals from the database), interactive feature demos, and localized copy for multilingual audiences.

5.2 Authentication

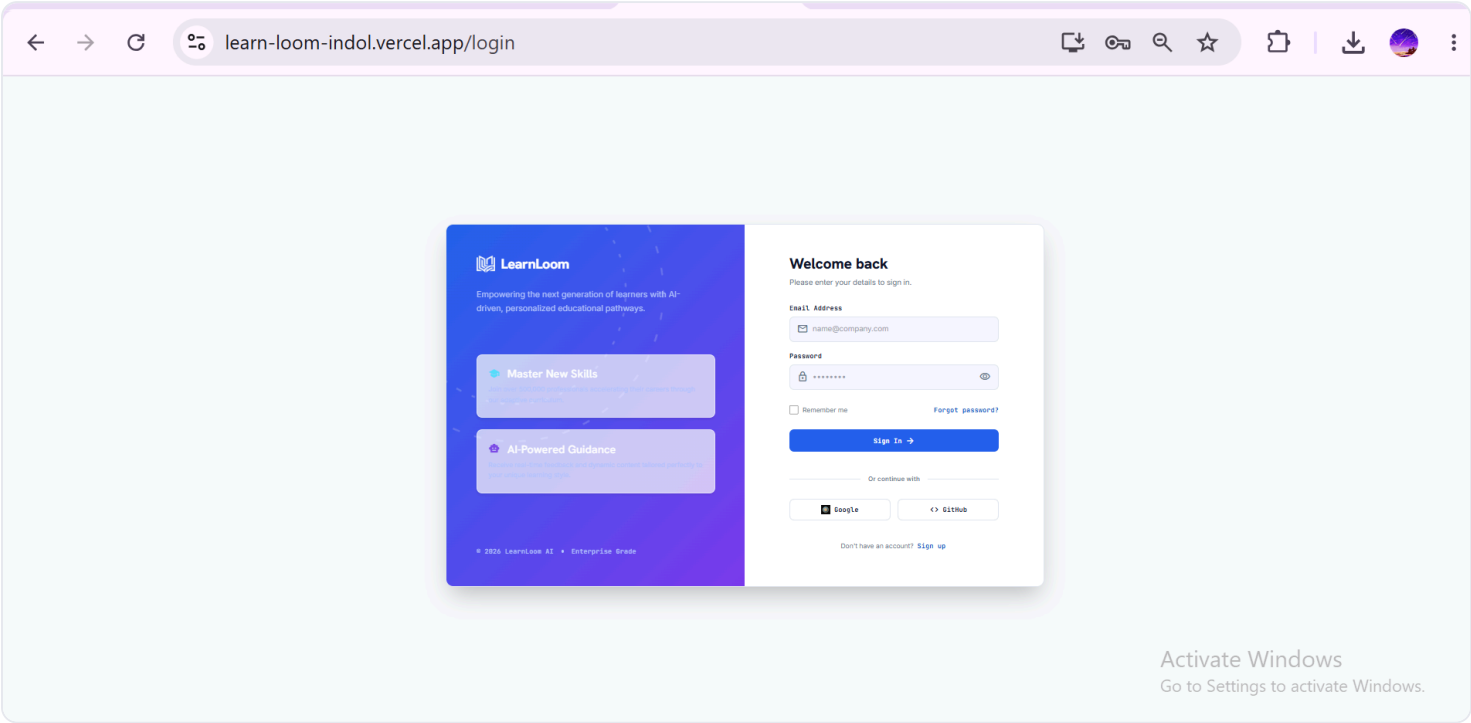


Figure 5.2 — Login (`/login`) with email/password plus Continue-with-Google and GitHub. Live capture from the deployed app.

LearnLoom's authentication is built on **Supabase Auth** and exposed through a single `AuthContext` (`src/contexts/AuthContext.tsx`) that the whole app consumes via a `useAuth()` hook. It supports three sign-in methods.

5.2.1 Email authentication

Purpose & workflow. New users sign up with email, password and full name; Supabase issues a verification step before the account is usable. `AuthContext` exposes `signUpWithEmail`, `signInWithEmail`, `verifyEmailCode` and `resendVerificationEmail`, and the route table includes `/signup`, `/login`, `/verify-email`, `/forgot-password` and `/reset-password`.

Technical implementation. On sign-up, Supabase inserts a row into `auth.users`; a `SECURITY DEFINER` trigger `handle_new_user()` (migration `00001`) then auto-creates a matching row in `public.profiles`, copying `email`, `full_name` and `avatar_url` from the auth metadata and defaulting `role` to `user`. Because that profile row is created asynchronously, the client `getProfile()` helper **polls up to 5 times at 500 ms intervals** to avoid a race on first login — a resilience detail visible directly in `AuthContext.tsx`.

5.2.2 Google login

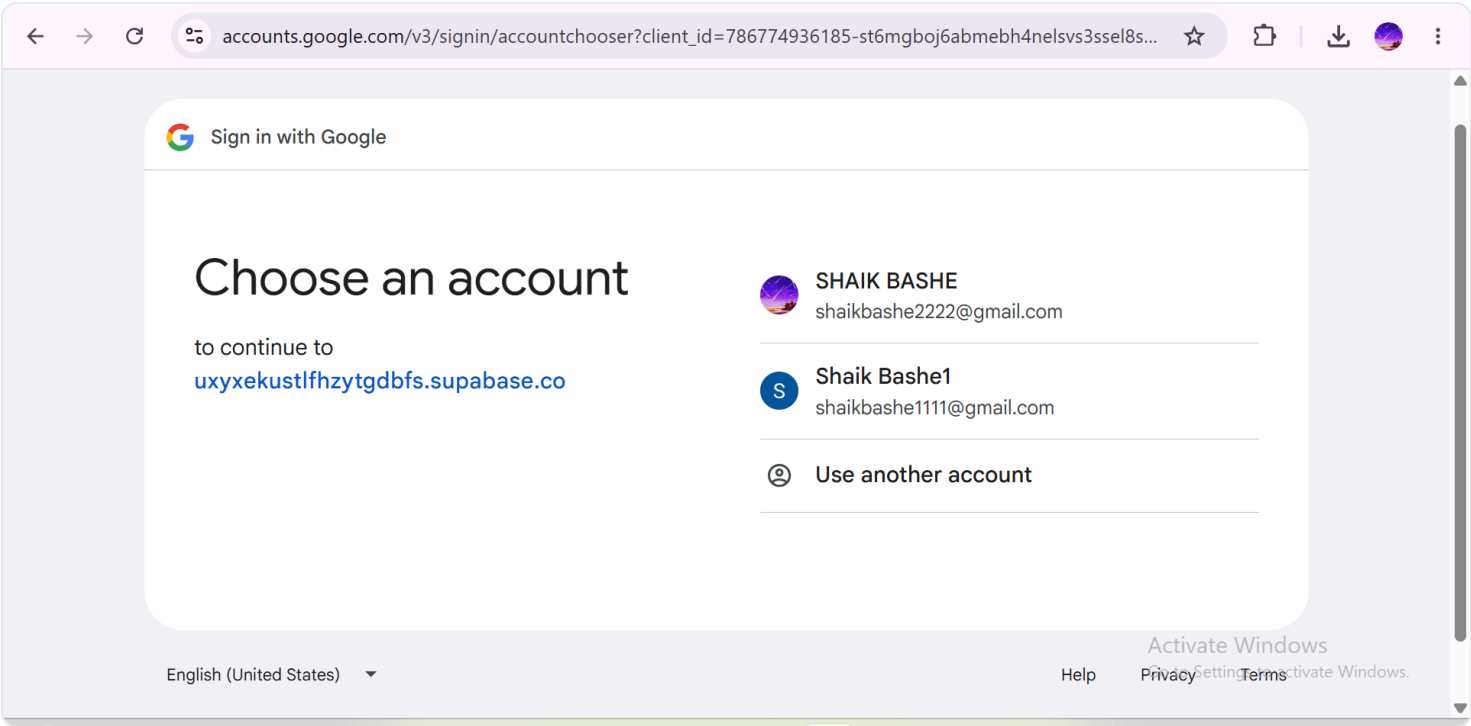


Figure 5.3 — Google OAuth flow returning to `/auth/callback`. Live capture from the deployed app.

Workflow. `signInWithGoogle()` calls `supabase.auth.signInWithOAuth({ provider: 'google', options: { redirectTo: <origin>/auth/callback } })`. After consent, Supabase redirects back to the `AuthCallbackPage`, which resolves the session and routes the user onward. The README documents the full Google Cloud Console + Supabase provider configuration required to enable it.

Resilience. The README and `AuthContext` describe retry-with-polling on the callback plus a 10-second safety timeout so a slow profile-creation trigger cannot leave the user stuck on an infinite loader.

5.2.3 GitHub login

Workflow. `signInWithGitHub()` mirrors the Google flow with `provider: 'github'` and the same `/auth/callback` redirect. *Note (honesty):* the GitHub method is fully implemented in the client; whether it is enabled in the live Supabase project depends on provider configuration, which cannot be confirmed from the repository alone.

Observed detail. The repository also contains Clerk artifacts — a `clerk-webhook` Edge Function, a `clerk_id` column (migration `00025`) and Clerk JWKS verification inside the `execute-code` Edge Function. Primary authentication is Supabase Auth; these Clerk elements appear to be a partial/legacy integration and are reported here as-is rather than presented as the active path.

Business value (all methods). OAuth lowers sign-up friction dramatically, while email verification keeps the user base genuine. A single `AuthContext` means every protected feature shares one consistent session and profile.

Advantages. One auth surface; automatic profile provisioning via DB trigger; resilient OAuth callbacks; role available everywhere through `profile.role`.

Future improvements. Add more providers (Microsoft, LinkedIn), magic-link sign-in, and multi-factor authentication for admin accounts.

5.3 Student Dashboard

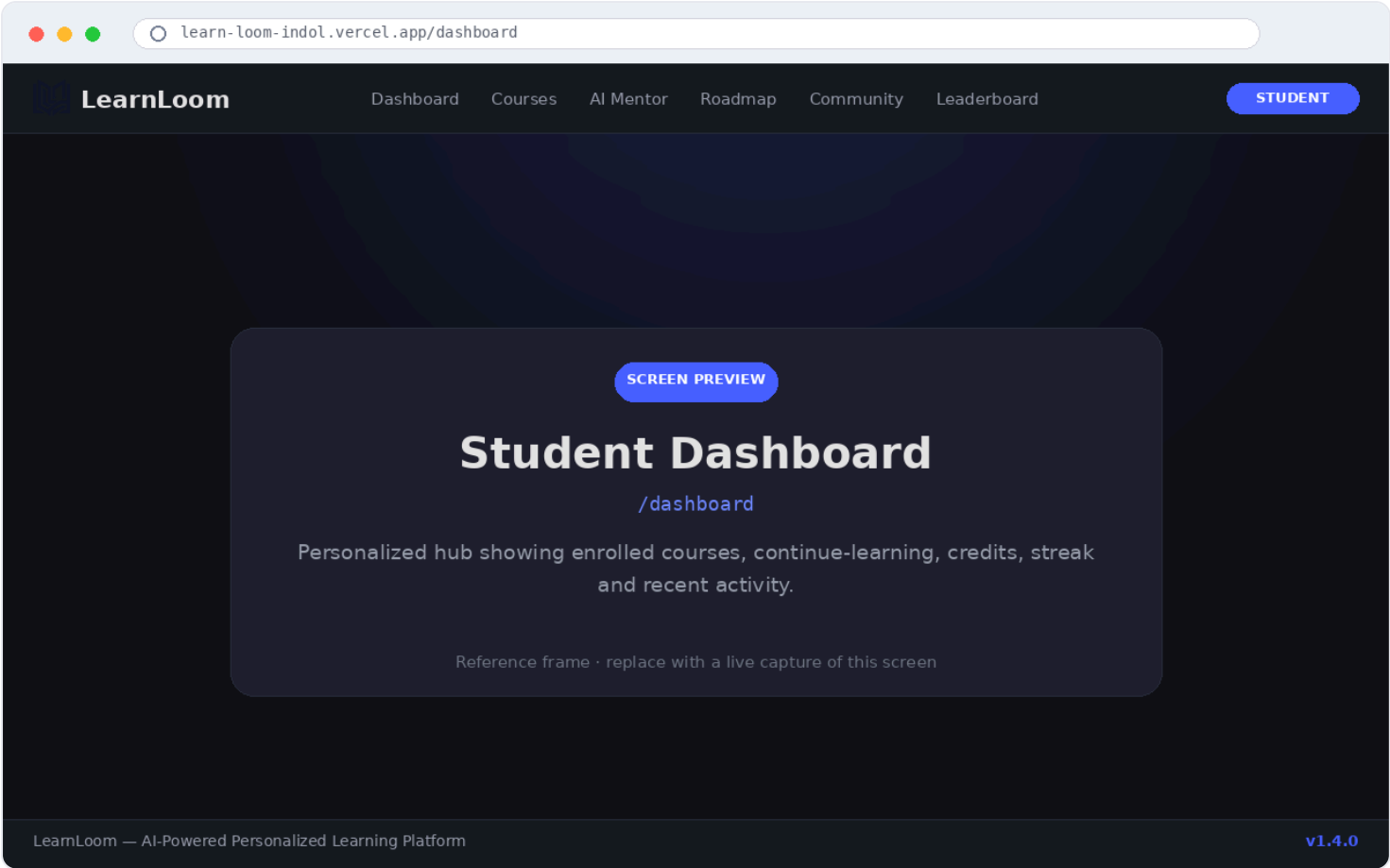


Figure 5.4 — Student dashboard (`/dashboard`): the personalized home for a logged-in learner. Reference frame.

Purpose. Give each learner a personalized home that surfaces what to do next.

Workflow & user interaction. After login, `RouteGuard` sends students to `/dashboard` (`StudentDashboard`). The page aggregates the learner's enrolled courses, a continue-learning entry point, gamification stats (credits, streak) and recent activity, then links out to courses, the mentor, the roadmap, the community and the leaderboard.

Business value. A strong dashboard is the retention engine — it reduces the "what now?" moment to a single glance and keeps the learning loop turning.

Technical implementation. Reads from `user_course_enrollments` (progress %, `last_module_id` , `completed_at`), `profiles` (credits, streak) and activity tables, all protected by RLS so a learner only ever sees their own rows.

Advantages. Personalized, data-driven, and the natural hub linking every other feature.

Future improvements. Predictive "next best action," spaced-repetition reminders and a weekly progress digest.

5.4 Course Explorer & Catalog

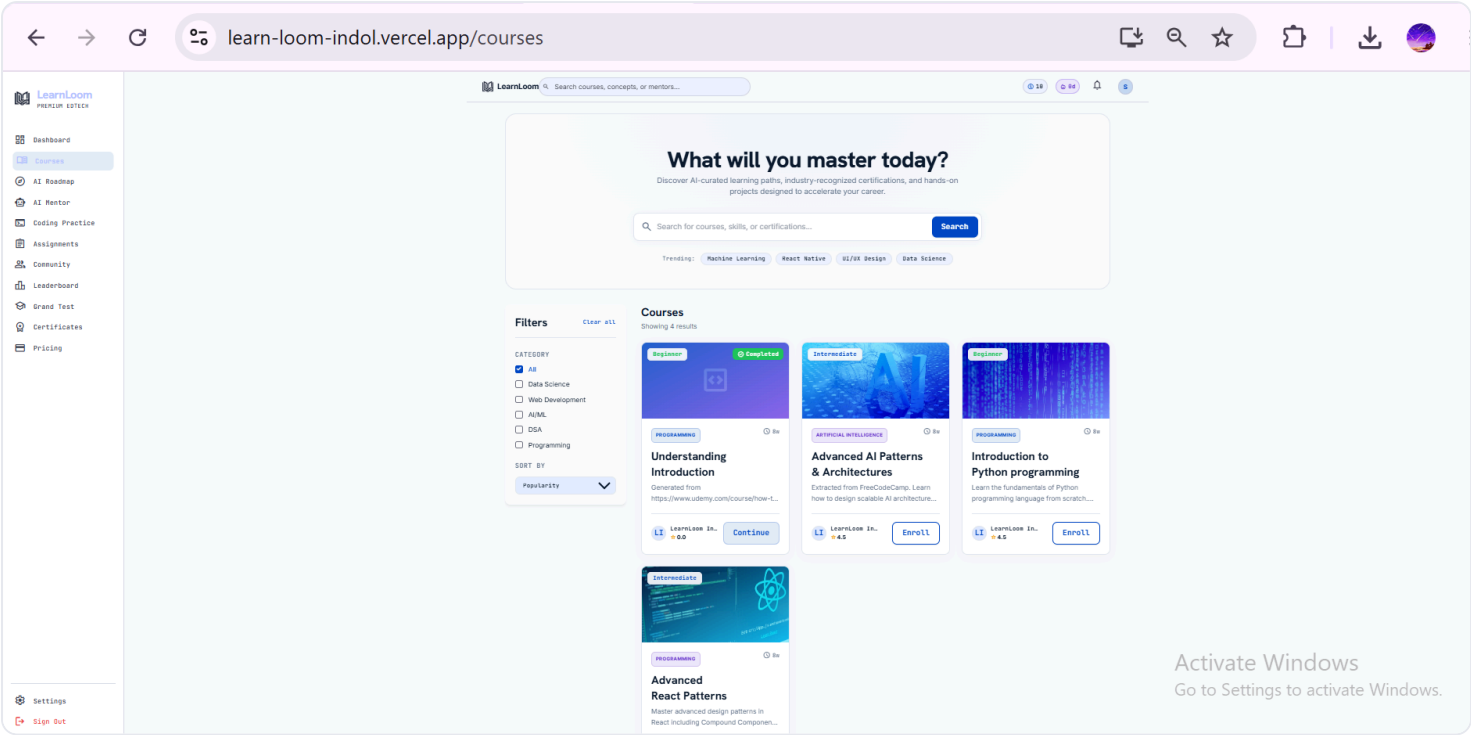


Figure 5.5 — Course catalog (/courses): browse, search and filter published courses. Live capture from the deployed app.

Purpose. Let learners discover and enrol in courses.

Workflow. `CourseCatalogPage` lists published courses; selecting one opens `CourseDetailPage` (/courses/:id) with the syllabus, ratings and an enrol/continue action.

User interaction. Search by keyword, filter by difficulty (Beginner / Intermediate / Advanced), inspect ratings, then enrol.

Business value. Discovery quality drives enrolment; ratings build trust and steer learners to high-quality content.

Technical implementation. The `courses` table (migration `00002`) carries `title`, `description`, `difficulty` (checked enum), `thumbnail_url`, `youtube_url`, `notes_url` and `is_published`. RLS exposes only published courses to the public (`public_read_published_courses`) while admins see everything. Later migrations add `category`, course `topics`, `rating` (synchronized from `course_ratings`) and `student_count` .

Advantages. Server-enforced visibility (drafts never leak), rating-driven trust, indexed queries for fast browsing.

Future improvements. Semantic/AI search, personalized "recommended for you" ordering, and saved/wishlist courses.

AI Course Recommendation — status note. The task brief lists "AI Course Recommendation." The repository contains AI **course generation** (`api/ai-course-generator.ts`, `api/ai-module-generator.ts`) and a course-generation pipeline, plus the personalized roadmap generator. A standalone collaborative "recommended courses" engine was **not** found as a discrete module; the closest verified capability is roadmap-driven guidance and difficulty/ category filtering. This is flagged rather than assumed.

5.5 Course Player & Strict Progression

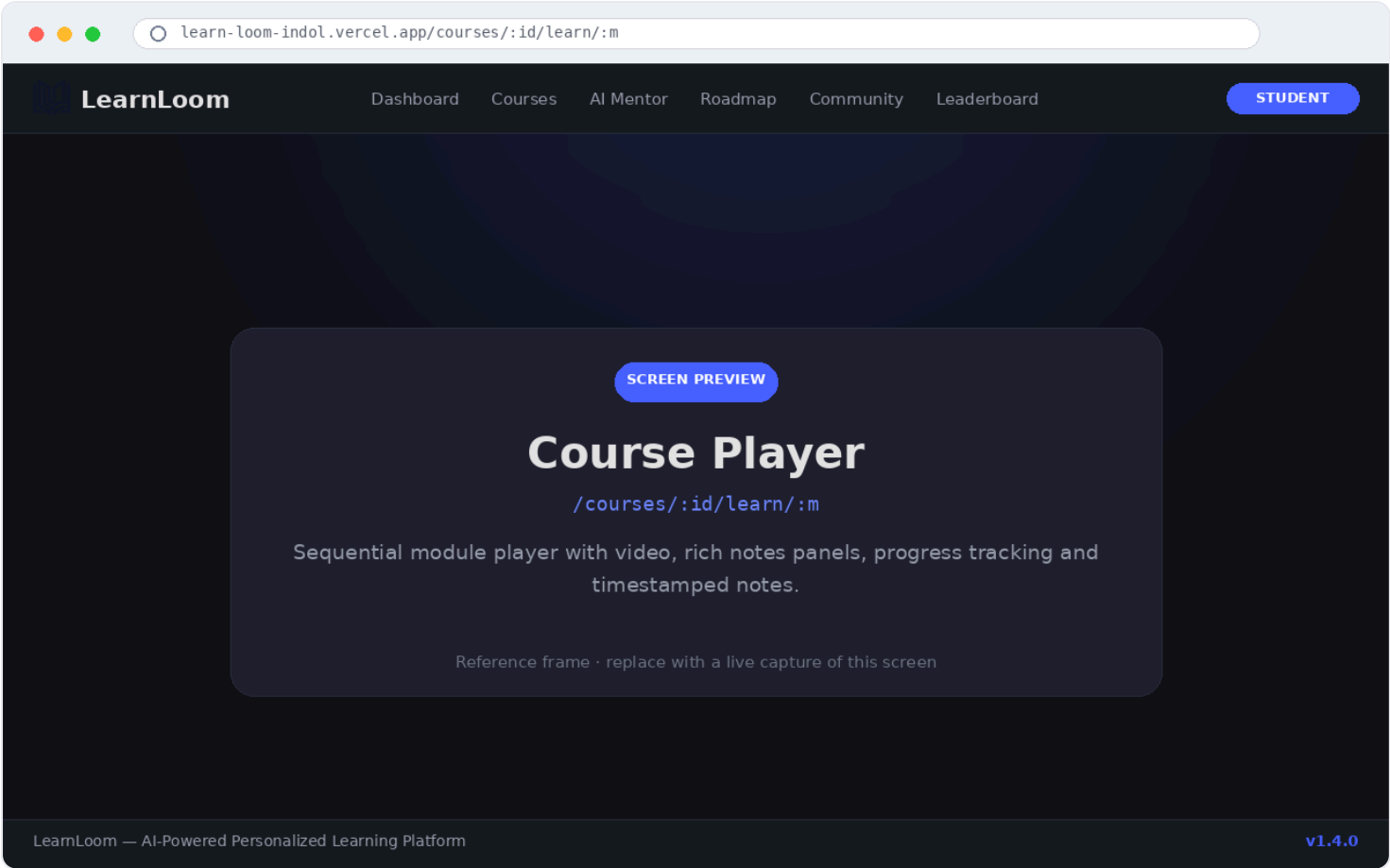


Figure 5.6 — Course player (`/courses/:id/learn/:moduleId`): sequential module consumption with progress tracking. Reference frame.

Purpose. Deliver course content in a deliberate, trackable sequence.

Workflow. From a course, the learner enters the player at a specific module. Modules unlock sequentially; completing one advances progress and unlocks the next. The README describes rich module fields — `examples` , `real_world_use_cases` , `key_concepts` , `summary` — rendered in interactive panels.

User interaction. Watch the module video (YouTube URL), read notes, mark complete, and optionally save **timestamped video notes** (`video_notes` table, migration `00023`).

Business value. Strict progression is what makes completion *meaningful* and certificates credible — learners cannot skip ahead to the certificate.

Technical implementation. Two tables drive this (migration `00003`): `user_course_enrollments` (one row per learner-course, with `progress_percent` , `last_module_id` , `completed_at`) and `user_module_progress` (one row per learner-module, with a `module_status` enum of `locked` / `unlocked` / `completed`). Database triggers award credits on module completion. The CHANGELOG documents the unification of module ordering on `order_index` and fixes to player navigation that always resolve a concrete `moduleId` .

Advantages. Enforced order, resumable learning (`last_module_id`), per-module granularity, and credit rewards that reinforce momentum.

Future improvements. Inline transcripts, adaptive pacing, and AI-generated module summaries surfaced directly in the player.

5.6 AI Mentor

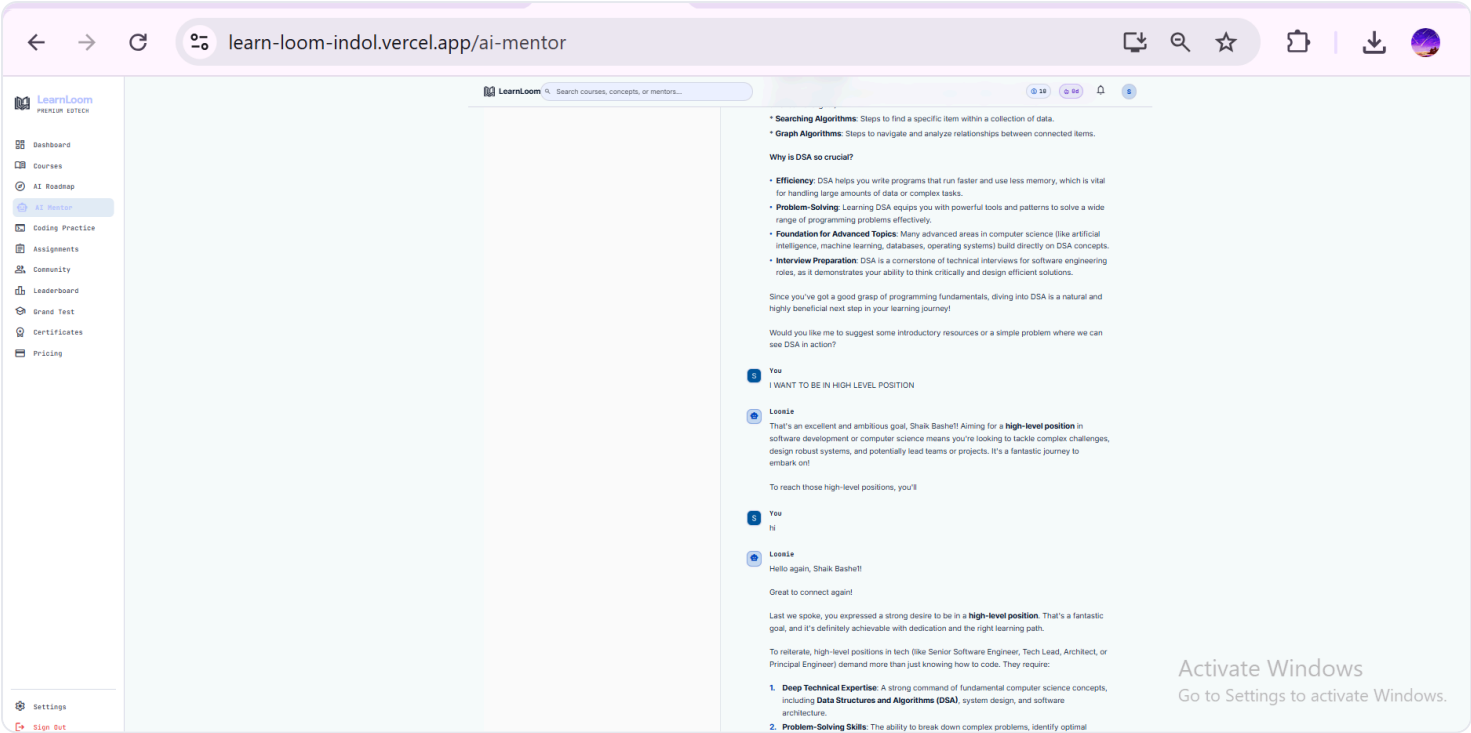


Figure 5.7 — AI Mentor (`/ai-mentor`): a streaming, Gemini-powered tutor. Live capture from the deployed app.

Purpose. Provide on-demand, contextual tutoring so learners are never stuck.

Workflow. The learner asks a question; the mentor streams an answer token-by-token. The system prompt (in `api/ai-mentor.ts`) defines the persona as *"LearnLoom AI Mentor — an expert computer science and programming tutor"* that explains step-by-step with working code, encourages the learner, and — importantly — **never hands over exam/test answers directly**, guiding the student to think instead.

User interaction. A familiar chat surface (`src/components/chat/`) backed by `streamdown` for streamed Markdown rendering.

Business value. Mentoring is the single biggest differentiator versus a static catalogue — it keeps learners moving and dramatically increases perceived value.

Technical implementation. The Vercel Edge route `api/ai-mentor.ts` calls Gemini 2.5 Flash's `streamGenerateContent` endpoint with `alt=sse` and pipes the upstream stream straight back to the client as `text/event-stream`. The route first calls `verifyAuth()` (a Bearer Supabase JWT is required) so only authenticated learners can spend AI quota. The API key is read from server environment (`INTEGRATIONS_API_KEY` / `GEMINI_API_KEY`) and **never reaches the browser**. A parallel Supabase Edge Function (`supabase/functions/ai-mentor/`) provides the same capability for self-hosted deployments.

Advantages. Real-time streaming UX, strict server-side auth, zero key exposure, an academic-integrity guardrail baked into the prompt, and two deployment paths (Vercel Edge or Supabase Edge).

Future improvements. Conversation memory across sessions (`ai_conversations` table already exists), retrieval grounding in the learner's current course, and voice input/output.

5.7 Personalized AI Roadmap

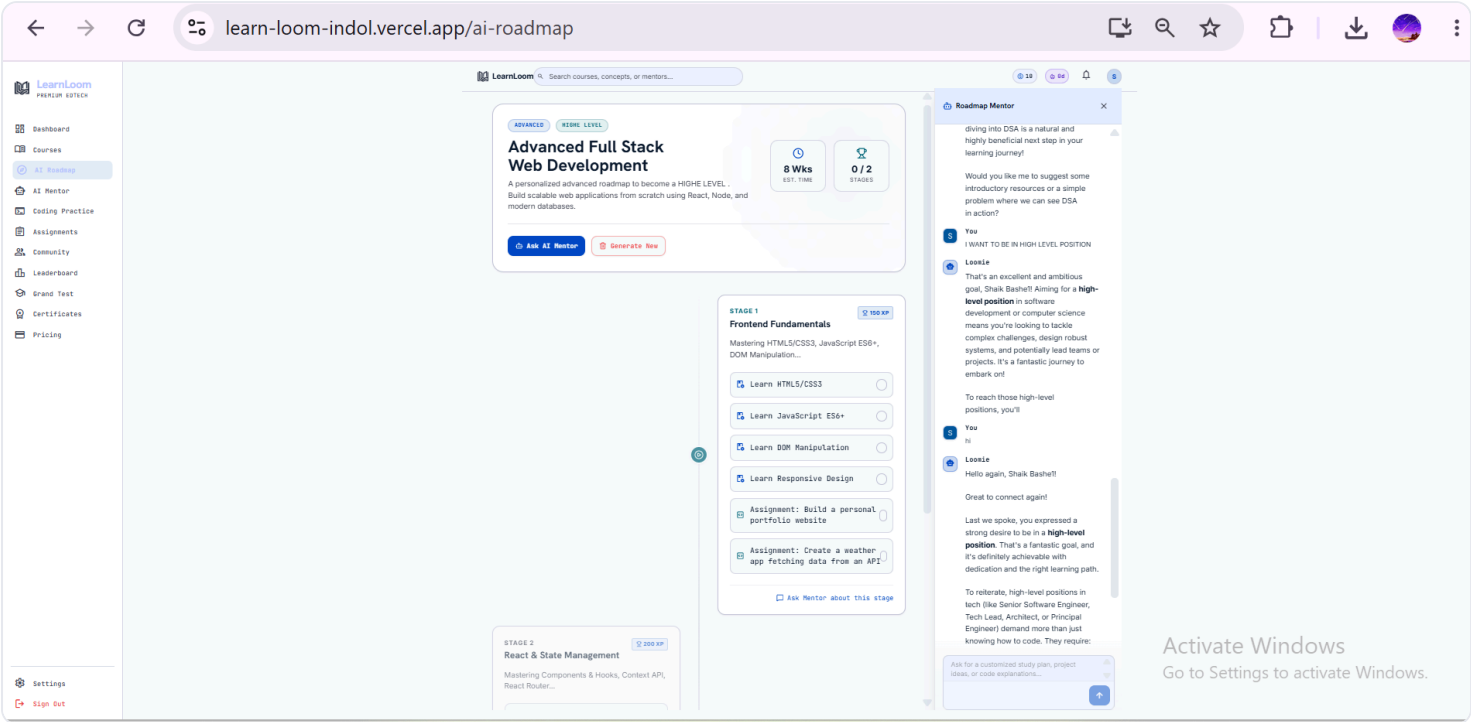


Figure 5.8 — Personalized AI Roadmap (`/ai-roadmap`): a phased plan generated from goal inputs. Live capture from the deployed app.

- Purpose.** Turn a high-level ambition into a concrete, phased, week-by-week plan.
- Workflow.** The learner provides a **domain**, a **target role/goal** and a **difficulty**. The system asks Gemini to act as an "expert EdTech curriculum designer" and return a strict JSON object: a title, a description, `estimated_weeks` and 3–6 `phases`, each with `topics` and `assignments`. The plan is then persisted and tracked.
- User interaction.** Pick a domain and role, choose a level, generate, then work through phases and watch `progress_percentage` climb.
- Business value.** This is the platform's career-guidance answer — it removes the hardest part of self-study (deciding the path) and anchors the rest of the learning loop.
- Technical implementation.** `src/lib/roadmapGenerator.ts` POSTs to `api/ai-roadmap.ts`. That Edge route is auth-gated, prompts Gemini with `responseMimeType: 'application/json'` and a low temperature (0.4) for deterministic structure, strips any stray Markdown fences, and returns raw JSON. **Graceful degradation:** if the AI call fails, the client falls back to curated `staticRoadmaps` (`src/data/roadmaps.ts`, persisted via the `roadmaps` table, migration `00026`), so the feature still works offline of the model. A difficulty multiplier compresses the estimated weeks for intermediate/advanced learners.
- Advantages.** Structured, deterministic JSON output; resilient static fallback; persisted and progress-tracked; admin-manageable templates (`roadmap_templates`, `roadmap_template_nodes`).
- Future improvements.** Link each phase directly to matching in-platform courses and coding problems, and re-plan dynamically as the learner progresses.

5.8 Coding Practice IDE

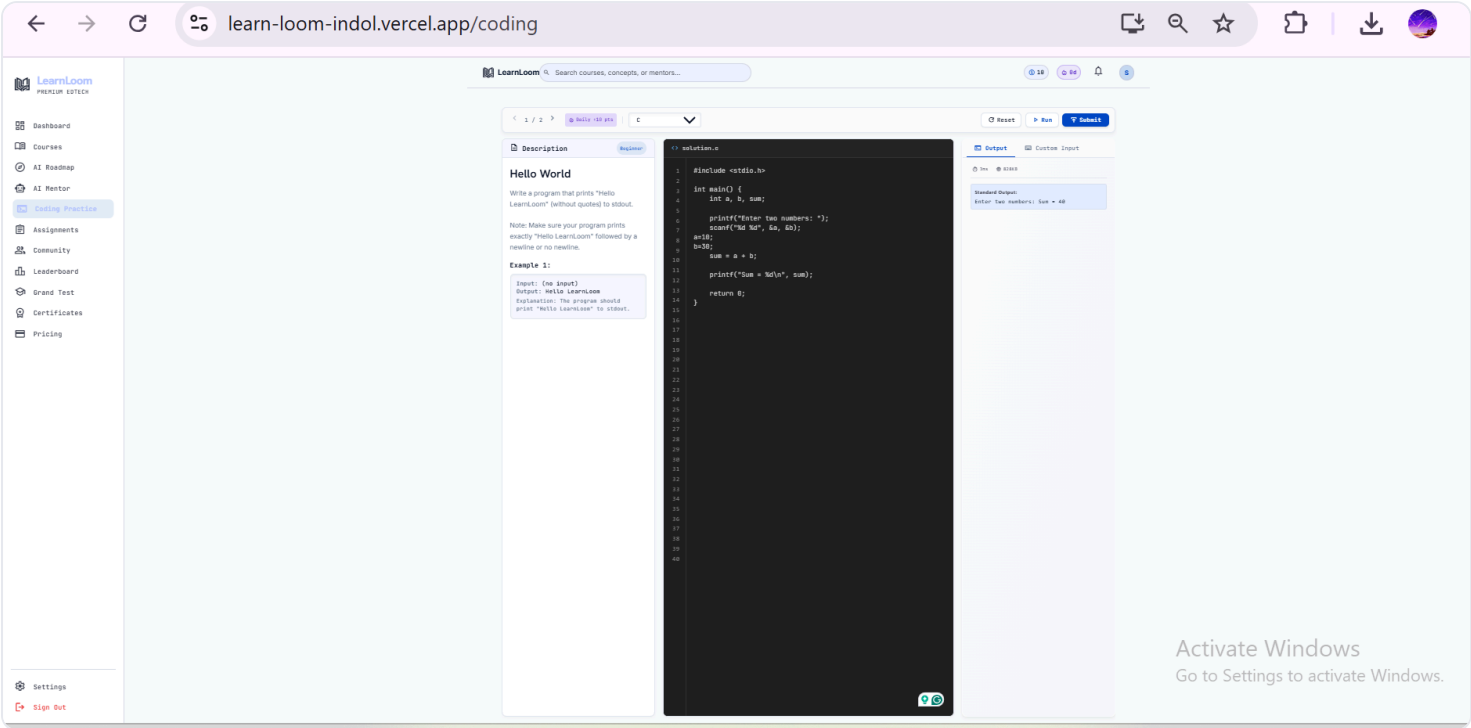


Figure 5.9 — Coding Practice IDE (/coding): write, run and test code in the browser. Live capture from the deployed app.

Purpose. Provide hands-on programming practice with immediate, automated feedback.

Workflow. The learner picks a problem, writes a solution in one of several languages, runs it against sample cases, then submits to be judged against the full (including hidden) test set.

User interaction. Language selector (`python` , `javascript` , `java` , `cpp` , `c`), an editor seeded with per-language starter code, *Run* (sample cases) and *Submit* (all cases, with verdict, timing and credits).

Business value. Practice-with-feedback is where real skill is built; it also feeds the coding portion of the certification gate, raising the credibility of certificates.

Technical implementation. Problems live in `coding_questions` + `coding_test_cases` (migration `00030`) with `starter_code` (JSONB per language), `constraints` , an `is_assessment` flag, and `is_hidden` test cases. Execution is delegated to the **Piston** engine: the client calls `/api/piston/execute` , which `vercel.json` rewrites to a self-hosted Piston instance, and a Supabase Edge Function `execute-code` provides an authenticated, **rate-limited** (10 requests/minute/user) path that maps languages, runs hidden tests server-side and records results. Accepted submissions award credits (`coding_submissions` , migration `00013`).

Advantages. Multi-language, real sandboxed execution (not mocked), hidden test cases to prevent gaming, server-side rate limiting, and credit rewards integrated with gamification.

Future improvements. Richer editor (Monaco) features, custom stdin, performance/complexity feedback, and AI hints from the mentor on failing cases.

5.9 Assessments, Quizzes & Certification Gate

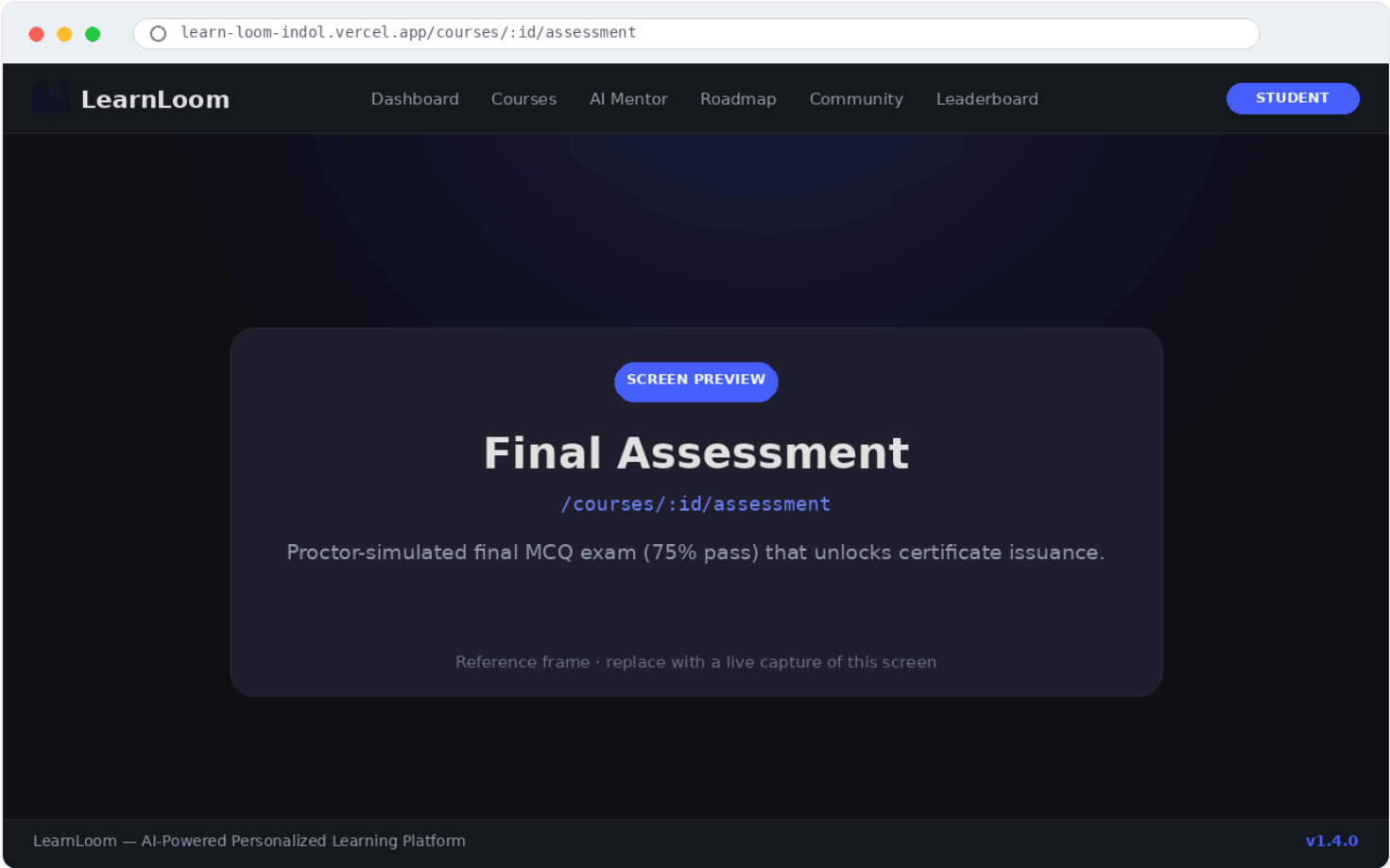


Figure 5.10 — Final assessment (/courses/:id/assessment): the proctor-simulated MCQ gate. Reference frame.

Purpose. Verify that a learner has actually mastered a course before certifying them.

Workflow (the gate, from the README). 1. The learner reaches **100% module progress** → the enrollment enters an "Assessments Unlocked" state. Crucially, `completed_at` is **not** set and no certificate is issued yet. 2. They must pass the **Final MCQ Assessment** (75% pass mark), which is proctor-simulated. 3. If the course contains coding questions, they must also pass the **Final Coding Assessment** (/courses/:id/coding-assessment). The system queries `coding_questions` for the course; if none exist, the coding gate is automatically bypassed. 4. On passing all required assessments, an average score is computed, a verified certificate is created, and `completed_at` is set to mark the learner certified.

User interaction. Quizzes (/quiz/:id) and the final assessment present MCQs; attempts are recorded; results determine whether the certificate unlocks.

Business value. This gate is the entire reason a LearnLoom certificate means something — it converts "I watched the videos" into "I demonstrably passed."

Technical implementation. `quizzes` + `quiz_questions` (migration `00002`, options as JSONB with an `answer_index`); `quiz_attempts`; `assessment_attempts` (migration `00030`, storing `score_percentage`, `is_passed`, `attempt_number`, `metrics` JSONB) — all RLS-scoped so a learner only reads/writes their own attempts. A separate **Grand Test** (/grand-test) offers a global proctored exam with a 1-hour cooldown trigger and an auto-certificate trigger on passing (migrations `00020`, `00021`).

Advantages. Dynamic requirement mapping (coding gate only when relevant), strict pass marks, attempt auditing, anti-spam cooldowns, and admin-auditable attempt records.

Future improvements. Adaptive question difficulty, question banks with randomization, and AI-generated explanations for wrong answers.

5.10 Assignments

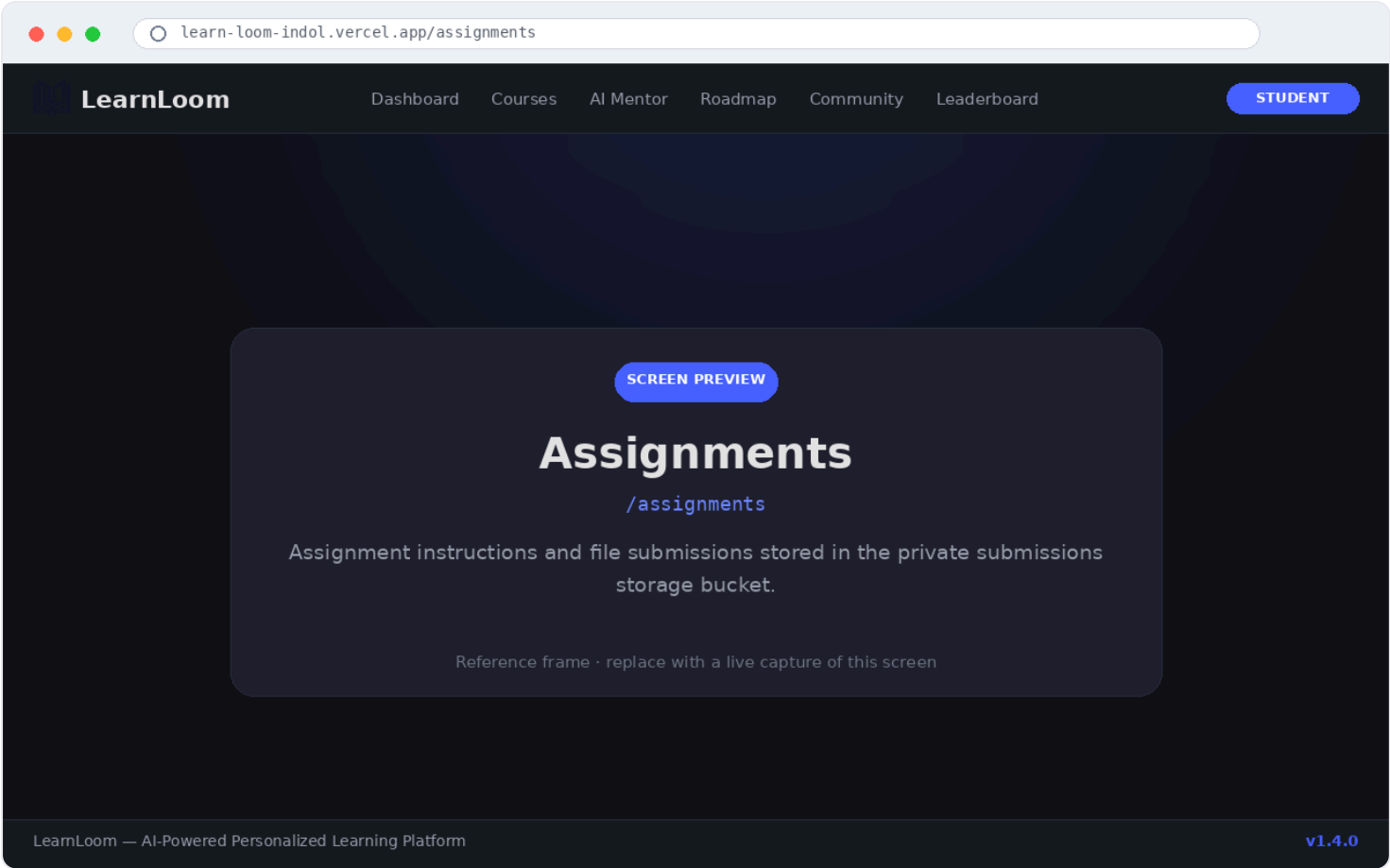


Figure 5.11 — Assignments (/assignments): instructions plus file submissions. Reference frame.

Purpose. Provide applied, submittable work beyond multiple-choice questions.

Workflow. A course defines assignments (assignments table: title , instructions , due_days); the learner reads the brief and uploads a submission; admins review submissions.

User interaction. Drag-and-drop upload (react-dropzone + src/components/dropzone.tsx), then track status.

Business value. Project-style work demonstrates deeper competence and gives admins concrete artifacts to assess.

Technical implementation. assignments (migration 00002) and assignment_submissions store the work; files go to a private submissions storage bucket (50 MB limit, per the README) with RLS that restricts read/write to the owning student and admins. Admins review via /admin/submissions .

Advantages. Private-by-default storage, owner-scoped RLS, clean admin review surface.

Future improvements. Inline rubric grading, peer review, and AI-assisted first-pass feedback.

5.11 Certificates & Public Verification

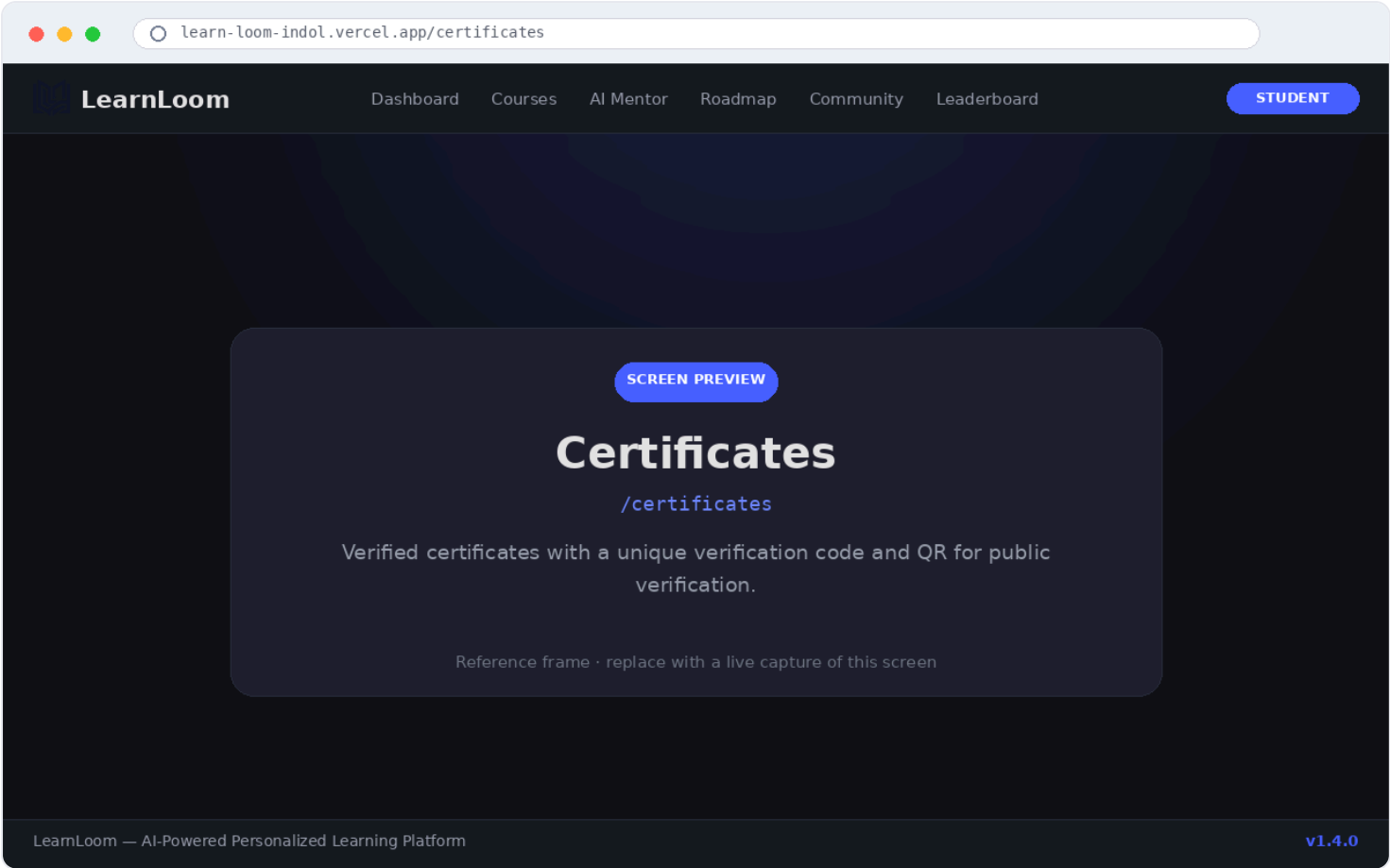


Figure 5.12 — Certificates (/certificates): issued, verifiable credentials. Reference frame.

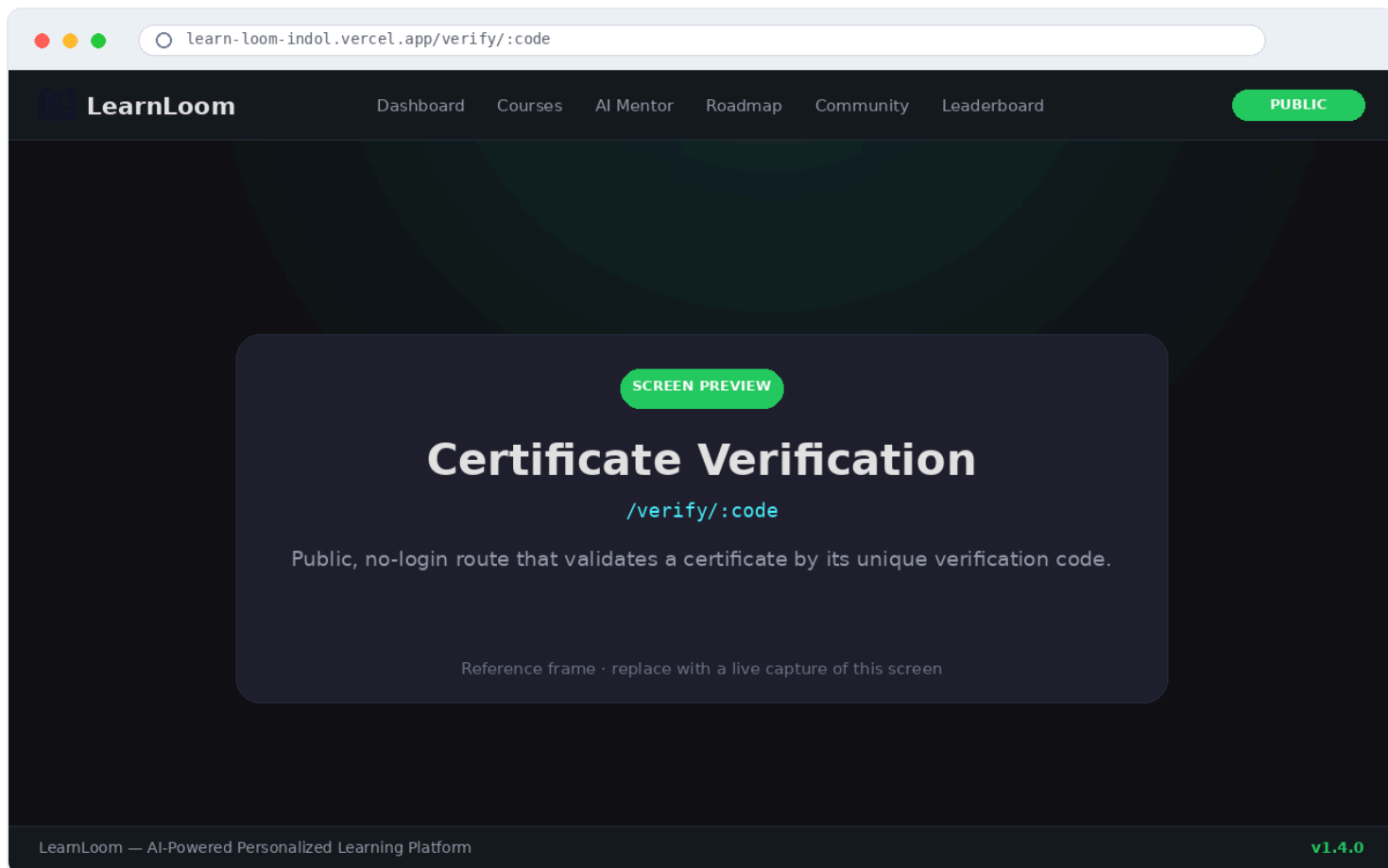


Figure 5.13 — Public verification (`/verify/:code`): validate a certificate with no login. Reference frame.

Purpose. Issue tamper-evident, publicly verifiable proof of completion.

Workflow. When a learner passes the required assessments, a row is written to `certificates` with a **unique verification code**; the learner can view/download it, and anyone can validate it at the public `/verify/:code` route.

User interaction. Learners see their certificates; third parties (employers) paste a code or scan a QR to confirm authenticity.

Business value. Verifiable credentials are the platform's "outcome currency" — they make learning provable to the outside world, which is the ultimate value proposition for learners.

Technical implementation. The `certificates` table (migration `00007`) stores `user_id`, `course_id`, `score`, a `UNIQUE verification_code`, `issued_at` and `is_valid`, with a `UNIQUE(user_id, course_id)` constraint and a database trigger (`generate_cert_code`) that auto-generates the code. Indexes on `user_id` and `verification_code` keep verification fast. QR generation uses the `qrcode` dependency. RLS hardening ensures learners can only read/write their own certificates (`auth.uid()::text = user_id::text`), while admins manage them via `/admin/certificates`. The Grand Test path can also auto-issue a course-independent certificate.

Advantages. Cryptographically-unique codes, public no-auth verification, QR support, strict RLS, and an admin audit/revoke surface (`is_valid`).

Future improvements. On-chain or signed-credential standards (e.g. Open Badges / Verifiable Credentials) and shareable LinkedIn integration.

5.12 Community Forum

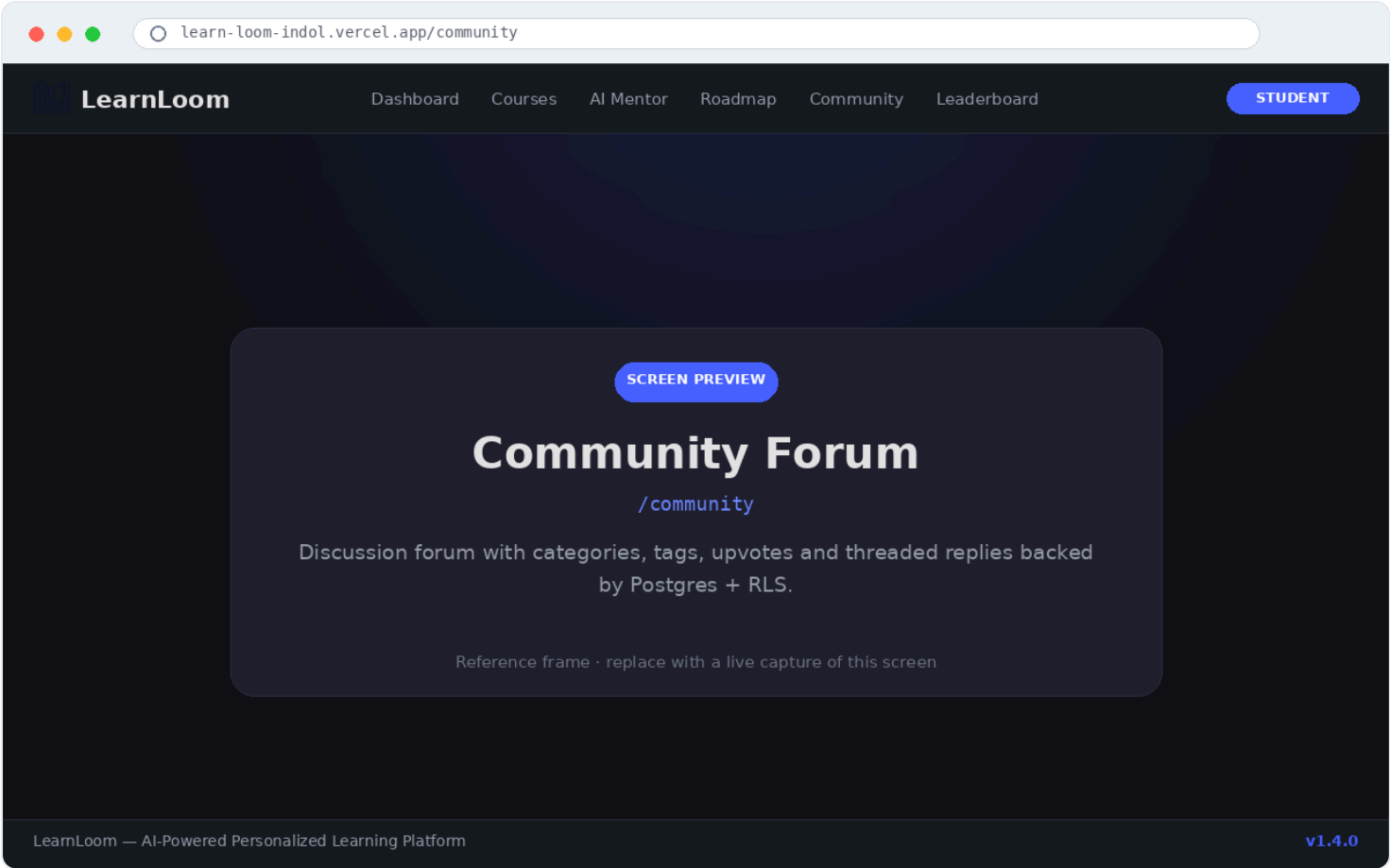


Figure 5.14 — Community forum (/community): categorized, votable discussions. Reference frame.

Purpose. Build a peer-learning community for doubts, challenges and study groups.

Workflow. Learners create posts in categories (general / doubt / challenge / study-group), tag them, reply, and upvote posts and replies.

User interaction. Browse/filter by category, post (title 5–200 chars, content ≥10 chars — enforced by DB CHECK constraints), reply, and toggle upvotes.

Business value. Community is a powerful retention and engagement lever and offloads some support to peers.

Technical implementation. forum_posts (migration 00007) carries category (checked enum), tags (text array), upvotes , reply_count and is_pinned , with indexes on user_id , created_at DESC and category . Replies live in forum_replies / forum_replies_v2 (migration 00011), and voting uses atomic, race-safe RPCs (toggle_reply_vote , forum_votes / forum_reply_votes join tables) so concurrent upvotes don't corrupt counts. Admins moderate via /admin/community with pin and flag controls (admin_moderation_flags , migration 00024).

Advantages. Input validation at the database layer, atomic vote toggles, category structure, and first-class moderation tooling.

Future improvements. Realtime updates (Supabase Realtime), rich text/code blocks, reputation tiers, and AI-assisted duplicate-question detection.

5.13 Leaderboard, Gamification, XP & Credits

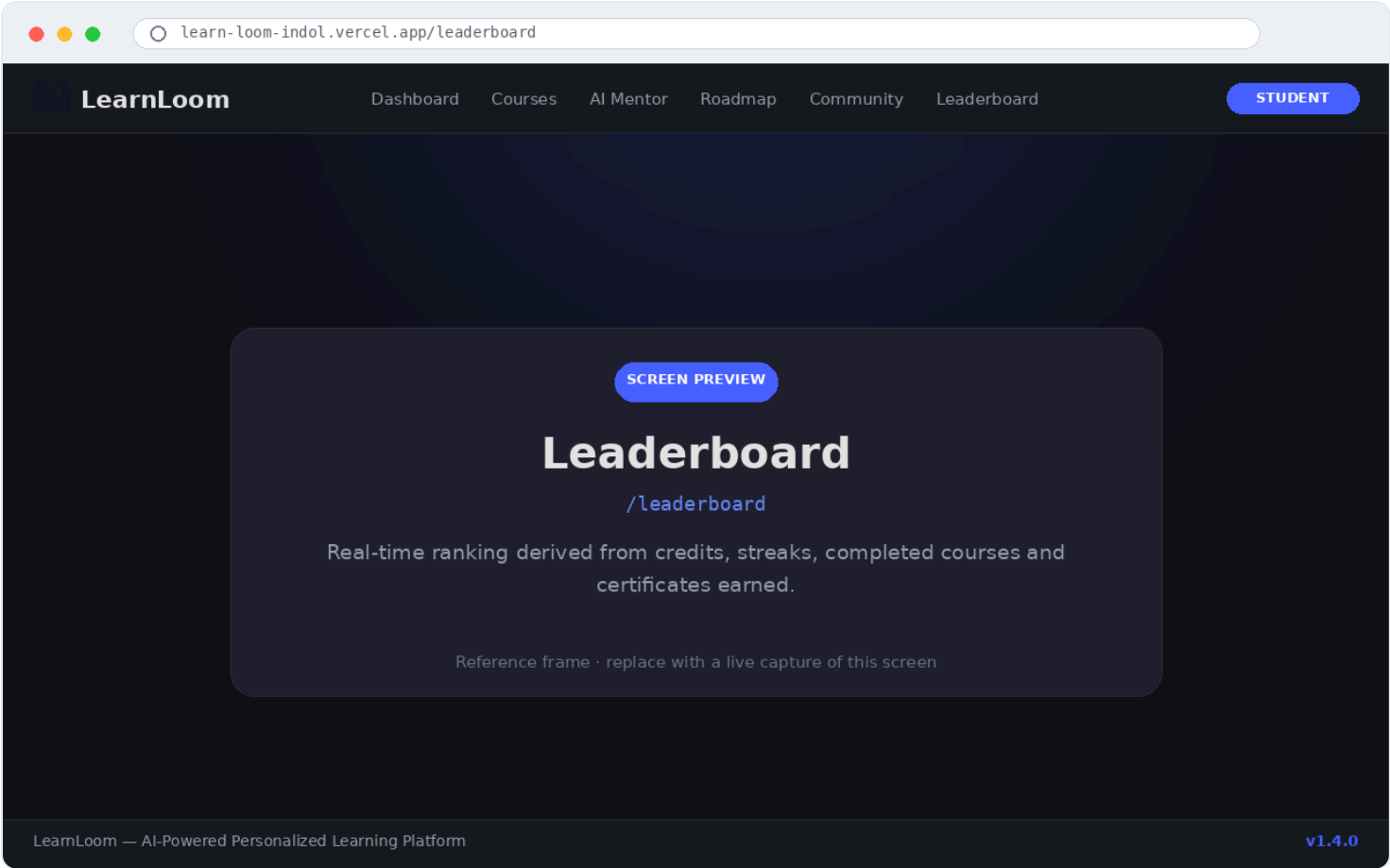


Figure 5.15 — Leaderboard (`/leaderboard`): real-time ranking by credits and streaks. Reference frame.

Purpose. Motivate sustained learning through points, streaks and friendly competition.

Workflow & user interaction. Learners earn **credits** (the platform's XP currency) and maintain daily **streaks**; the leaderboard ranks them; the dashboard and profile surface their totals.

Business value. Gamification is a proven engagement multiplier — it converts isolated study into a visible, rewarded, competitive activity that pulls learners back daily.

Technical implementation (verified credit economy):

Action	Reward	Source
Complete a module	+10 credits (and updates <code>last_activity_date</code>)	<code>award_module_credits()</code> trigger, migration <code>00007</code>
Pass the Grand Test	+100 credits	grand-test trigger, migrations <code>00007</code> / <code>00020</code>
Accepted coding submission	credits awarded (<code>credits_awarded</code>)	migration <code>00013</code>
Subscription tier	monthly credit allowance (e.g. 500 / 1500)	<code>subscription_plans</code> , migration <code>00012</code>

Credits live on `profiles.credits` (migration `00007`) alongside `streak_days` ; an atomic `increment_credits(p_user_id, p_amount)` RPC (migration `00010`) mutates them safely. The `leaderboard_view` (migration `00007`) ranks learners with `ROW_NUMBER() OVER (ORDER BY credits DESC, streak_days DESC)` and joins in courses completed and certificates earned; a later fix (migration `00019`) filters the view to students so admin accounts don't pollute rankings, plus rewards configuration tables (`reward_configurations`).

Advantages. Trigger-driven (server-authoritative) rewards that can't be faked client-side, an atomic credit RPC, a derived real-time leaderboard, and admin-tunable reward configs.

Future improvements. Badges/achievements, seasonal leaderboards, streak-freeze items, and team/cohort competitions.

5.14 Student Profile & Progress Tracking

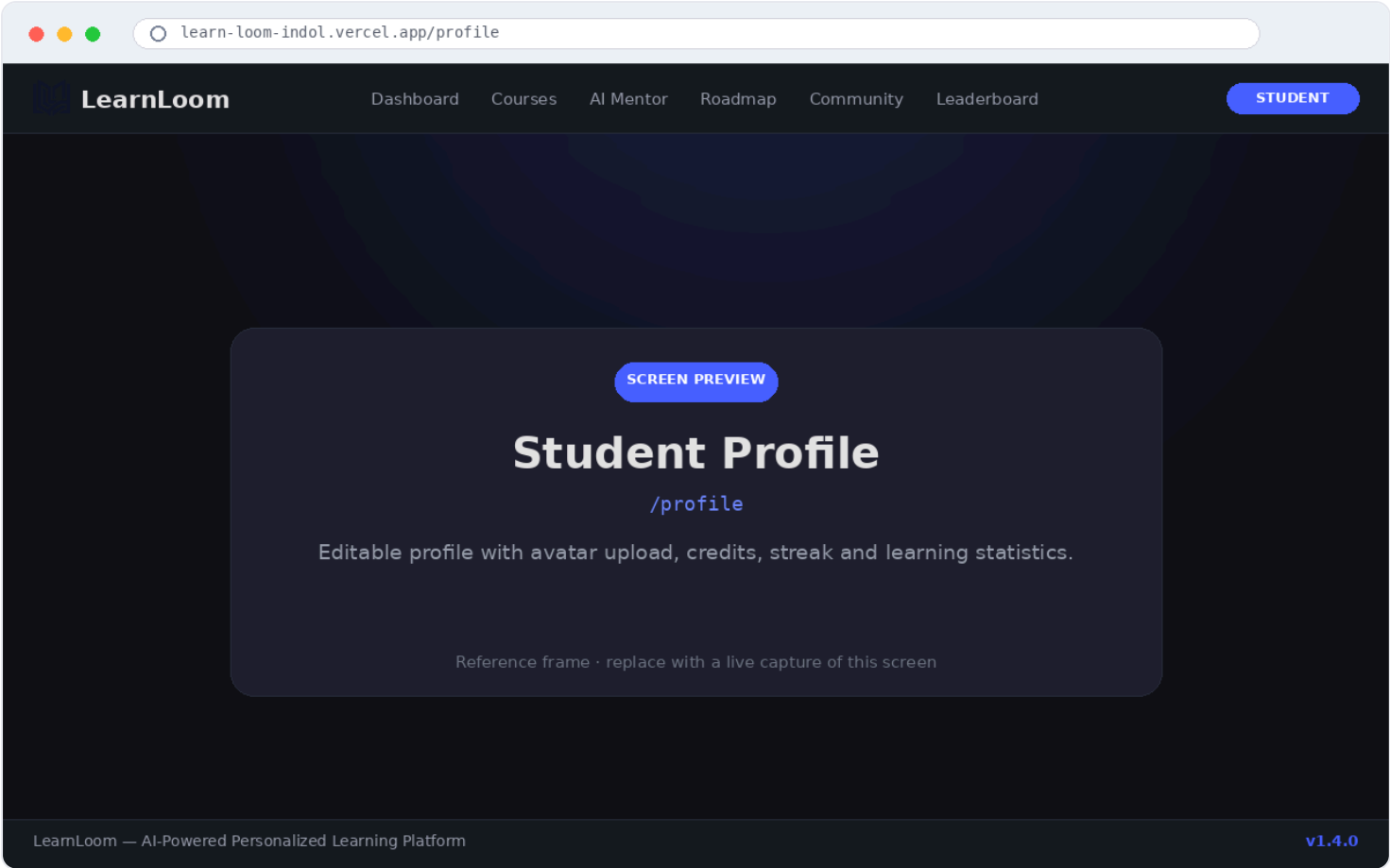


Figure 5.16 — Student profile (/profile): identity, avatar and learning stats. Reference frame.

Purpose. Centralize identity, settings and a learner's progress at a glance.

Workflow & interaction. Learners edit their name, upload an avatar, and review credits, streak, enrolments and certificates.

Technical implementation. Backed by `profiles` (migration `00001`) with `full_name`, `avatar_url`, `role`, `credits` and `streak_days`. Avatars upload to a public `avatars` bucket with owner-scoped insert/update RLS (README). Progress derives from `user_course_enrollments` and `user_module_progress`; a `public_profiles` view exposes only safe public fields (id, name, avatar, role). Activity is logged via `logUserActivity` (`src/lib/activity.ts`) into `user_activity_logs` / `daily_activity` (migrations `00028`, `20260615043500`).

Business value. Profiles personalize the experience and provide the data backbone for streaks, leaderboards and the dashboard.

Advantages. Safe public projection of profile data, owner-scoped avatar storage, and a unified activity log feeding analytics.

Future improvements. Skill graphs, public learner portfolios, and shareable progress cards.

5.15 Notifications

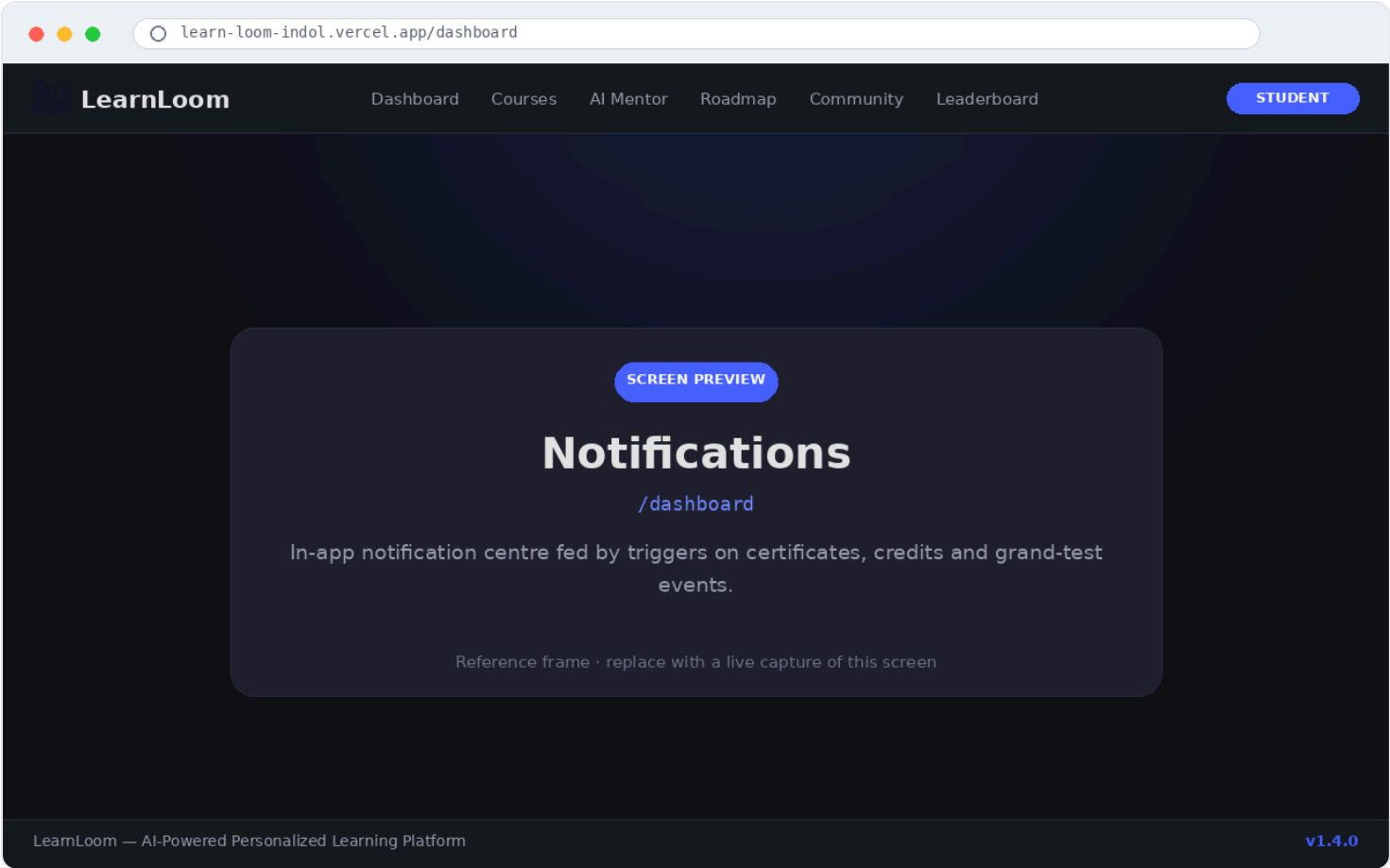


Figure 5.17 — In-app notification centre, fed by database triggers. Reference frame.

Purpose. Keep learners informed of meaningful events.

Workflow. Events such as passing the Grand Test create notifications that appear in-app.

Technical implementation. The `notifications` table (migration `00016`) stores `user_id`, `type`, `message`, `read` and `created_at`, with RLS so a user reads only their own. A `NotificationContext` (`src/contexts/NotificationContext.tsx`) provides them app-wide, and database triggers (e.g. the grand-test trigger) insert notifications server-side.

Business value. Timely nudges drive re-engagement and celebrate milestones.

Advantages. Server-generated (trustworthy) events, owner-scoped RLS, and a global context for consistent display.

Future improvements. Email/push delivery, granular preferences, and digest batching.

5.16 Search

Purpose. Help learners quickly find courses and content.

Workflow & implementation. Course discovery supports keyword search and difficulty/category filtering over the `courses` table (with `category` and `topics` added in later migrations). The UI toolkit includes `cmdk` (a command-palette/search primitive) and a `use-debounce` hook, indicating debounced, responsive search input.

Business value. Fast discovery shortens time-to-enrolment.

Advantages. Debounced input and indexed columns keep search snappy.

Future improvements. Global, cross-entity search (courses, roadmaps, forum) and AI semantic ranking. *Honesty note: a single global search index across all entities was not found as a discrete module; current search is centred on the catalogue.*

5.17 Subscriptions & Payments

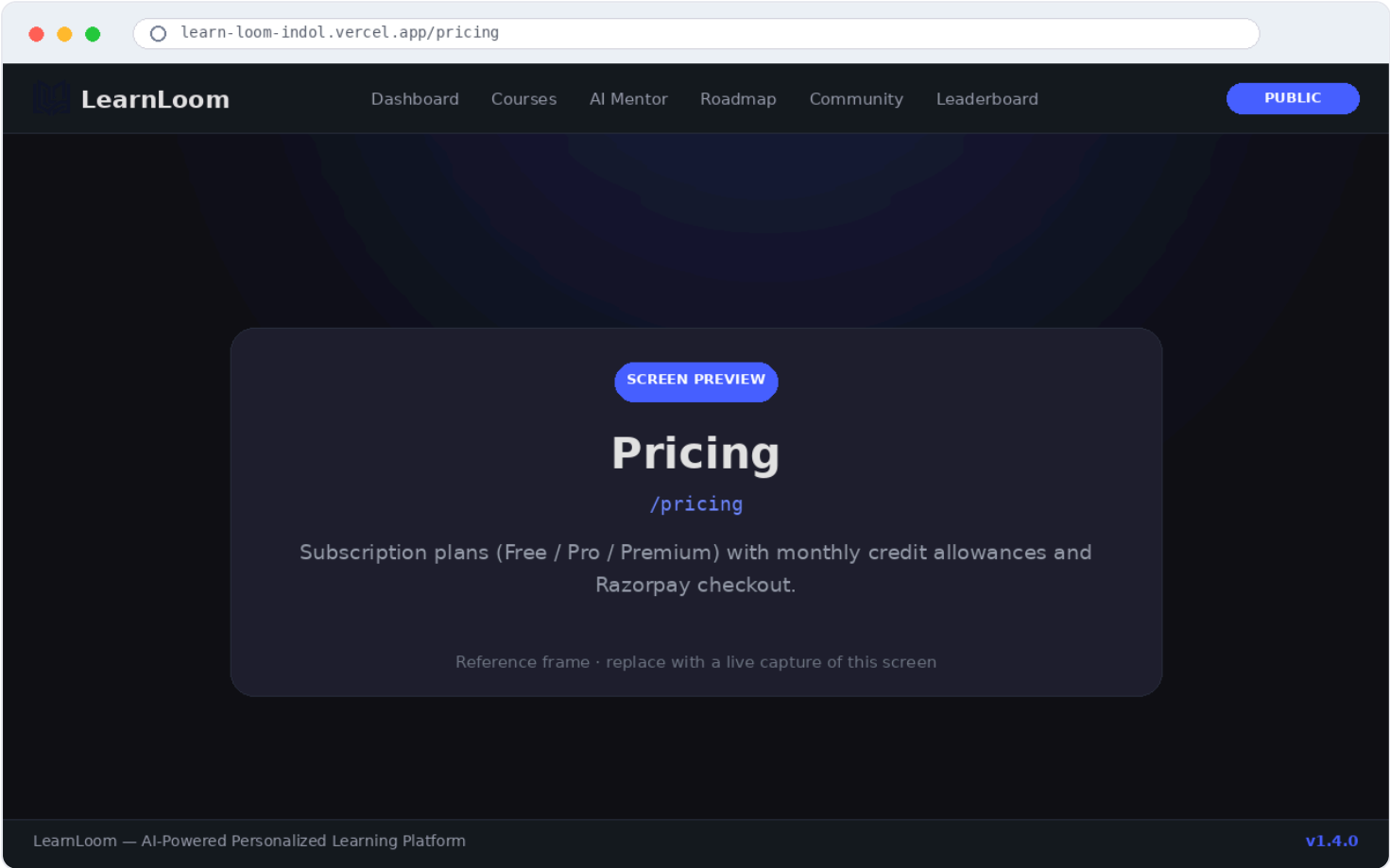


Figure 5.18 — Pricing (/pricing): Free / Pro / Premium tiers with Razorpay checkout. Reference frame.

Purpose. Monetize premium capability and fund monthly AI credit allowances.

Workflow. Learners view plans, choose a tier, and pay via Razorpay; a successful payment activates a subscription and grants monthly credits.

Technical implementation. `subscription_plans` (migration `00012`) seeds Free / Pro / Premium with `price_inr`, `credits_per_month` and a JSONB feature list; `user_subscriptions` tracks `status` (`active` / `cancelled` / `expired` / `trialing`) and `expires_at`; `payment_orders` records orders. Two Supabase Edge Functions handle the flow: `create-razorpay-order` and `verify-razorpay-payment` (the latter validating the payment signature server-side).

Business value. A clear, credit-based monetization model aligned with AI usage costs.

Advantages. Server-side order creation and signature verification (no trust in the client), plan features stored as data, and a credits-per-month model that ties revenue to value delivered.

Future improvements. Annual billing, regional pricing, coupons, and usage-based AI metering.

5.18 Responsive Design

Purpose. Deliver a first-class experience on every screen size.

Implementation. The README states the UI is a "Premium Glassmorphism Design System" with dark/light modes, "fully responsive audited down to 320px with strict touch-target compliance." Tailwind CSS plus @tailwindcss/container-queries , a use-mobile hook, and Radix UI primitives (which are accessible and adaptive) underpin this. next-themes powers theme switching.

Business value. Mobile-first reach is essential for learners who study on phones.

Advantages. Container queries for component-level responsiveness, audited 320px floor, and consistent design tokens.

Future improvements. A dedicated installable PWA/mobile app (a manifest.json already ships in public/).

5.19 Admin Console

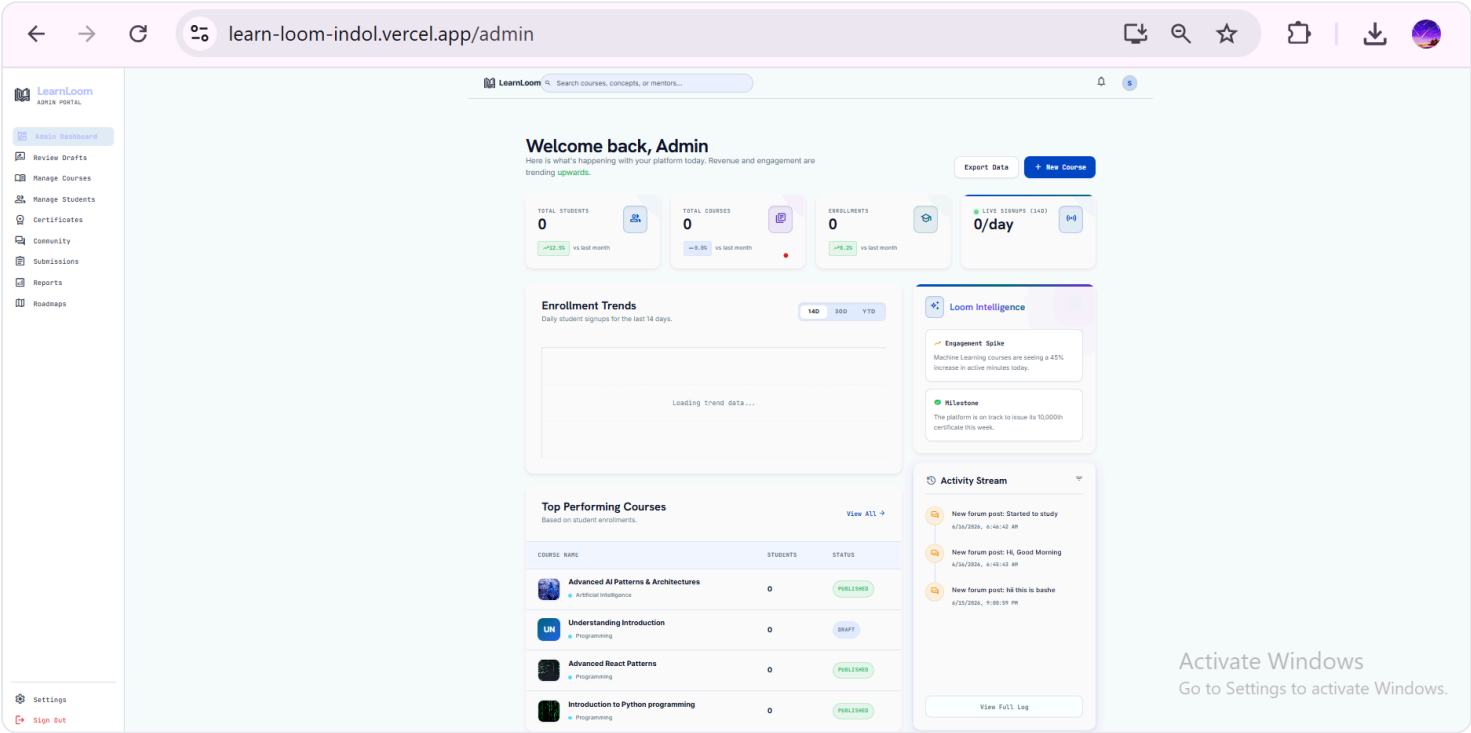


Figure 5.19 — Admin dashboard (/admin): platform stats, enrolment trends and recent activity. Live capture from the deployed app.

The admin console is a full operations surface gated by strict RBAC. The detailed admin journey is in User Flows; the screens are:

Admin screen	Route	Responsibility
Dashboard	<code>/admin</code>	Stats via <code>get_admin_stats</code> RPC, enrolment trends, top courses, activity
Course Moderation	<code>/admin/courses</code>	Create/edit/publish courses, modules, quizzes, coding questions
Draft Review	<code>/admin/courses/drafts</code>	Review AI/pipeline-generated draft courses before publishing
Student Management	<code>/admin/students</code>	Directory and account management
Certificate Management	<code>/admin/certificates</code>	Audit, validate, revoke certificates
Community Moderation	<code>/admin/community</code>	Moderate posts/replies, pin, action flags
Roadmap Management	<code>/admin/roadmaps</code>	Manage roadmap templates and phases
Reports & Analytics	<code>/admin/reports</code>	Platform analytics and exports
Submissions Review	<code>/admin/submissions</code>	Review assignment submissions

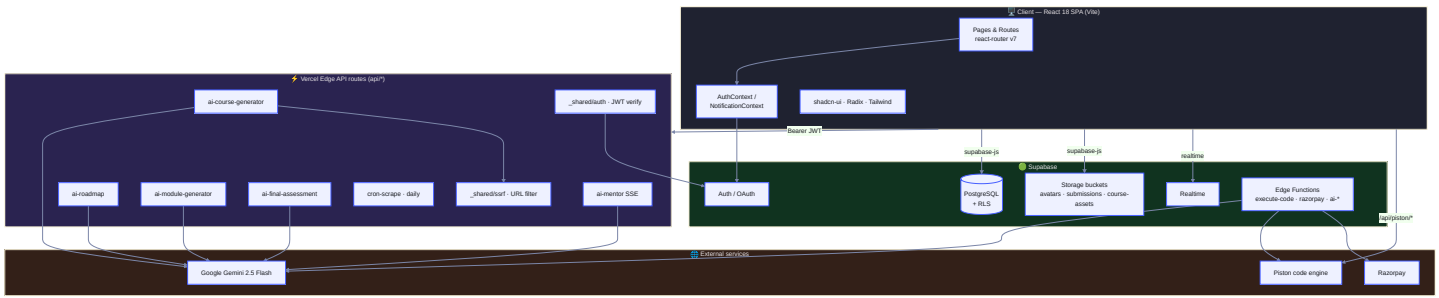
Additional admin pages present in the repository include AI settings, compiler/Piston config, notifications, organizations, rewards and a grand-test manager.

RBAC (verified). `RouteGuard` and `AuthContext` enforce that **only the email `shaikbashe2222@gmail.com` with role `super_admin`** can reach `/admin/*` ; any other user hitting an admin route is redirected to `/dashboard` , and unauthorized elevated roles are downgraded to `user` . This is a deliberately strict, single-owner admin model for the current deployment.

6. System Architecture

LearnLoom is a **static SPA + serverless** architecture. The browser runs a React/Vite bundle; all privileged work (AI calls, payments, code execution) happens in serverless Edge functions that authenticate every request; Supabase provides the database, auth, storage and realtime layer; and Google Gemini provides intelligence.

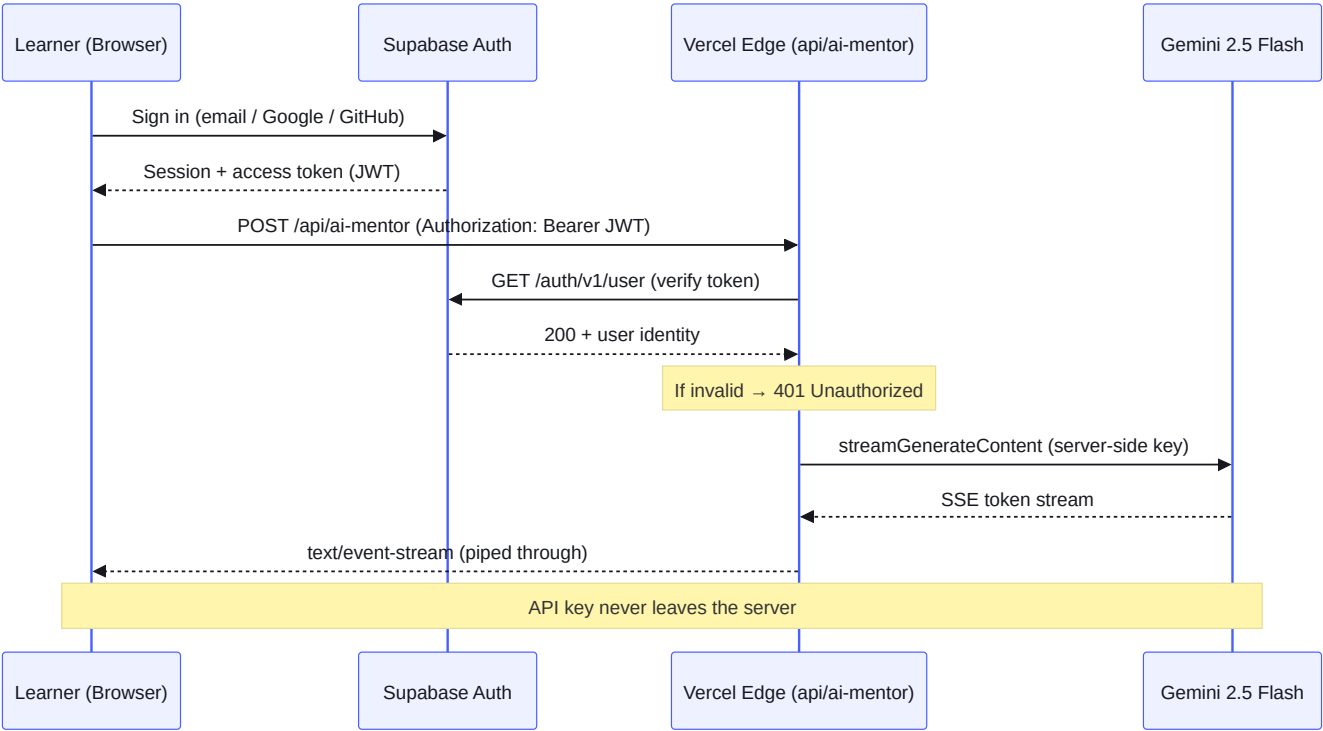
6.1 High-level architecture



6.2 Layer responsibilities

Layer	Technology	Responsibility
Frontend	React 18 + TS + Vite, React Router v7, Tailwind, Radix/shadcn	Rendering, routing, client state, route guarding, realtime subscriptions
Edge / API	Vercel Edge Functions (<code>api/*.ts</code> , <code>runtime: 'edge'</code>)	Auth-gated AI proxying, SSRF protection, scheduled scraping cron
Backend / Data	Supabase PostgreSQL + RLS	Source of truth, triggers, RPCs, derived views, row-level authorization
Auth	Supabase Auth + OAuth (Google/GitHub)	Sessions, JWTs, profile provisioning via trigger
Storage	Supabase Storage	Avatars (public), submissions (private), course assets (public)
AI	Google Gemini 2.5 Flash	Mentor, roadmap, course/module generation, assessment generation
Execution	Piston (Judge0 referenced)	Sandboxed multi-language code execution
Deployment	Vercel (+ Netlify config present)	Static hosting, Edge routes, cron, rewrites

6.3 Request data flow (authenticated AI call)



6.4 Frontend composition

`src/App.tsx` wires the app shell: `AuthProvider` → `NotificationProvider` → `RouteGuard` → `ErrorBoundary` → `Suspense` → routed pages. Every route is **lazy-loaded** (`React.lazy`), giving automatic code-splitting. The route registry in `src/routes.tsx` cleanly separates **public**, **student** and **admin** routes, and each route is individually wrapped in an `ErrorBoundary` so a failure in one screen never takes down the app.

6.5 Backend & auxiliary services

Beyond the SPA and Edge routes, the repository contains supporting Python services — a `scraping_engine/` (Scrapy-based content scraper with Alembic migrations and Docker), `backend/` (Azure Functions app), and `services/` (an autonomous crawler and queue worker) — used to assemble course content. *Honesty note:* migration `00029_remove_jobs_and_scraping` removes jobs/scraping objects from the core schema, so portions of this pipeline appear deprecated or optional relative to the live app; they are documented as present in the repo rather than asserted to be active in production.

7. Technology Stack

Every technology below was confirmed in `package.json`, configuration files or source.

7.1 Frontend

Technology	Version (repo)	Why it was selected
React	18.x	Mature component model, huge ecosystem, concurrent rendering; the basis for a complex multi-view app
TypeScript	~5.9	Static typing across a large surface (auth, DB types, AI payloads) catches errors at compile time and documents intent
Vite	8.x	Extremely fast dev server (HMR) and optimized production builds; outputs a static <code>dist/</code> deployable anywhere
React Router	7.x	Declarative routing with lazy loading and nested layouts for public/student/admin areas
Tailwind CSS	3.4	Utility-first styling enabling the consistent glassmorphism design system and rapid, responsive UI
Radix UI / shadcn-ui	"new-york" style	Accessible, unstyled primitives (dialog, dropdown, tabs, toast...) composed into the design system
Lucide React	0.576	Clean, consistent icon set
Recharts	2.15	Admin analytics charts (enrolment trends, daily activity)
react-hook-form + zod	7.x / 3.25	Performant forms with schema validation
streamdown	1.x	Renders streamed Markdown for the AI mentor
motion	12.x	Animations/transitions
qrcode	1.5	Certificate QR codes

7.2 Backend, data & services

Technology	Why it was selected
Supabase	An all-in-one backend (Postgres + Auth + Storage + Realtime + Edge Functions) that removes the need to build and operate a bespoke server, and is fully self-hostable — matching the project's SaaS-in-a-box goal
PostgreSQL	A powerful relational engine with the constraints, triggers, RPCs, JSONB and Row-Level Security the certification/progression logic depends on
Row-Level Security (RLS)	Authorization enforced at the database, not just the client — the cornerstone of the security model
REST APIs (Vercel Edge)	<code>runtime: 'edge'</code> functions give low-latency, globally-distributed, auth-gated endpoints for AI and integrations
Deno Edge Functions (Supabase)	Server-side execution for code running, payments and AI in self-hosted deployments
GitHub	Source control and the integration point for Vercel's CI/CD
Vercel	Zero-config static + Edge hosting with built-in cron and rewrites; the primary production target

7.3 AI & execution

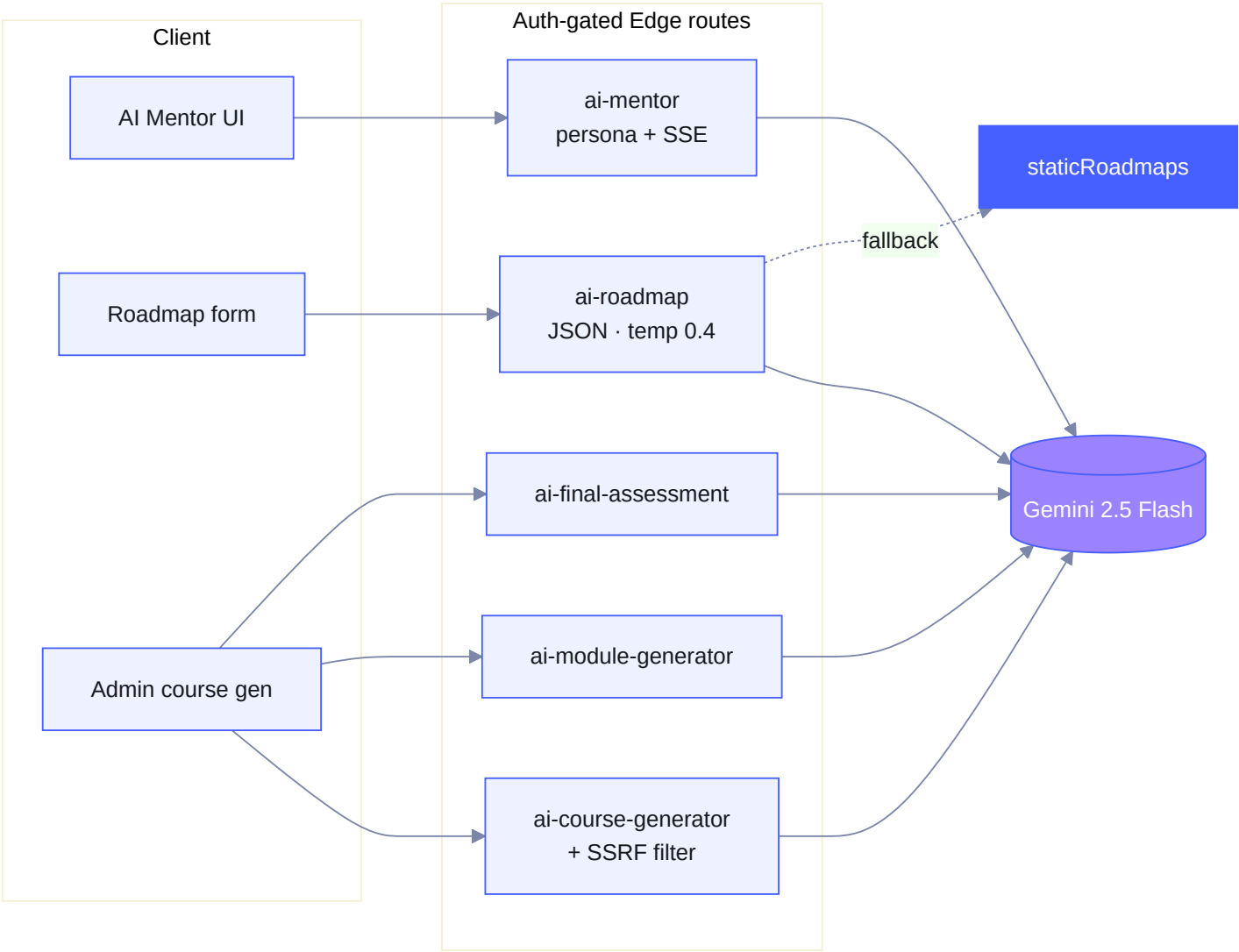
Technology	Why it was selected
Google Gemini 2.5 Flash	A fast, cost-effective, capable model well-suited to streaming tutoring and structured JSON generation (roadmaps, courses, assessments)
Piston	Lightweight, self-hostable multi-language code execution; reached via a Vercel rewrite and a rate-limited Edge function
Razorpay	Payments provider for the India-focused (INR) subscription tiers, with server-side order creation and signature verification

7.4 Tooling & quality

`Biome` (lint/format), `Vitest` (testing), `vite-plugin-svg` (SVG-as-component), and strict TypeScript configs (`tsconfig.check.json`) round out the developer experience. The repository also ships multiple internal reports (`SECURITY_REPORT.md` , `PRODUCTION_VERIFICATION_REPORT.md` , `CHANGELOG.md`) reflecting an audit-driven development process.

8. AI Architecture

AI in LearnLoom is **server-mediated and purpose-built per task** rather than a single generic chatbot. Five distinct AI capabilities each have their own prompt, endpoint and output contract.



8.1 AI Mentor

A **streaming tutor**. The prompt fixes a tutor persona and an academic-integrity rule (never give exam answers directly). The Edge route calls Gemini's `streamGenerateContent` with server-sent events and pipes the stream to the client for a live "typing" experience. Auth is mandatory (Bearer JWT verified against Supabase) before any tokens are spent.

8.2 AI Roadmap

A **structured generator**. Gemini is prompted as an EdTech curriculum designer and constrained to emit raw JSON (`responseMimeType: 'application/json'` , low temperature for determinism). The output schema — `title` , `description` , `estimated_weeks` , `phases[] {phase, title, duration_weeks, topics[], assignments[]}` — is fixed so the client can render and persist it reliably. A **static fallback** guarantees the feature degrades gracefully if the model is unavailable.

8.3 Course & module generation

`ai-course-generator` and `ai-module-generator` let administrators produce structured course content with Gemini. Because the course generator can accept user-provided URLs, it runs every target through the **SSRF filter** (`_shared/ssrf.ts`) before fetching, blocking internal/metadata addresses.

8.4 Final assessment generation

`ai-final-assessment` generates assessment content for the certification gate, again behind auth and structured output.

8.5 Prompt engineering principles (observed in code)

- **Role priming** — each prompt opens by assigning Gemini an expert persona.
- **Strict output contracts** — JSON-only responses with explicit schemas and fence-stripping.
- **Determinism where it matters** — low temperature (0.4) for roadmaps to keep structure stable.
- **Safety guardrails** — the mentor refuses to leak exam answers; generators are SSRF-filtered.
- **Graceful degradation** — static fallbacks so a model outage doesn't break the UX.

8.6 Personalization & adaptivity

Personalization is driven by explicit learner inputs (domain, role, difficulty) plus the learner's own data (enrolments, progress, attempts). *Adaptive learning* in the verified codebase takes the form of **dynamic requirement mapping** (the coding gate appears only when a course has coding questions) and difficulty-scaled roadmap durations, rather than a model that re-weights content per learner in real time. This is reported precisely rather than overstated.

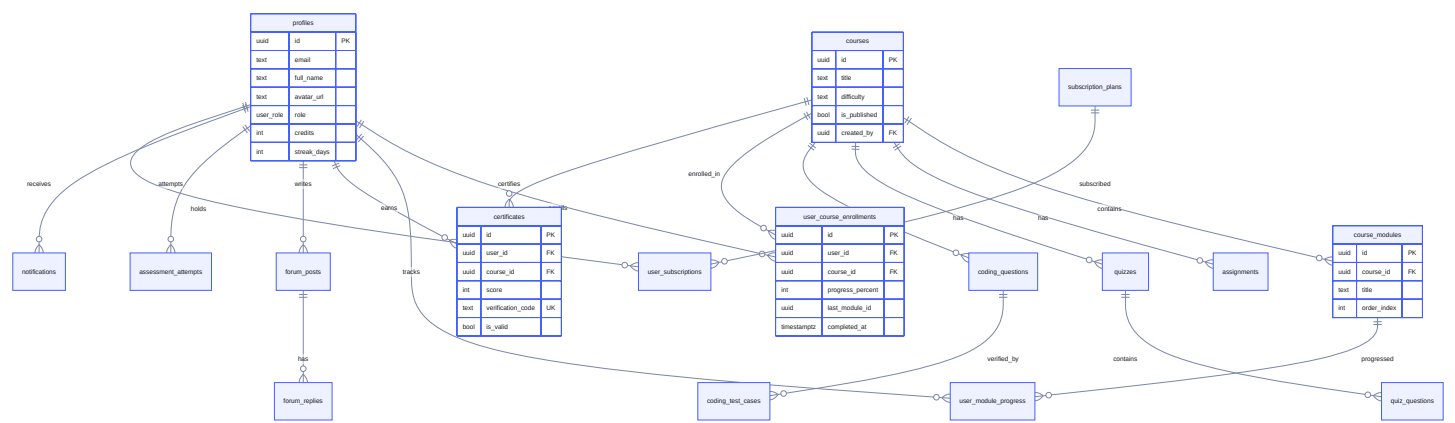
8.7 Learning analytics

Activity logging (`logUserActivity` → `user_activity_logs` / `daily_activity`), enrolment-trend RPCs (`get_admin_enrollment_trends`) and the admin stats RPC (`get_admin_stats`) feed Recharts visualizations in the admin console, closing the loop from learner behaviour to operator insight.

9. Database Design

The schema is defined across **48 SQL migrations** in `supabase/migrations/` , applied in order. It is a relational PostgreSQL design with extensive Row-Level Security, triggers, RPCs and derived views.

9.1 Core entity-relationship diagram



9.2 Key tables explained

Table	Purpose	Notable columns / rules
<code>profiles</code>	Mirror of <code>auth.users</code> + app data	<code>role</code> (enum), <code>credits</code> , <code>streak_days</code> ; auto-created by <code>handle_new_user()</code> trigger
<code>courses</code>	Course catalogue	<code>difficulty</code> CHECK enum, <code>is_published</code> (RLS gate), <code>rating</code> , <code>category</code> , <code>student_count</code>
<code>course_modules</code>	Ordered lessons	<code>order_index</code> (unified ordering), video & notes URLs, rich JSONB fields
<code>user_course_enrollments</code>	Learner ↔ course link	<code>progress_percent</code> (0–100 CHECK), <code>last_module_id</code> (resume), <code>completed_at</code> (certified)
<code>user_module_progress</code>	Per-module state	<code>module_status</code> enum <code>locked/unlocked/completed</code> ; <code>UNIQUE(user_id, module_id)</code>
<code>quizzes</code> / <code>quiz_questions</code>	MCQs	options as JSONB, <code>answer_index</code>
<code>coding_questions</code> / <code>coding_test_cases</code>	Coding problems	<code>starter_code</code> JSONB per language, <code>is_assessment</code> , <code>is_hidden</code> test cases
<code>assessment_attempts</code>	Exam attempts	<code>score_percentage</code> , <code>is_passed</code> , <code>attempt_number</code> , <code>metrics</code> JSONB
<code>grand_test_questions</code> / <code>grand_test_attempts</code>	Global test	cooldown + auto-certificate triggers
<code>certificates</code>	Issued credentials	<code>UNIQUE verification_code</code> , <code>is_valid</code> , <code>UNIQUE(user_id, course_id)</code>
<code>forum_posts</code> / <code>forum_replies</code>	Community	<code>category</code> CHECK, <code>tags[]</code> , atomic vote RPCs
<code>notifications</code>	In-app alerts	<code>type</code> , <code>message</code> , <code>read</code> ; owner-scoped RLS

Table	Purpose	Notable columns / rules
<code>roadmaps</code> / <code>roadmap_templates</code>	Roadmap data	phases as JSONB; static fallback source
<code>subscription_plans</code> / <code>user_subscriptions</code> / <code>payment_orders</code>	Billing	INR pricing, monthly credits, Razorpay orders
<code>user_activity_logs</code> / <code>daily_activity</code>	Analytics	feeds admin trends/charts
<code>organizations</code> / <code>user_organizations</code>	Multi-tenant grouping	added in RBAC/organizations migration
<code>audit_logs</code> / <code>admin_audit_logs</code> / <code>automation_logs</code>	Auditing	administrative and automation traceability

9.3 Relationships, integrity & indexes

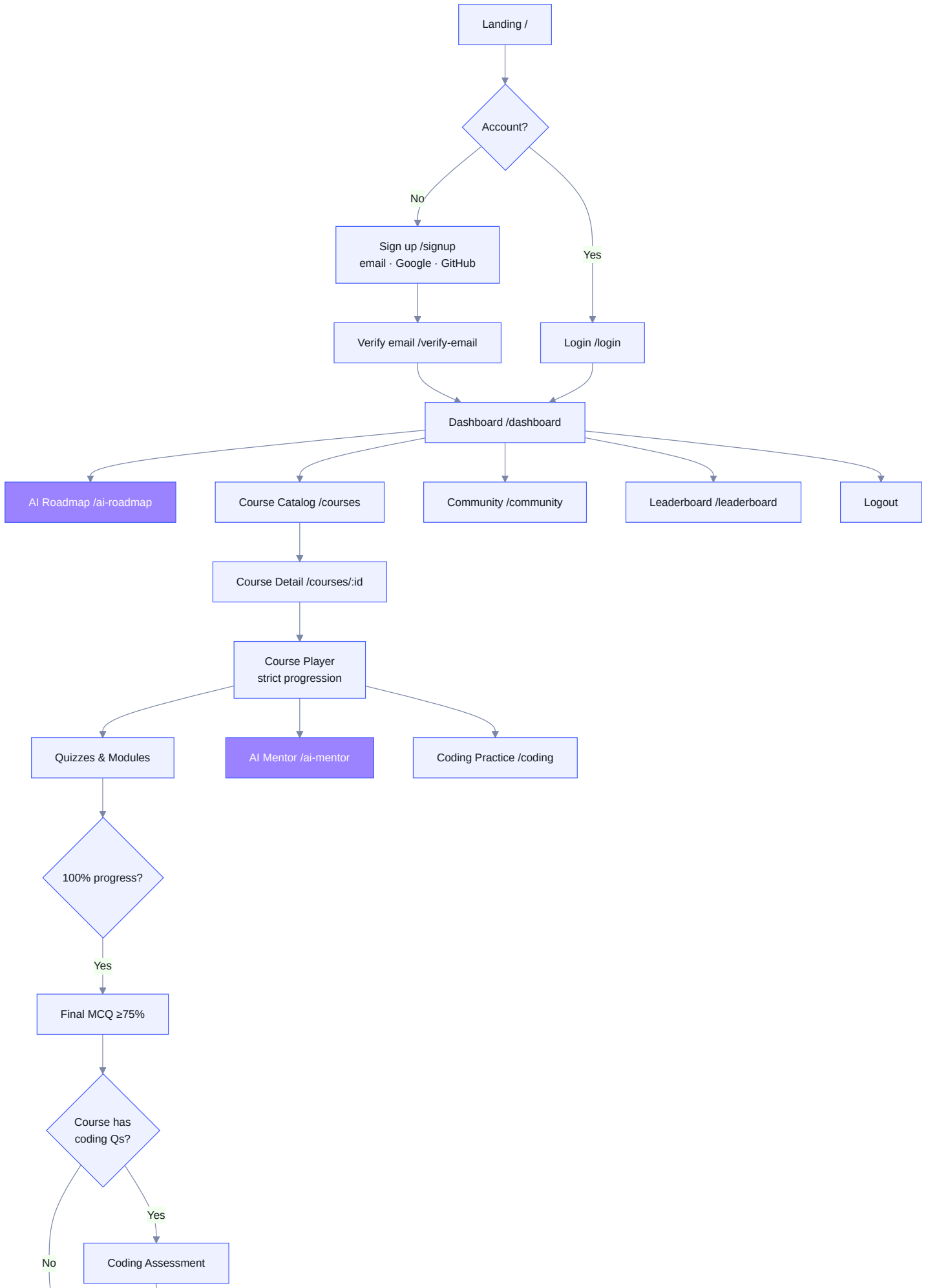
- **Referential integrity** via foreign keys with `ON DELETE CASCADE` (e.g. deleting a course removes its modules, assignments, quizzes and enrolments) or `ON DELETE SET NULL` (e.g. a course's `created_by`).
- **Uniqueness** enforced where it matters: `certificates.verification_code` , `UNIQUE(user_id, course_id)` on enrolments and certificates, `UNIQUE(user_id, module_id)` on module progress.
- **Indexes** on hot paths: `certificates(user_id)` , `certificates(verification_code)` , `forum_posts(user_id)` , `forum_posts(created_at DESC)` , `forum_posts(category)` , and composite rate-limit indexes on `auth_rate_limit(user_id, endpoint)` (migration `00019`).
- **Derived views**: `leaderboard_view` (ranking) and `public_profiles` (safe public fields).

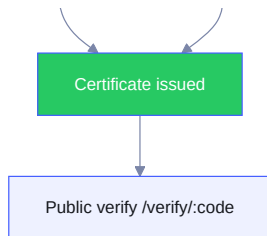
9.4 Security at the data layer

Row-Level Security is enabled on essentially every table. A `SECURITY DEFINER` helper `get_user_role(uid)` (and `is_admin(uid)`) lets policies check roles without recursive RLS evaluation. Representative policies: learners read only published courses; learners read/write only their own enrolments, attempts, certificates and notifications; admins get full access through role-checked policies. Storage buckets carry their own object-level RLS (public read for avatars/course-assets; owner+admin only for submissions). This is covered further in Security.

10. User Flows

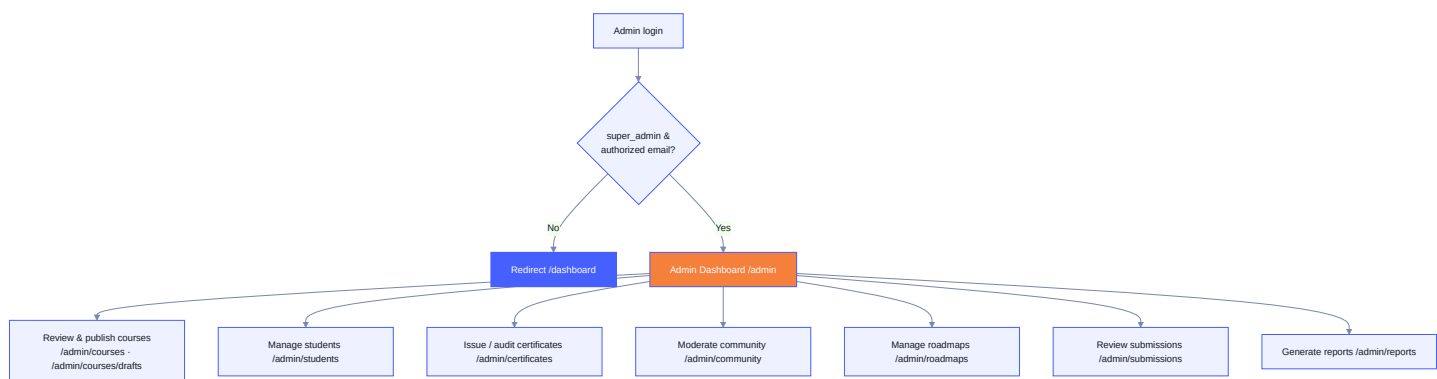
10.1 Student journey





A learner lands, registers (verifying their email or using OAuth), and arrives at a personalized **dashboard**. They can generate a **roadmap**, browse the **catalog**, enrol, and learn through the **course player** under strict progression — pausing to ask the **AI mentor** or to **practise code**. At 100% progress the **assessment gate** opens; passing the MCQ (and the coding exam where applicable) issues a **verifiable certificate**. Throughout, the **community** and **leaderboard** keep them engaged, and **logout** ends the session cleanly.

10.2 Admin journey



The admin signs in; **RouteGuard** verifies both the authorized email **and** the **super_admin** role before granting **/admin/*** (everyone else is redirected). From the **dashboard** (driven by **get_admin_stats** and trend RPCs) the admin reviews and publishes courses — including **AI-generated drafts** — manages **students**, audits and issues **certificates**, moderates the **community**, manages **roadmaps**, reviews **submissions**, and reads **reports/analytics**.

11. Security

Security is a first-class concern in this codebase, documented in the README and **SECURITY_REPORT.md** and verifiable in source. The model is **defense-in-depth**: the client is treated as untrusted, every privileged action is authenticated server-side, and the database enforces authorization independently via RLS.

11.1 Authentication

Supabase Auth issues JWT sessions for email and OAuth (Google/GitHub) sign-in. Profiles are provisioned by a **SECURITY DEFINER** trigger, and the client tolerates provisioning latency with bounded polling and a callback safety timeout.

11.2 Authorization & Row-Level Security

Authorization lives in the database. RLS is enabled across tables; policies use `get_user_role()` / `is_admin()` to grant learners access to **only their own rows** and admins to role-checked full access. Even if a client tried to query another user's certificates or attempts, Postgres would return nothing. The README highlights hardened policies on `certificates` and `assessment_attempts` (`auth.uid()::text = user_id::text` , no wildcard inserts).

11.3 Protected routes & role-based access

Client-side, `RouteGuard` classifies routes as public, student or admin; unauthenticated users are bounced to `/login` , and `/admin/*` is restricted to the single authorized `super_admin` account, with unauthorized elevated roles defensively downgraded.

11.4 API security

Every Vercel Edge AI route calls `verifyAuth()` first, which validates the Bearer token against Supabase's `/auth/v1/user` endpoint and returns **401** otherwise. This prevents anonymous use of (paid) AI endpoints and credit exhaustion.

11.5 Secret management

Upstream API keys (Gemini, Razorpay, Judge0) are read from server environment only and **never shipped to the browser**. All client → model traffic is proxied through the authenticated Edge layer, so keys cannot be extracted from the production JavaScript bundle.

11.6 SSRF protection

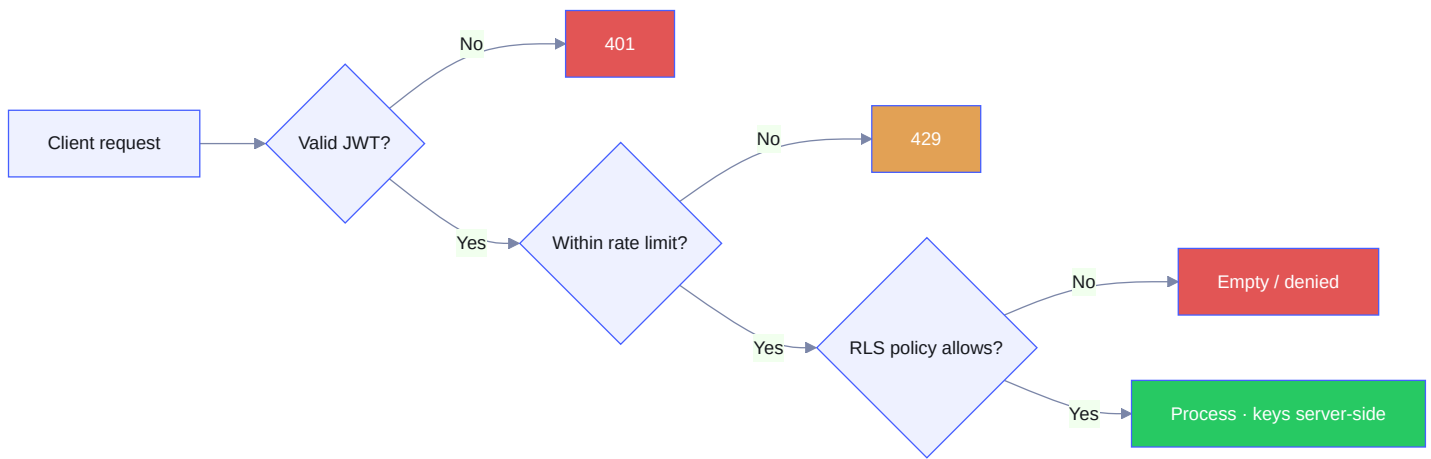
URL-accepting AI routes pass targets through `_shared/ssrf.ts` , which permits only `http(s)` and blocks loopback, RFC-1918 private ranges, link-local and cloud-metadata addresses (e.g. `169.254.169.254`), preventing the server from being tricked into requesting internal resources.

11.7 Input validation & XSS sanitization

Forms validate with `react-hook-form` + `zod` ; the database adds `CHECK` constraints (e.g. forum title/length, score ranges, difficulty enums). LLM- or user-generated HTML is run through a `sanitizeHtml()` utility (`src/lib/sanitize.ts`) before any `dangerouslySetInnerHTML` , stripping scripts, `javascript:` URIs and dangerous embeds. A `dompurify` override is pinned in `package.json` .

11.8 Rate limiting

The `execute-code` Edge function enforces **10 executions/minute/user**, and an `auth_rate_limit` table with composite indexes guards authentication endpoints — both defending against abuse and resource exhaustion.



11.9 Session handling

Sessions are managed by Supabase Auth and surfaced through `AuthContext`, which subscribes to auth-state changes, keeps refs to avoid stale closures, and exposes a clean `signOut()`.

12. Performance

12.1 Frontend optimization

- **Code splitting / lazy loading** — every route is `React.lazy`-loaded (`src/routes.tsx`), so the initial bundle stays small and screens load on demand.
- **Suspense + skeletons** — `PageSkeleton` and component skeletons provide instant perceived feedback while chunks and data load.
- **Debounced input** — the `use-debounce` hook limits expensive operations (search) to reduce re-renders and queries.

12.2 Build optimization

- **Vite** produces tree-shaken, minified, hashed production bundles in `dist/`.
- Two Vite configs (`vite.config.ts` , `vite.config.dev.ts`) separate production and development concerns.
- `vite-plugin-svg` imports SVGs as components, avoiding extra network requests for icons.

12.3 Data & query optimization

- **Indexes** on frequently-filtered columns (certificates, forum posts, rate limits) keep queries fast at scale.
- **Derived views** (`leaderboard_view`) compute rankings in the database rather than the client.
- **Parallel fetching** — the admin dashboard issues five queries via `Promise.all`, minimizing round-trip latency.
- **Atomic RPCs** (`increment_credits` , `toggle_reply_vote`) avoid read-modify-write races.

12.4 Edge & caching

- Vercel **Edge runtime** places AI proxy routes close to users for low latency.
 - AI mentor responses **stream** (SSE), so the learner sees output immediately instead of waiting for a full completion.
 - Static assets are served from Vercel's CDN with content hashing for long-term caching.
-

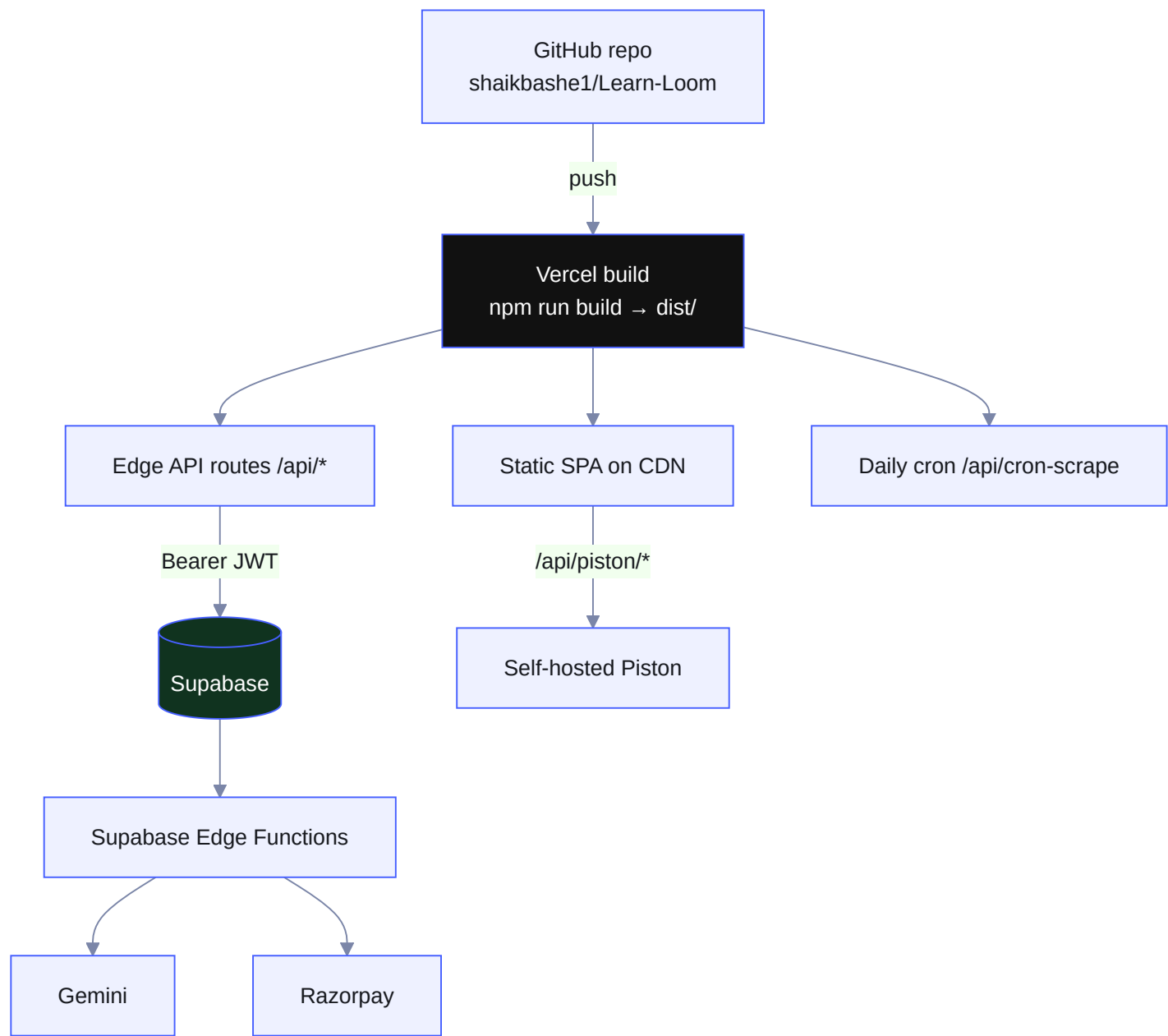
13. Accessibility

- **Responsive design** — Tailwind plus `@tailwindcss/container-queries` and a `use-mobile` hook; the README states the UI is *audited down to 320px with strict touch-target compliance*.
- **Mobile compatibility** — a `manifest.json` and theme/touch handling support phone use; the layout adapts from mobile to desktop.
- **Keyboard navigation & semantics** — the UI is built on **Radix UI** primitives, which ship with accessible focus management, ARIA attributes and keyboard interaction by default (dialogs, menus, tabs, toasts, etc.).
- **Color contrast & theming** — `next-themes` provides dark/light modes; the design tokens in `tailwind.config.js` and `src/index.css` centralize colors for consistent, controllable contrast.

Honesty note: a formal WCAG conformance audit result was not found in the repository. The above reflects accessible **foundations** (Radix, responsive tokens, theming) rather than a certified conformance level. A dedicated audit is recommended (see Future Enhancements).

14. Deployment

14.1 Deployment architecture



14.2 Vercel (primary)

`vercel.json` defines two **rewrites** — `/api/piston/*` proxies to a self-hosted Piston instance, and a SPA catch-all rewrites non-API paths to `/index.html` — plus a **daily cron** at midnight (`0 0 * * *`) hitting `/api/cron-scrape`. Build command `npm run build`, output `dist/`, framework preset Vite. The required environment variables are `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` (client), with server secrets like `INTEGRATIONS_API_KEY` set in the Vercel/Supabase environment.

14.3 Supabase

The backend is provisioned by pushing the `supabase/migrations/` in order (CLI `supabase db push` or via the SQL editor), creating the three storage buckets (`avatars` , `submissions` , `course-assets`), deploying the Edge Functions, and setting secrets (e.g. `INTEGRATIONS_API_KEY`). Google OAuth is configured in the Supabase Auth providers panel.

14.4 Alternative targets

The README documents **Render** (static site) and **self-hosted Nginx/Docker** deployments, and a `netlify.toml` is present for Netlify. Because the production artifact is a static bundle plus serverless functions, hosting is inexpensive and portable.

14.5 Environment variables

Variable	Scope	Purpose
<code>VITE_SUPABASE_URL</code>	Client	Supabase project URL
<code>VITE_SUPABASE_ANON_KEY</code>	Client	Public anon key for supabase-js
<code>INTEGRATIONS_API_KEY</code> / <code>GEMINI_API_KEY</code>	Server	Gemini API key (Edge routes/functions)
<code>PISTON_URL</code> / <code>PISTON_API_URL</code>	Server	Code-execution endpoint
<code>RAZORPAY_KEY_ID</code> / <code>RAZORPAY_KEY_SECRET</code>	Server	Payments
<code>JUDGE0_API_KEY</code>	Server	Alternative execution provider

15. Installation Guide

Prerequisites: `Node.js` $\geq 20.x$ and `npm` $\geq 10.x$ (per the README), plus a Supabase project.

1. Clone the repository

```
git clone https://github.com/shaikbashe1/Learn-Loom.git
cd Learn-Loom
```

2. Install dependencies

```
npm install
```

3. Configure environment variables

```
cp .env.example .env
# then set:
# VITE_SUPABASE_URL=https://<your-project-ref>.supabase.co
# VITE_SUPABASE_ANON_KEY=<your-anon-key>
```

4. Provision the database

```
npm install -g supabase
supabase link --project-ref <your-supabase-project-ref>
supabase db push          # applies migrations 00001 ... in order
```

Then create the `avatars`, `submissions` and `course-assets` storage buckets with the RLS policies described in the README, deploy the Edge Functions, and set the Gemini secret:

```
supabase functions deploy ai-mentor
supabase functions deploy ai-roadmap
supabase functions deploy large-language-model
supabase secrets set INTEGRATIONS_API_KEY=<your_google_gemini_api_key>
```

5. Run locally

```
npm run dev          # http://localhost:5173
```

6. Build & preview production

```
npm run build        # outputs dist/
npm run preview       # serves the production build locally
```

7. Deploy — connect the repo to Vercel (preset `Vite`, build `npm run build`, output `dist`), add the two `VITE_*` env vars, and deploy. (Render and Nginx instructions are in the README.)

Useful scripts: `npm run lint` (TypeScript check + Biome) and `npm run test` (Vitest).

16. API Documentation

LearnLoom exposes two families of server endpoints: **Vercel Edge API routes** (`api/*.ts` , `runtime: 'edge'`) and **Supabase Edge Functions** (`supabase/functions/*`). All sensitive routes require a Supabase access token in the `Authorization: Bearer <token>` header, validated server-side; an invalid/missing token returns **401**.

16.1 Vercel Edge API routes

Method	Endpoint	Auth	Purpose
POST	/api/ai-mentor	Bearer JWT	Streaming (SSE) tutor responses from Gemini
POST	/api/ai-roadmap	Bearer JWT	Generate a phased roadmap as JSON
POST	/api/ai-course-generator	Bearer JWT	Generate course content (SSRF-filtered)
POST	/api/ai-module-generator	Bearer JWT	Generate module content
POST	/api/ai-final-assessment	Bearer JWT	Generate final-assessment content
GET	/api/models	Bearer JWT	List available Gemini models
GET	/api/cron-scrape	Vercel cron	Scheduled daily content task
*	/api/piston/*	(rewrite)	Proxy to the Piston code-execution engine

Example — AI Mentor (streaming)

Request


```
POST /api/ai-mentor
Authorization: Bearer <supabase-access-token>
Content-Type: application/json

{
  "contents": [
    { "role": "user", "parts": [{ "text": "Explain binary search with a Python example." } ]
  }
}
```

Response — `Content-Type: text/event-stream` ; Gemini tokens are streamed as server-sent events and rendered live by the client.
Missing/invalid token → `401` ; missing body array → `400` .

Example — AI Roadmap

Request

```
POST /api/ai-roadmap
Authorization: Bearer <supabase-access-token>
Content-Type: application/json

{ "domain": "web-development", "role": "Full-Stack Developer", "difficulty": "beginner" }
```

Response `200 application/json`

```
{
  "title": "Full-Stack Web Development Roadmap",
  "description": "A structured path from fundamentals to deployable full-stack apps.",
  "estimated_weeks": 16,
  "phases": [
    { "phase": 1, "title": "Web Foundations", "duration_weeks": 3,
      "topics": ["HTML", "CSS", "JS basics"], "assignments": ["Build a landing page"] }
  ]
}
```

`domain` and `role` are required (else `400`). On model failure the **client** substitutes a static roadmap so the user still receives a plan.

16.2 Supabase Edge Functions (Deno)

Function	Purpose	Notes
<code>execute-code</code>	Run learner code via Piston	Token-verified; rate-limited 10/min/user ; maps languages; runs hidden tests
<code>ai-mentor</code>	Tutor (self-hosted path)	Mirrors the Vercel route
<code>ai-roadmap</code>	Roadmap (self-hosted path)	Mirrors the Vercel route
<code>large-language-model</code>	Generic LLM gateway	Deployed per README
<code>create-razorpay-order</code>	Create a payment order	Server-side; no client keys
<code>verify-razorpay-payment</code>	Verify payment signature	Prevents forged confirmations
<code>clerk-webhook</code>	Clerk integration webhook	Legacy/partial auth artifact (see §5.2.3)

16.3 Data access (supabase-js)

Most reads/writes use the Supabase client directly from the browser, **authorized by RLS** — e.g. `supabase.from('courses').select(...)`, `supabase.from('user_course_enrollments')...`, `supabase.rpc('increment_credits', {...})`, `supabase.rpc('get_admin_stats')`. There is no hand-written REST controller layer; PostgreSQL + RLS *is* the data API, which keeps the surface small and the authorization centralized.

17. Challenges Faced

The repository's `CHANGELOG.md`, `SECURITY_REPORT.md`, `REMAINING_ISSUES_REPORT.md` and `PRODUCTION_VERIFICATION_REPORT.md` document an audit-driven build, and the following challenge areas are evidenced directly in those records and the code.

Technical. Coordinating a strict, multi-table progression model (enrolments + per-module status + assessments + certificates) without race conditions required atomic RPCs (`increment_credits`, `toggle_reply_vote`) and careful trigger design. The CHANGELOG also records fixing broken course-player navigation (a missing `:moduleId` segment) and unifying module ordering on `order_index` across multiple files.

Design. Building a consistent glassmorphism system that stays usable and touch-compliant down to 320px, across dark/light themes, demanded a disciplined token system (Tailwind config + CSS variables) and reliance on accessible Radix primitives.

AI. Getting reliable, *structured* output from an LLM is non-trivial: the roadmap route pins a low temperature, forces `application/json`, and strips stray Markdown fences — and still wraps the whole thing in a static fallback for when the model misbehaves or is unavailable.

Deployment. Splitting responsibilities across Vercel (SPA + Edge + cron), Supabase (data + auth + Edge Functions) and a self-hosted Piston instance means several environments and secret stores must line up; `vercel.json` rewrites and documented env-var matrices address this.

Authentication. OAuth introduced an asynchronous race — the profile row is created by a trigger *after* the session exists — solved with bounded polling (5×500 ms) and a 10-second callback safety timeout to prevent infinite loaders. A parallel Clerk integration also left artifacts that had to be reconciled around Supabase Auth as the primary path.

Performance. Keeping a feature-rich SPA fast required route-level code splitting, debounced search, parallelized dashboard queries and database-side ranking via views rather than client-side computation.

18. Future Enhancements

The following extend the verified platform along its natural axis — from learning to career-readiness. Some already have schema scaffolding in the repository (noted inline).

Enhancement	Description	Repo signal
AI Interview Coach	Mock interviews with feedback, extending the mentor persona	Premium plan lists "AI Mock Interviewer"
Résumé Analyzer	Parse and score résumés against target roles	<code>resumes</code> , <code>resume_analysis</code> tables exist
Voice Tutor	Speech in/out for the AI mentor	builds on existing mentor stream
Mobile App	Installable PWA / native wrapper	<code>manifest.json</code> already shipped
Live Collaboration	Realtime pair-learning and study rooms	Supabase Realtime available
Gamification Expansion	Badges, seasons, streak-freezes, cohort contests	<code>reward_configurations</code> table present
Adaptive Assessments	Difficulty that adapts to performance	extends <code>assessment_attempts.metrics</code>
Multilingual Support	Localized UI and AI responses	i18n layer to be added
Learning Analytics Dashboard	Deeper learner-facing insights	<code>user_activity_logs</code> / <code>daily_activity</code> feed it

Additional natural steps: linking roadmap phases directly to in-platform courses, AI hints on failing test cases, verifiable-credential standards for certificates, and a formal WCAG audit.

19. Impact

- Educational impact.** By enforcing structured progression and an assessment gate, LearnLoom shifts learners from passive consumption to demonstrated competence. The AI roadmap removes the single biggest blocker to self-study — not knowing where to start — and the mentor keeps learners unblocked, both of which target the high drop-off rates that plague online education.
- Technical impact.** The project is a reference example of a modern, secure, AI-integrated EdTech stack: a static React/Vite SPA, serverless Edge functions, Supabase with thorough RLS, and server-mediated LLM usage with structured outputs and graceful fallbacks. Its defense-in-depth security (JWT-gated APIs, no client keys, SSRF filtering, sanitization, rate limiting, RLS) is a template others can learn from.
- Social impact.** Verifiable certificates make learning outcomes portable and trustworthy, which can widen access to opportunity for self-taught learners who lack formal credentials. The community forum and leaderboard foster peer support rather than isolated study.
- Business impact.** A self-hostable, low-cost (static + serverless) architecture with a clear credit-based subscription model (Free/Pro/Premium via Razorpay) makes LearnLoom viable to operate and to monetize — whether as a SaaS product or deployed privately by an institution.

20. Conclusion

LearnLoom is not a single feature dressed up as a platform; it is a **complete, AI-powered learning ecosystem** in which every stage of the learner's journey is connected and, crucially, *verifiable*. A learner moves from an ambition to a generated roadmap, through structured courses governed by strict progression, supported by a streaming AI mentor and a real coding sandbox, to a rigorous assessment gate, and finally to a publicly verifiable certificate — all inside one coherent product, all backed by a database that enforces correctness and authorization at its core.

What makes it innovative is the *integration*: AI personalizes the path and the tutoring; gamification sustains motivation; the community adds peer learning; and the security model treats the whole thing as production software rather than a demo. The result directly attacks the real failure modes of online education — lack of structure, weak personalization, poor engagement and non-credible outcomes — and replaces them with a guided, motivating, provable learning loop.

For judges and investors, the takeaway is that LearnLoom demonstrates both **product vision** (a unified learning ecosystem) and **engineering maturity** (RLS-first data design, server-mediated AI, audited security, graceful degradation). For developers, it is a well-structured, modern codebase that is genuinely self-hostable. And for learners, it is a place where finishing a course finally *means* something.

21. References

Project - GitHub repository — <https://github.com/shaikbashe1/Learn-Loom> - Live demo — <https://learn-loom-indol.vercel.app> - In-repo docs — [README.md](#) , [CHANGELOG.md](#) , [SECURITY_REPORT.md](#) , [PRODUCTION_VERIFICATION_REPORT.md](#) , [REMAINING_ISSUES_REPORT.md](#) , [docs/prd.md](#)

Core frameworks & libraries - React — <https://react.dev> - TypeScript — <https://www.typescriptlang.org> - Vite — <https://vitejs.dev> - React Router — <https://reactrouter.com> - Tailwind CSS — <https://tailwindcss.com> - Radix UI / shadcn-ui — <https://www.radix-ui.com> · <https://ui.shadcn.com> - Lucide Icons — <https://lucide.dev> - Recharts — <https://recharts.org> - react-hook-form — <https://react-hook-form.com> · Zod — <https://zod.dev>

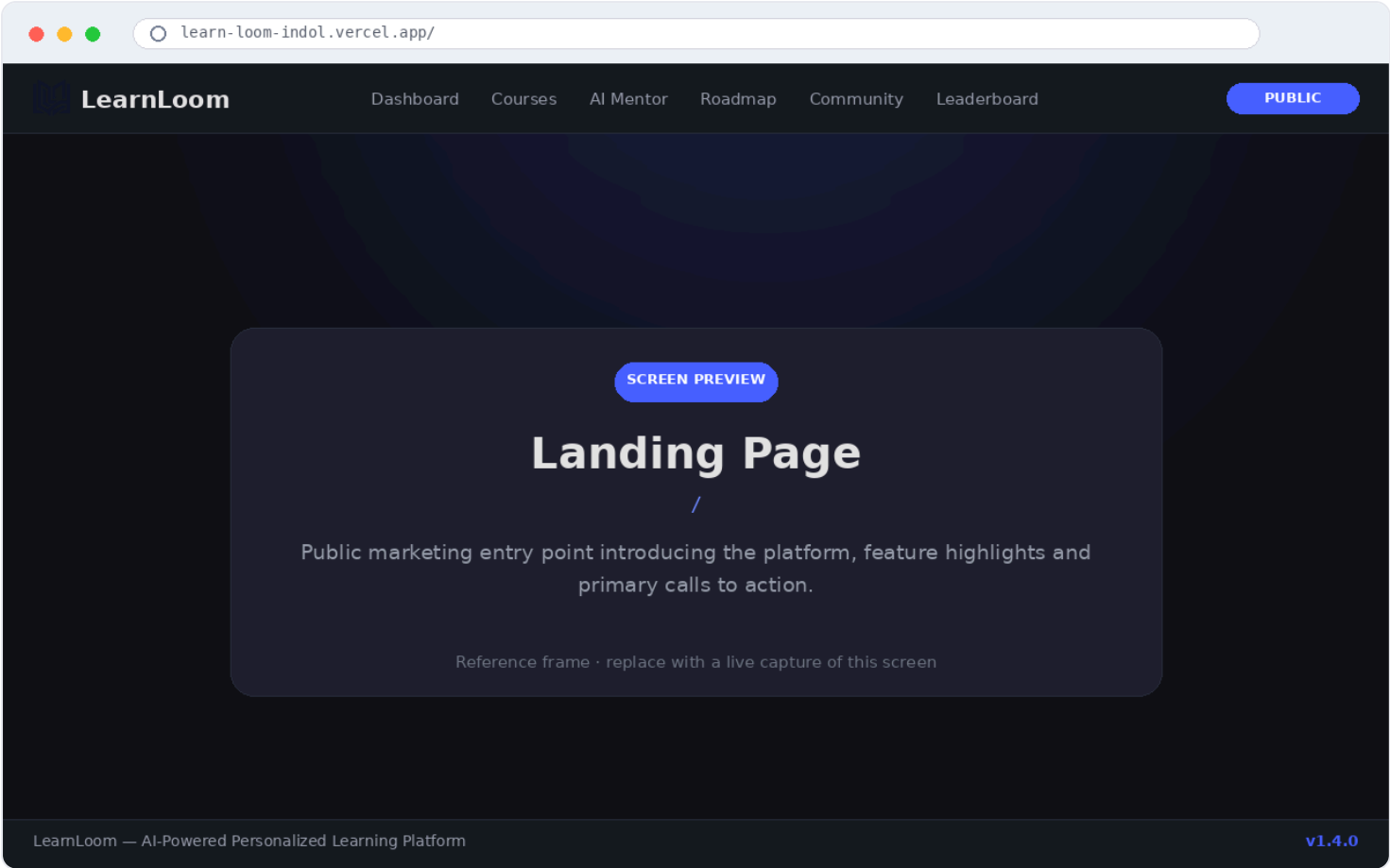
Backend, AI & services - Supabase — <https://supabase.com> - PostgreSQL — <https://www.postgresql.org> - Google Gemini API — <https://ai.google.dev> - Piston — <https://github.com/engineer-man/piston> · Judge0 — <https://judge0.com> - Razorpay — <https://razorpay.com> - Vercel — <https://vercel.com>

Library versions referenced throughout are taken from [package.json](#) in the repository.

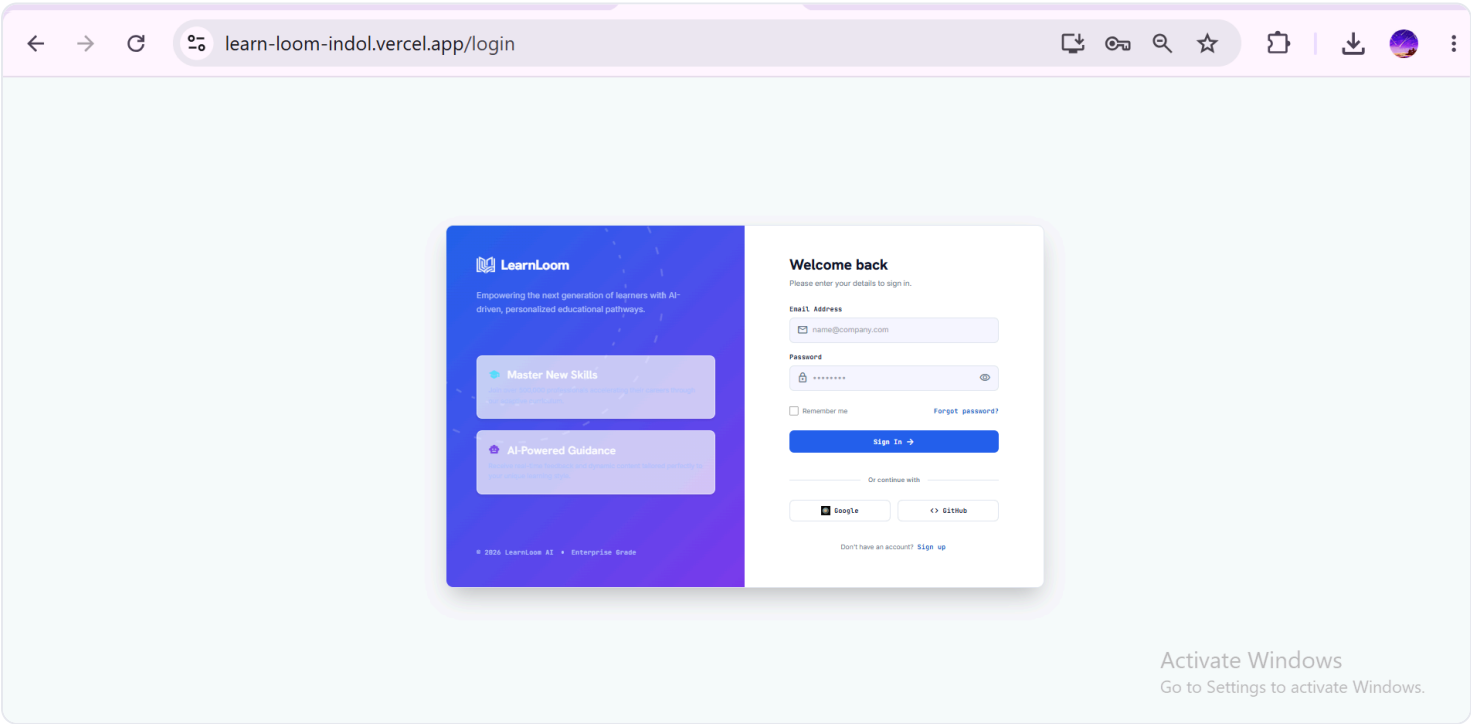
22. Screenshots Gallery

About this gallery. Several screens — Login, Google sign-in, Course Catalog, AI Mentor, AI Roadmap, Coding IDE, Admin Dashboard, Course Moderation, Student Directory, Certificate Management and Community Moderation — are **live captures from the deployed application**. The remaining entries are **brand-accurate reference frames** that label each screen's exact route and purpose; replace any [assets/NN-*.png](#) with a real capture of the matching route on <https://learn-loom-indol.vercel.app> — the filenames and captions are already mapped 1:1, so swapping images requires no edits to this document.

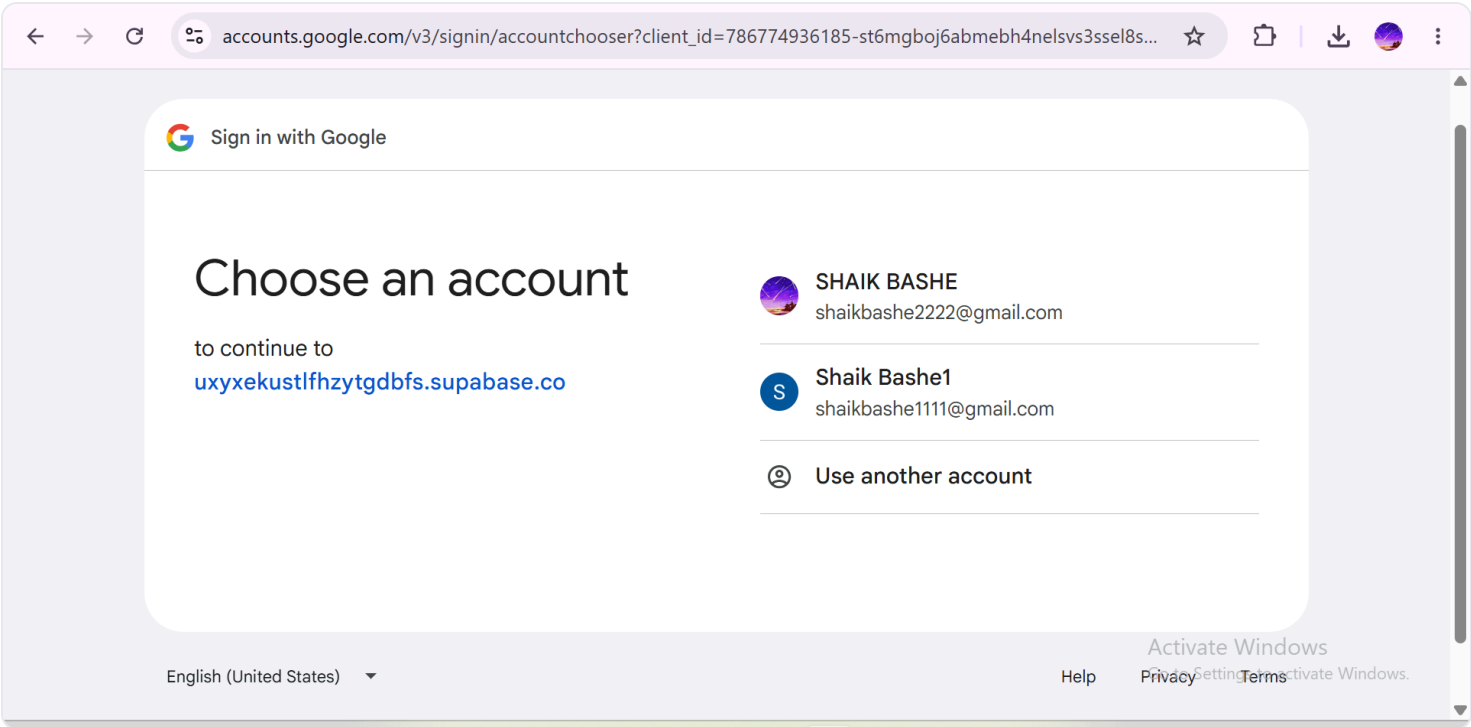
Public & authentication



Title: Landing Page · **Route:** / **Description:** Public marketing entry introducing the platform and its features. **Purpose:** Convert visitors into registered learners.

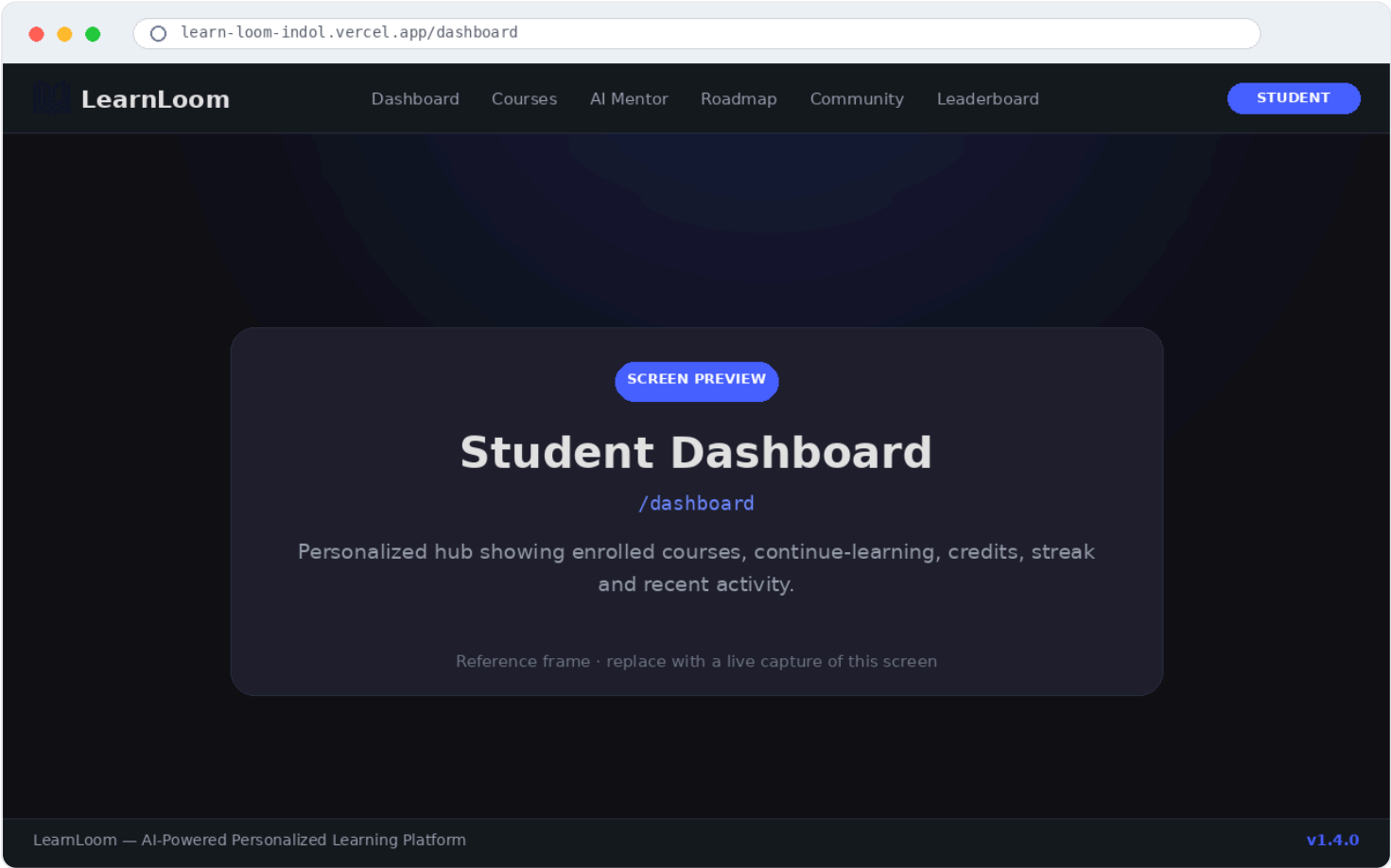


Title: Login · **Route:** /login **Description:** Email/password sign-in with Google and GitHub OAuth. **Purpose:** Authenticate returning users and route them onward.

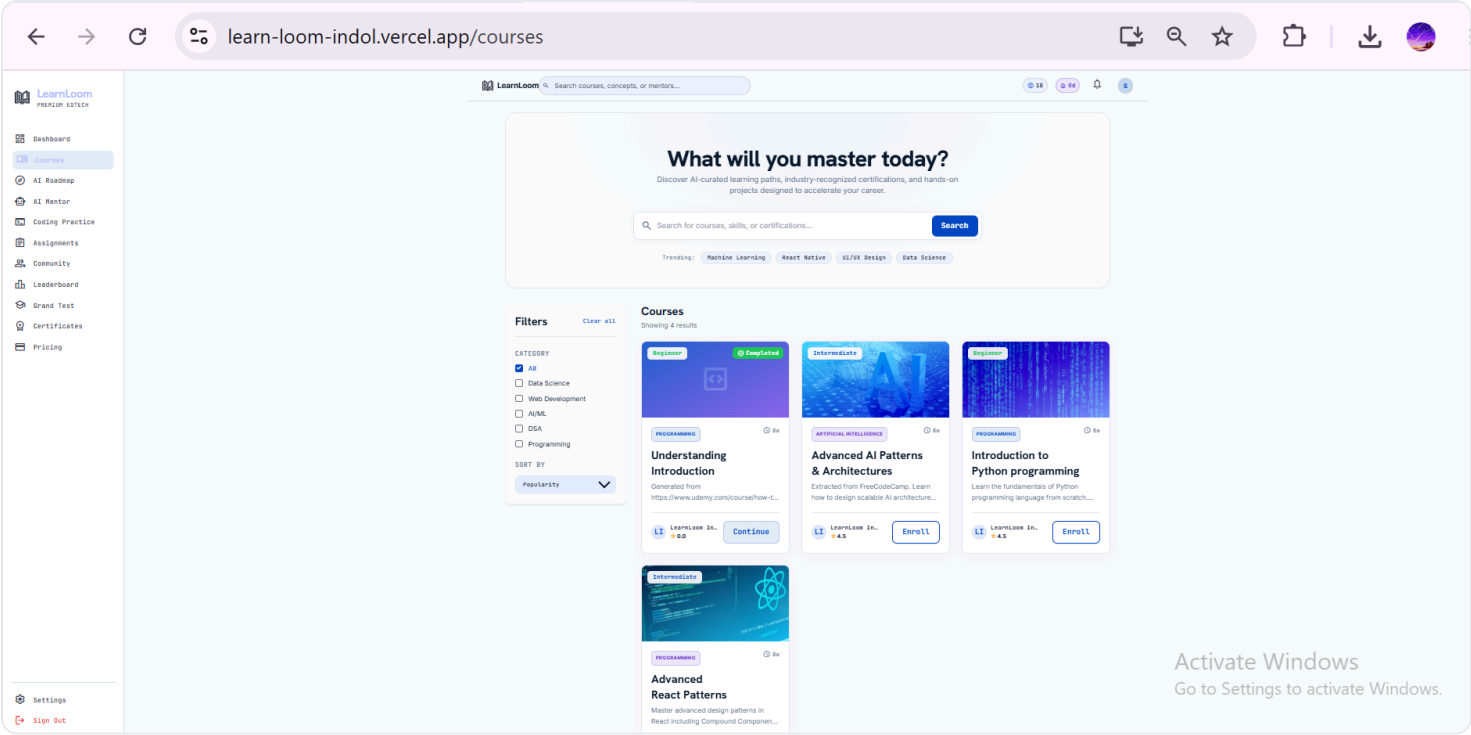


Title: Google Authentication · **Route:** `/auth/callback` **Description:** Supabase-mediated Google OAuth returning to the callback handler.
Purpose: Frictionless one-click sign-in with automatic profile provisioning.

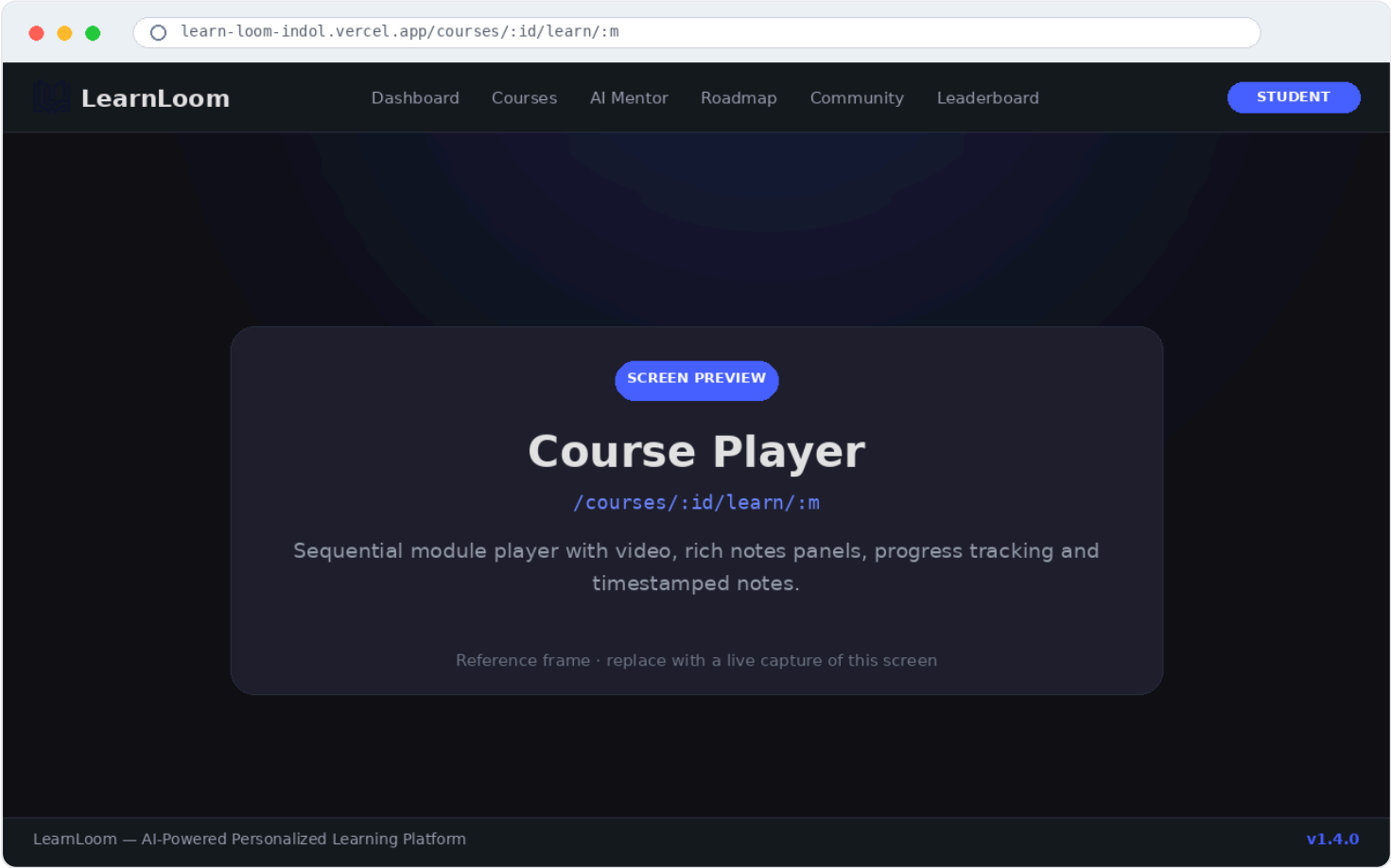
Learner core



Title: Student Dashboard · **Route:** `/dashboard` **Description:** Personalized hub — enrolled courses, continue-learning, credits, streak, activity. **Purpose:** Give each learner an at-a-glance "what to do next."

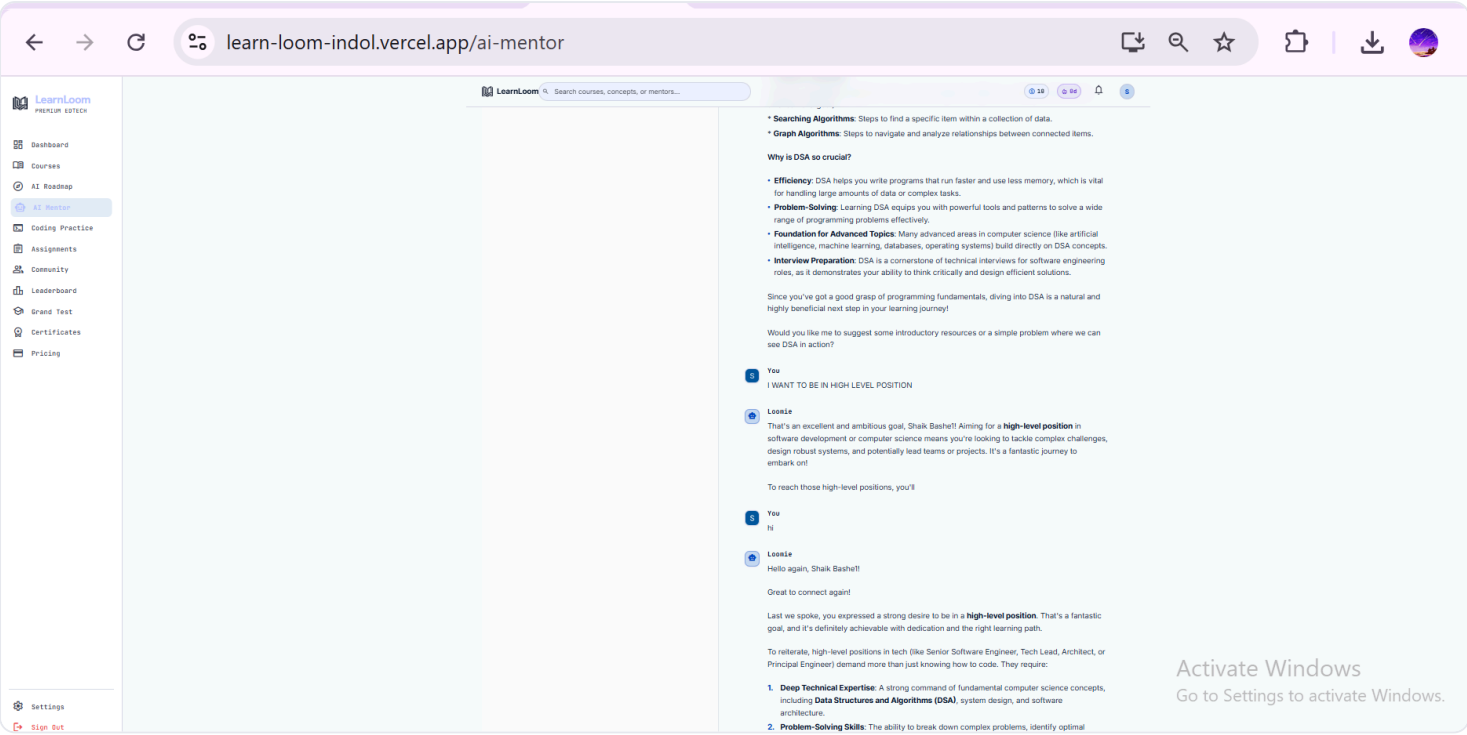


Title: Course Catalog · **Route:** `/courses` **Description:** Searchable, filterable catalogue of published courses with ratings. **Purpose:** Course discovery and enrolment.

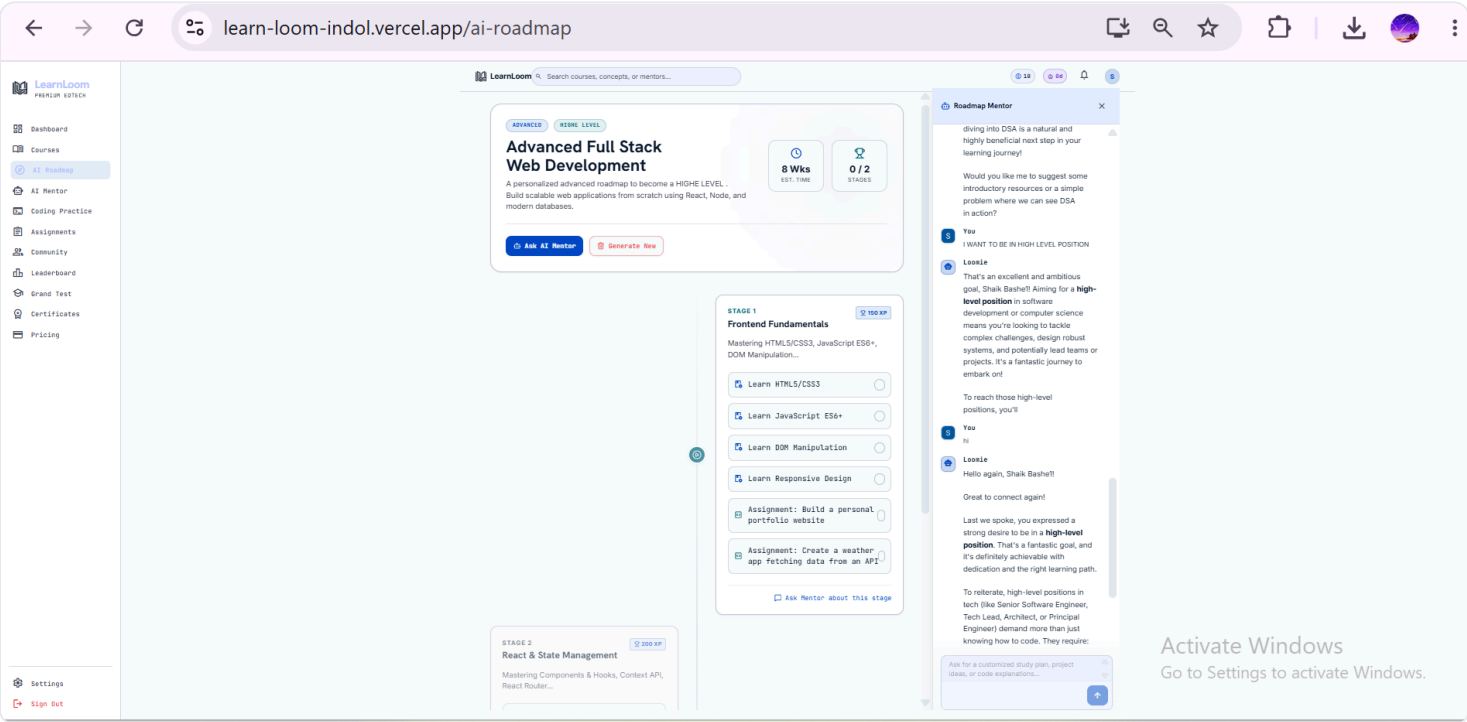


Title: Course Player · **Route:** `/courses/:id/learn/:moduleId` **Description:** Sequential module player with video, notes and progress tracking. **Purpose:** Deliver content under strict, trackable progression.

AI features

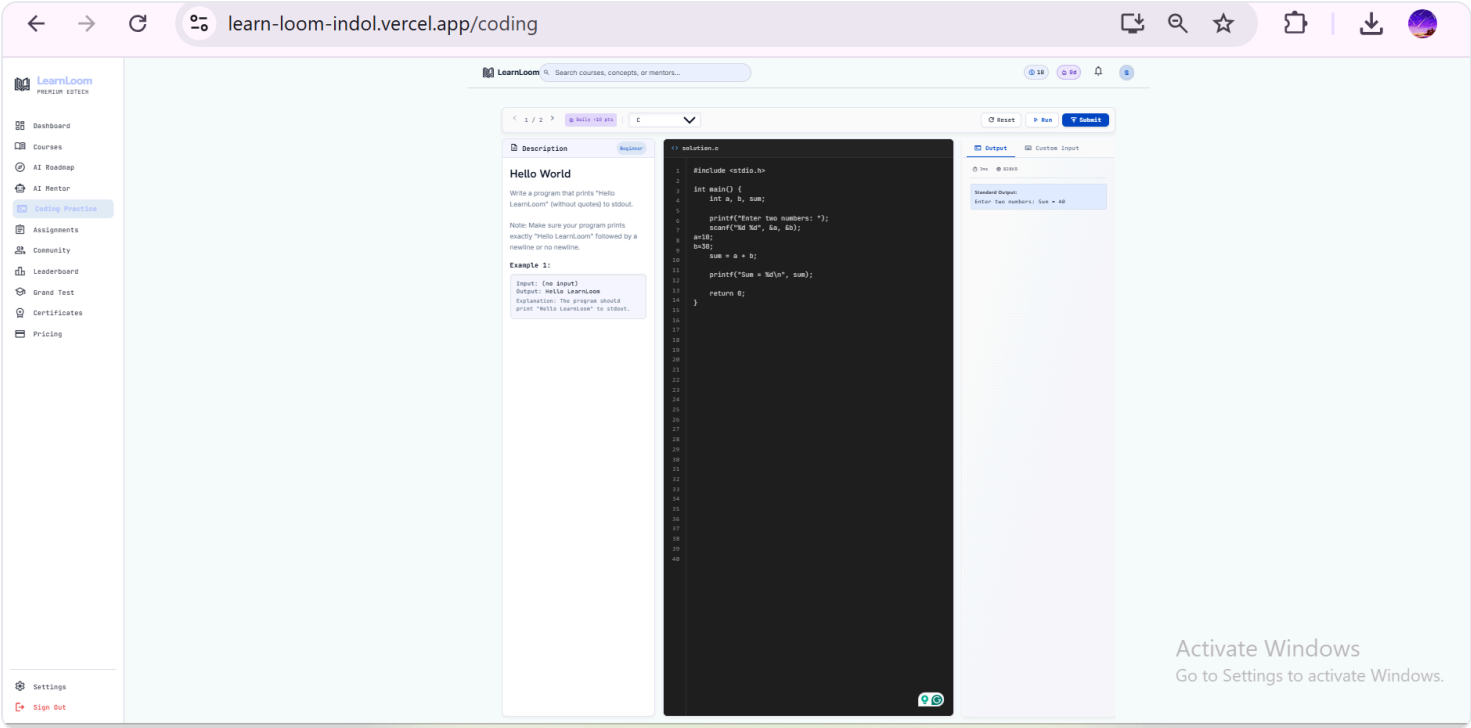


Title: AI Mentor · **Route:** /ai-mentor **Description:** Streaming Gemini-powered tutor for coding, DSA and interview prep. **Purpose:** Keep learners unblocked with contextual, real-time help.

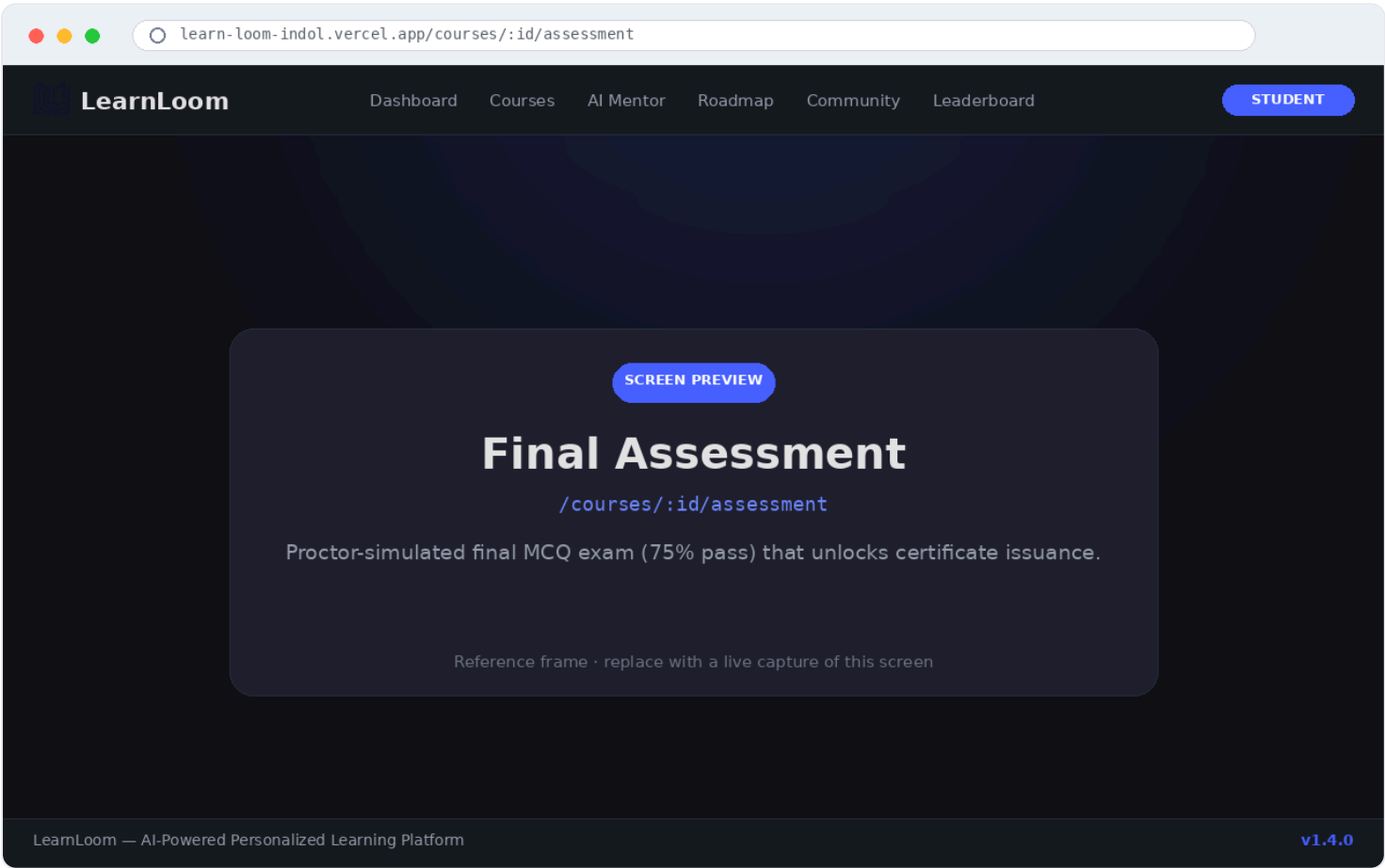


Title: Personalized AI Roadmap · **Route:** /ai-roadmap **Description:** Phased, week-estimated plan generated from domain, role and level. **Purpose:** Turn a career goal into a concrete, ordered path.

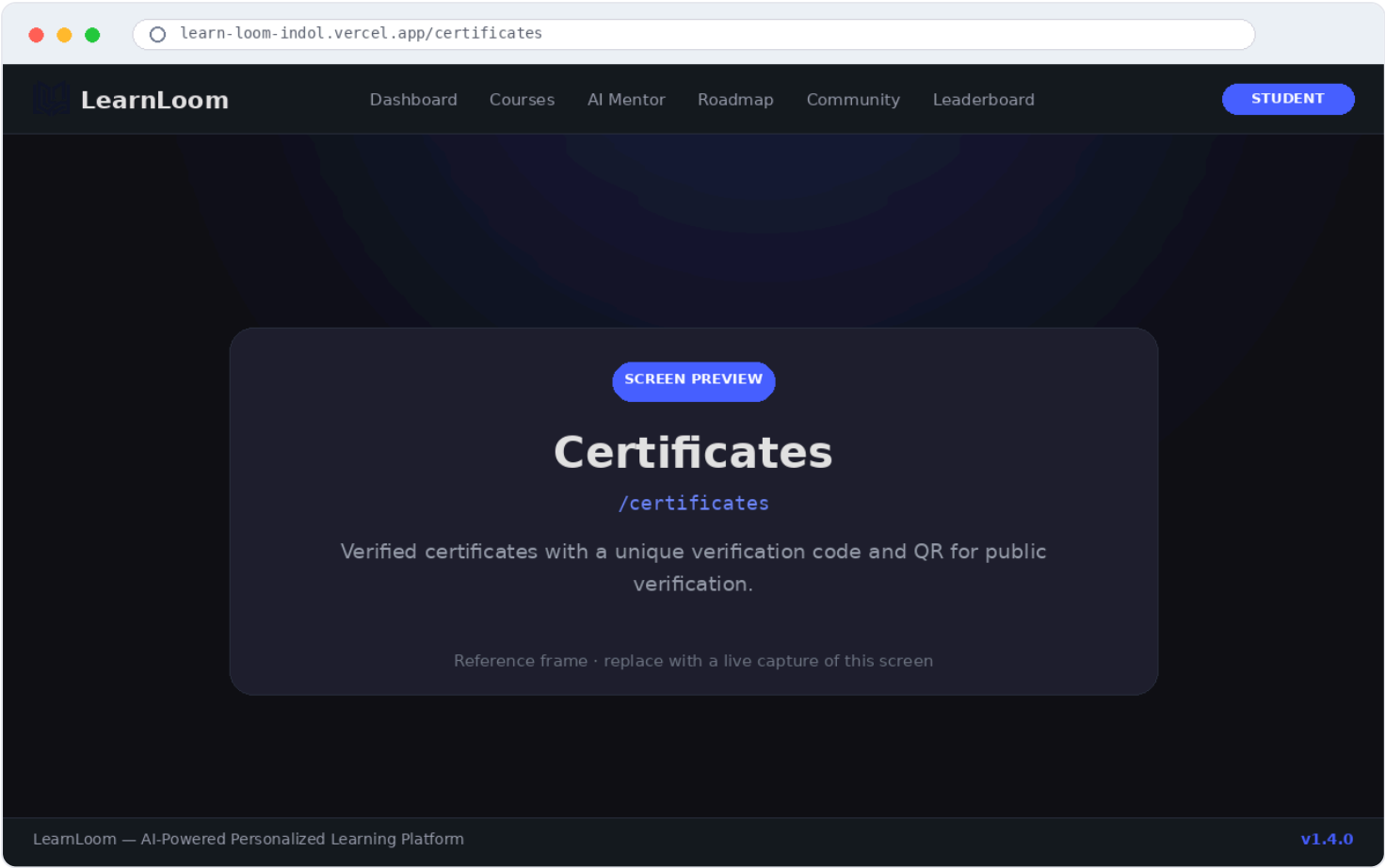
Practice, assessment & certification



Title: Coding Practice IDE · **Route:** `/coding` **Description:** Multi-language in-browser editor executing via Piston with test verdicts. **Purpose:** Hands-on practice with automated feedback.

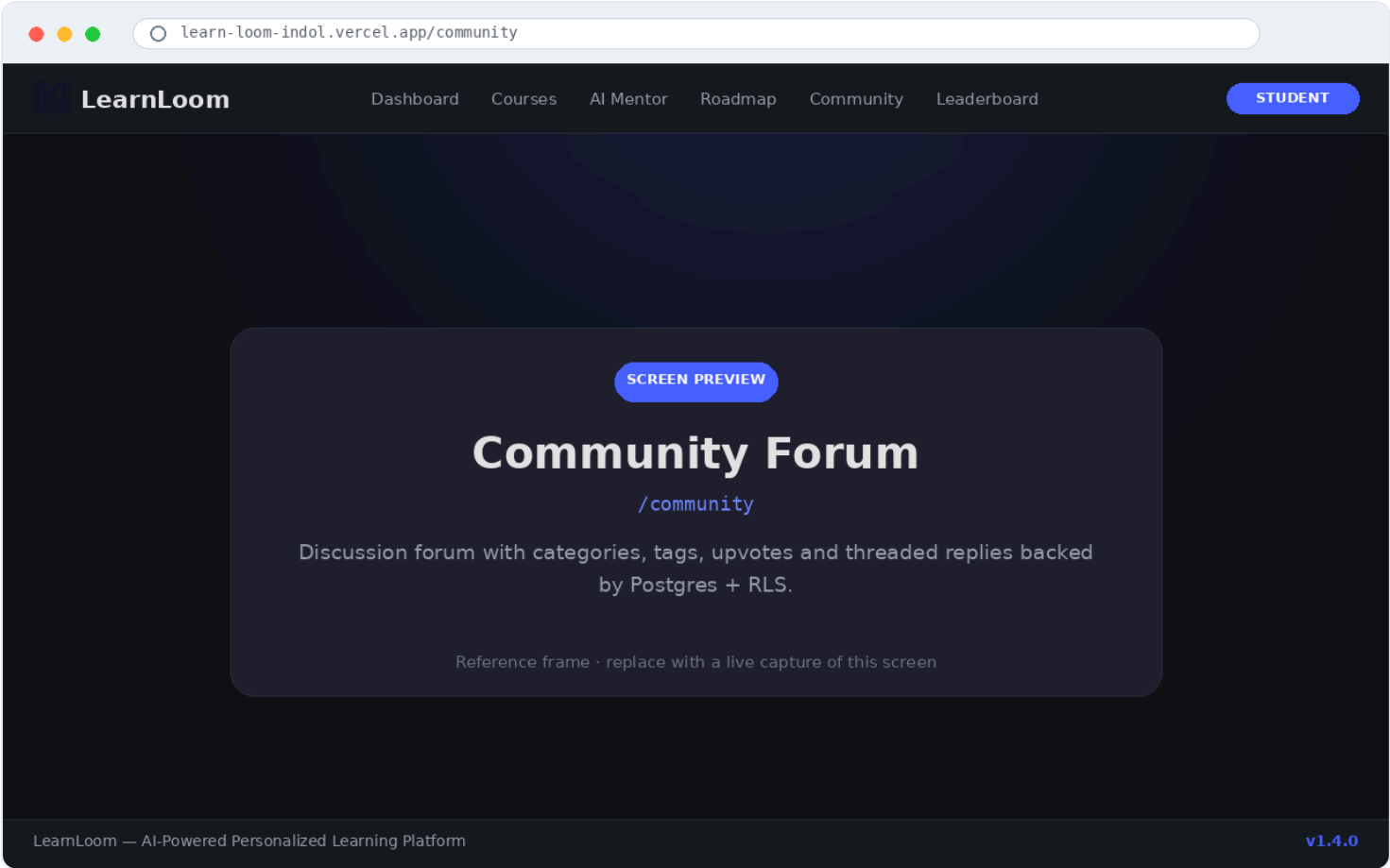


Title: Final Assessment · **Route:** `/courses/:id/assessment` **Description:** Proctor-simulated MCQ exam (75% pass) gating certification. **Purpose:** Verify mastery before any certificate is issued.

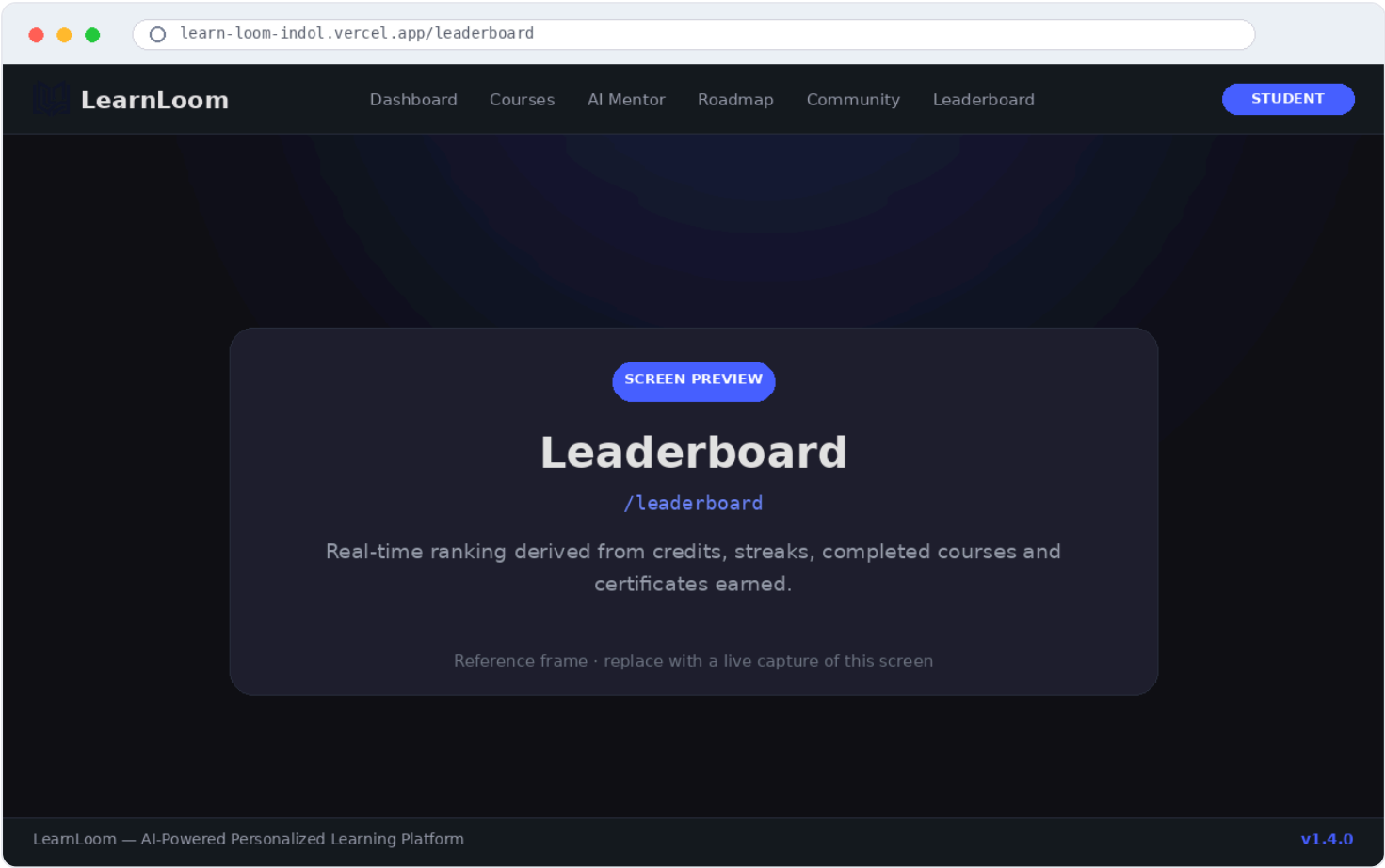


Title: Certificates · **Route:** `/certificates` **Description:** Verifiable credentials with unique codes and QR. **Purpose:** Provide credible, shareable proof of completion.

Community & engagement

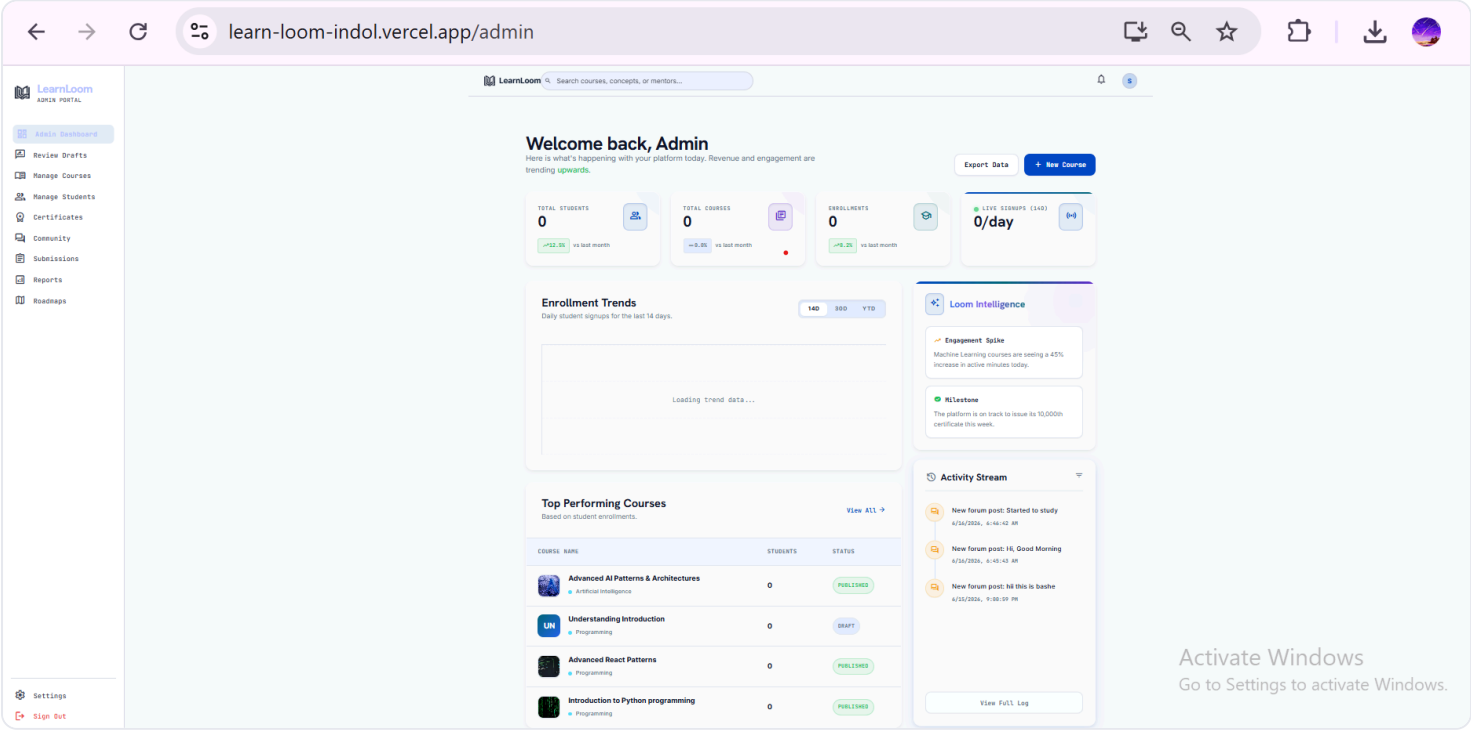


Title: Community Forum · **Route:** `/community` **Description:** Categorized, tagged, votable discussions with atomic vote counts. **Purpose:** Peer learning, support and engagement.

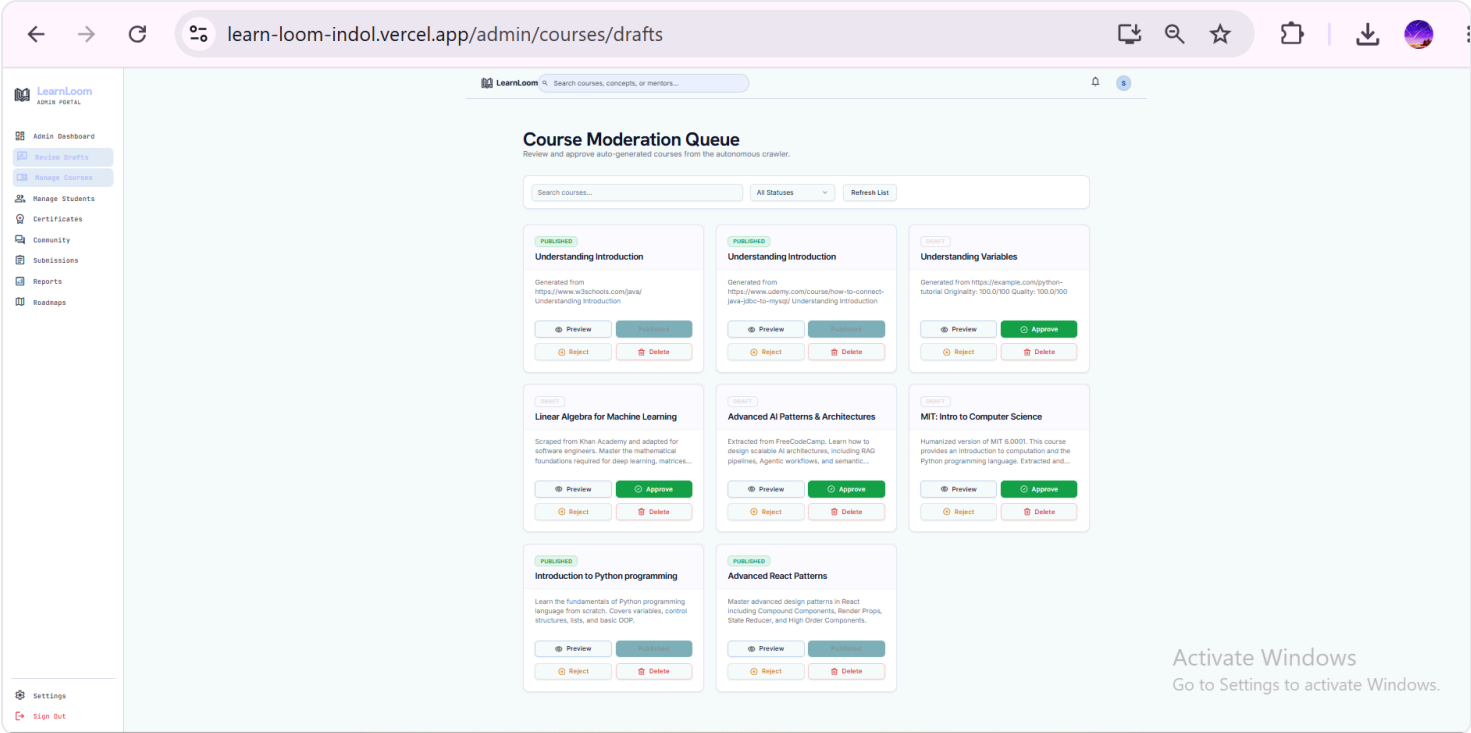


Title: Leaderboard · **Route:** `/leaderboard` **Description:** Real-time ranking by credits, streaks, completions and certificates. **Purpose:** Motivate sustained learning through friendly competition.

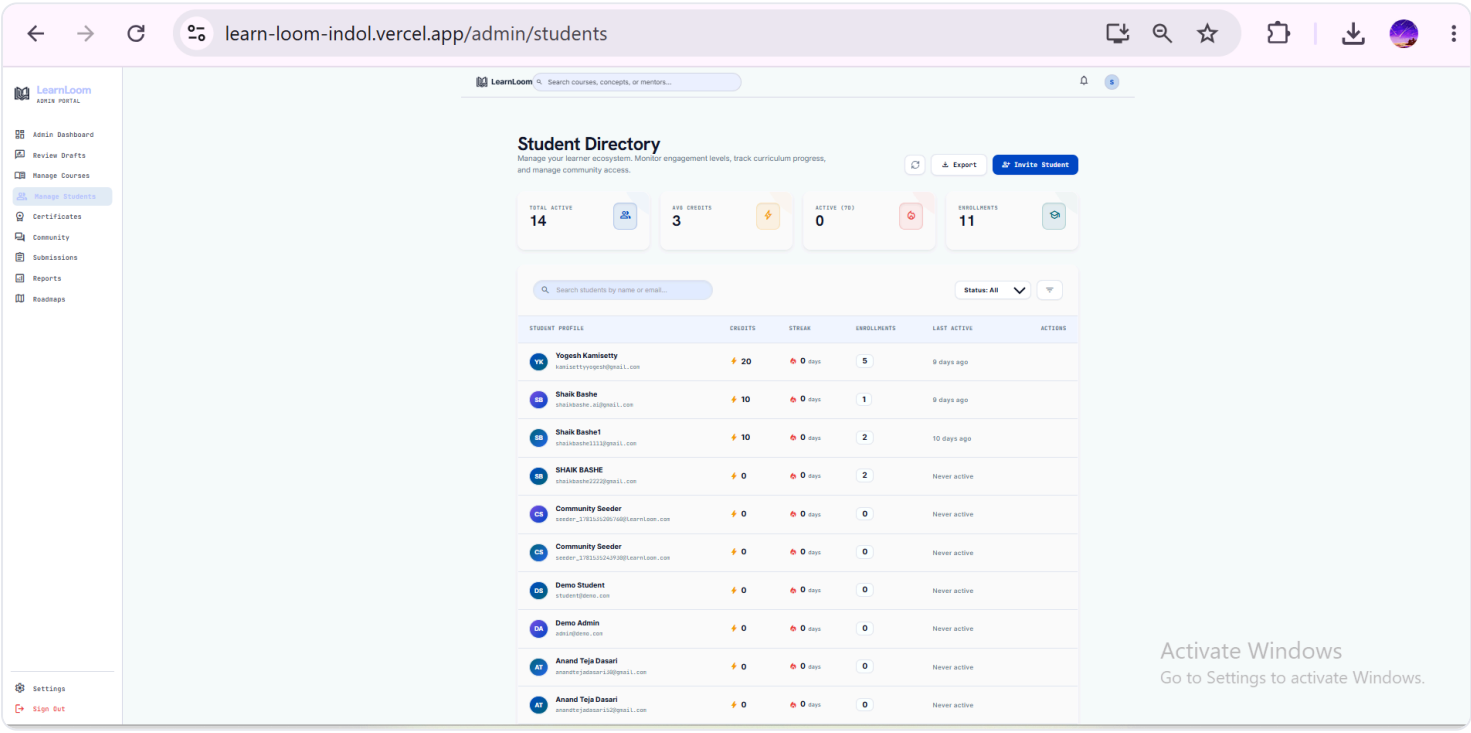
Administration



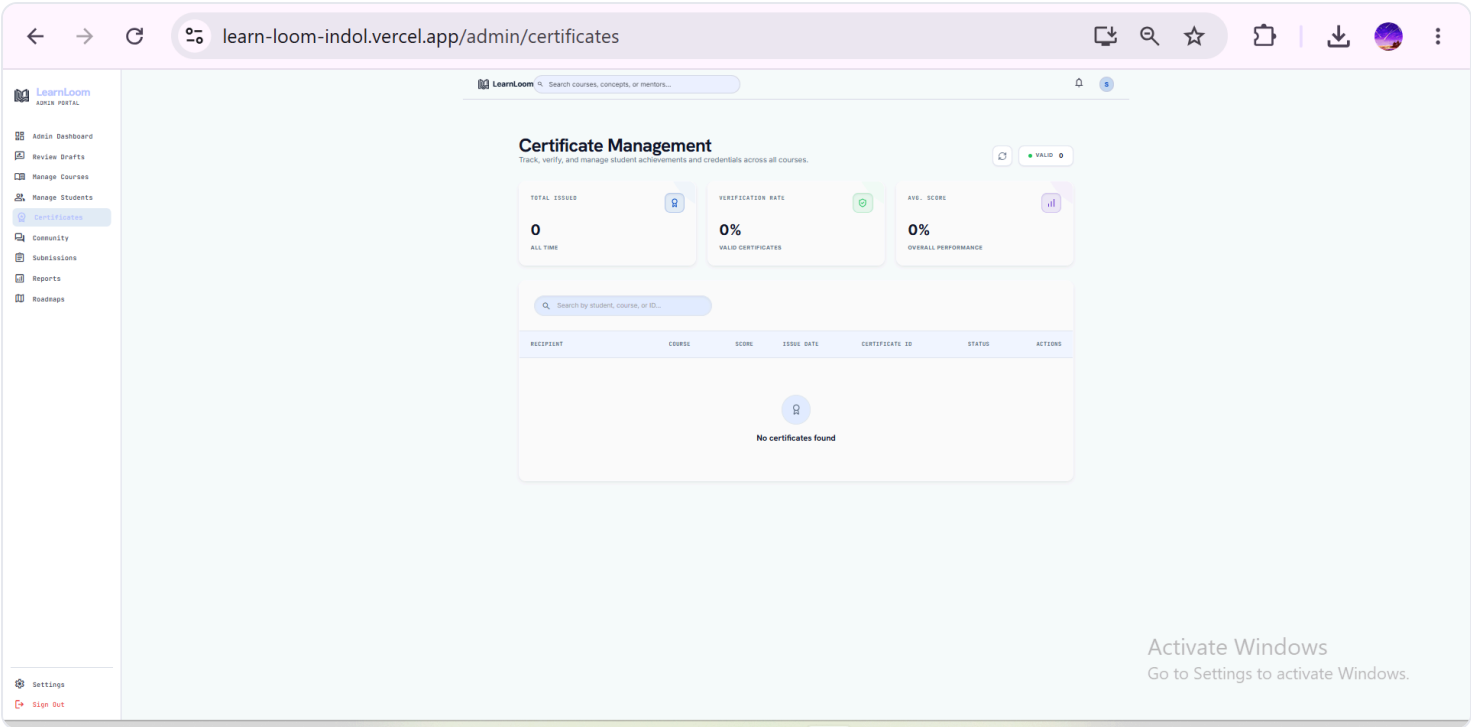
Title: Admin Dashboard · **Route:** `/admin` **Description:** Platform stats, enrolment trends, top courses and recent activity. **Purpose:** Operational overview for platform owners.



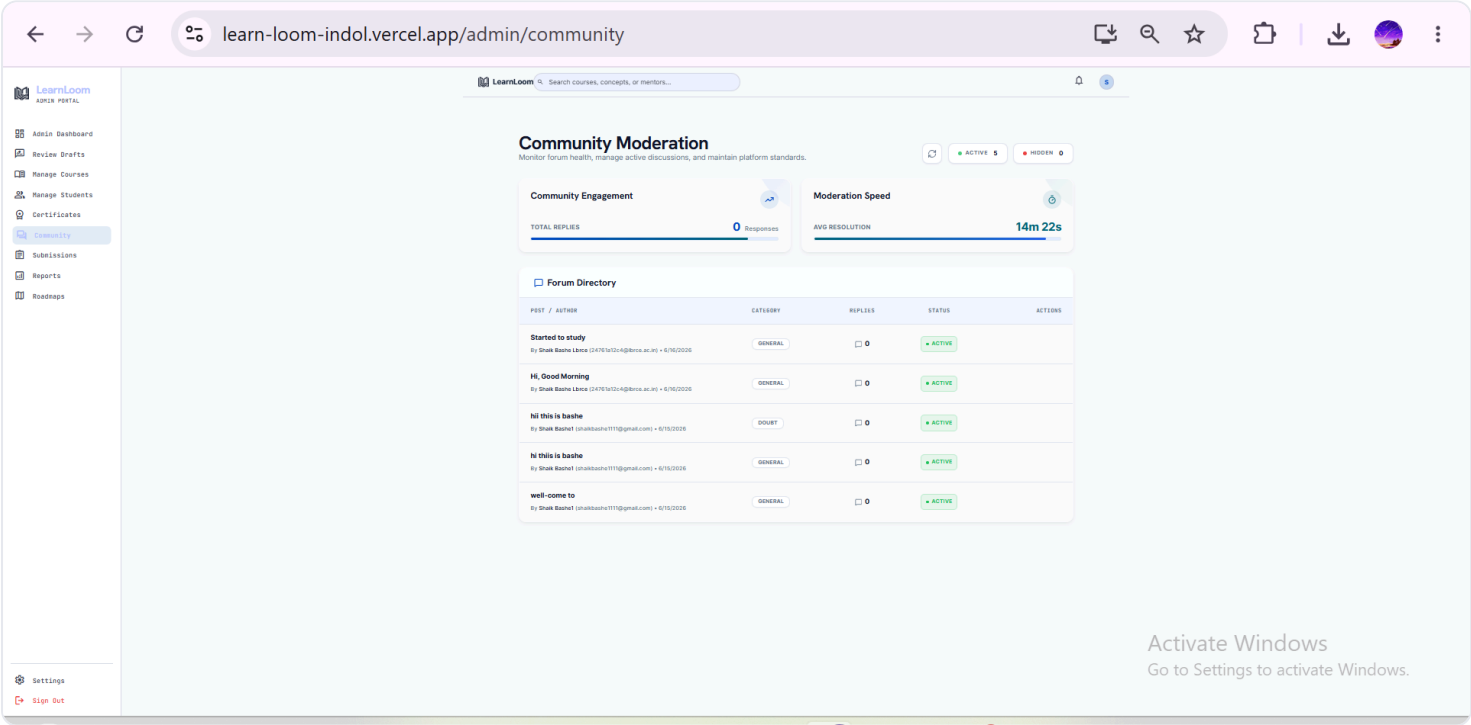
Title: Course Moderation Queue · **Route:** `/admin/courses/drafts` **Description:** Review, preview, approve or reject AI-generated course drafts from the crawler before publishing. **Purpose:** Content lifecycle management.



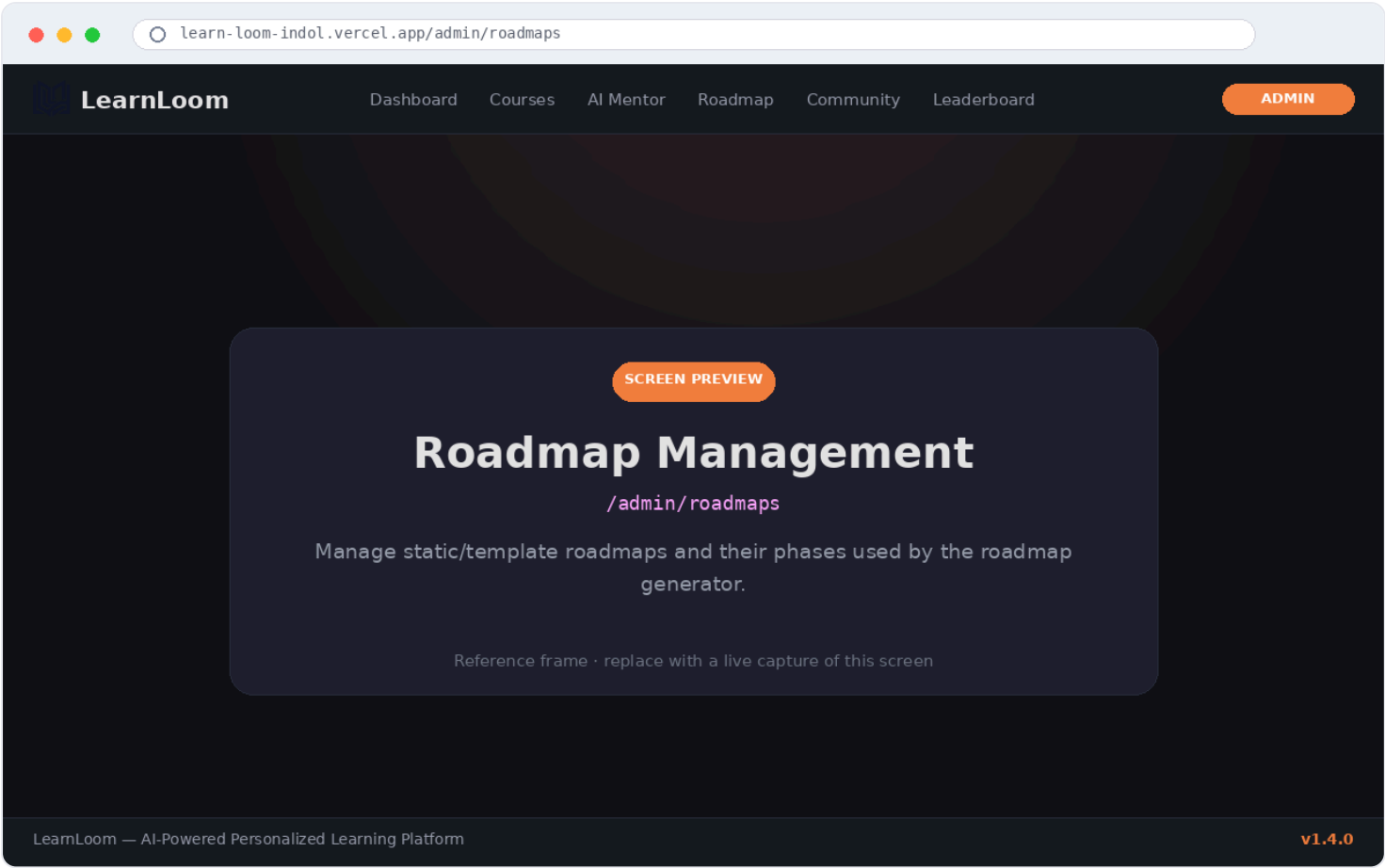
Title: Student Management · **Route:** `/admin/students` **Description:** Directory of registered students with account controls. **Purpose:** Manage the learner base.



Title: Certificate Management · **Route:** `/admin/certificates` **Description:** Audit, validate and revoke issued certificates. **Purpose:** Maintain credential integrity.



Title: Community Moderation · **Route:** `/admin/community` **Description:** Moderate posts/replies, pin threads, action flagged content. **Purpose:** Keep the community healthy and on-topic.



Title: Roadmap Management · **Route:** `/admin/roadmaps` **Description:** Manage roadmap templates and their phases. **Purpose:** Curate the guidance that powers the roadmap generator.

LearnLoom — AI-Powered Personalized Learning Platform Documentation generated from repository analysis · Version 1.4.0 · June 2026
Repository: `github.com/shaikbashe1/Learn-Loom` · Demo: `learn-loom-indol.vercel.app`